



FRANCISCO DUARTE LOURENÇO SIMÕES

B.Sc. in Computer Science

DEBUGGING STATECHARTS EXTENDED WITH CLASS DIAGRAMS

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

April, 2024

DEBUGGING STATECHARTS EXTENDED WITH CLASS DIAGRAMS

FRANCISCO DUARTE LOURENÇO SIMÕES

B.Sc. in Computer Science

Adviser: Prof. Dr. Miguel Carlos Pacheco Afonso Goulão

Associate Professor, NOVA University Lisbon

Co-adviser: Prof. Dr. Vasco Miguel Moreira do Amaral

Associate Professor, NOVA University Lisbon

Examination Committee

Chair: Nuno Manuel Ribeiro Preguiça

Full Professor, NOVA University Lisbon

Rapporteur: Dominique Blouin

Associate Professor, Telecom Paris

Adviser: Prof. Dr. Miguel Carlos Pacheco Afonso Goulão

Associate Professor, NOVA University Lisbon

Co-adviser: Prof. Dr. Vasco Miguel Moreira do Amaral

Associate Professor, NOVA University Lisbon

Debugging Statecharts extended with Class Diagrams

Copyright © Francisco Duarte Lourenço Simões, NOVA School of Science and Technology, NOVA University Lisbon.

The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

*To my family, friends and everyone who has been supporting me
for the last 23 years.*

ACKNOWLEDGEMENTS

I would like to start by deeply thanking my advisers, Prof. Miguel Goulão and Prof. Vasco Amaral, for all the guidance, support, availability, and flexibility they provided throughout the development of this dissertation. Without them, this project would not have been possible to execute. On a similar note, I would also like to thank our collaborators at the Universiteit Antwerpen for their guidance and support. I am also thankful to the NOVA SST University and the Informatics Department, DI, for the education, tools and experience they provided me with over the last 5 years.

Secondly, I would like to thank my family and friends (*M.S.*) for their help and support during my whole life. Without them, I would have never started this path and most definitely, I would not have finished it. So I am deeply thankful to all of them.

” *“Wir haben alles, was wir brauchen, Doch noch lange
nicht genug, Wir wollen mehr, mehr, mehr, mehr!”*

— **OK KID**, Deutschland

ABSTRACT

Model-Driven Development (MDD) is a software development methodology based on models which has been slowly becoming more relevant in the past decades. However, several barriers still stand in the way of a wider adoption of **MDD** methodologies and approaches, one of them being how lacklustre some of the currently available **MDD** tools still are.

One of the main weaknesses of the existing tools is the difficulty of debugging models and systems synthesised from those models. As it is well known in the software development world, testing and debugging are both fundamental so that quality software is produced.

In this work, we propose a debugger for models, with which it will be possible to debug models and model-generated applications from those models while always reasoning at the model level. The debugger will be integrated into a modelling and simulation open-source tool, in which it will be possible to model statecharts and class diagrams, through the **SCXML extended with Class Diagrams (SCCD)** formalism, which combines both.

We have implemented an open-source modelling tool that makes use of a powerful formalism and combines debugging and simulation, representing a leap forward towards a global improvement of **MDD** tools and their usability.

Keywords: Model-Driven Development, Debugger, Statecharts, Class Diagrams, Model level, **SCCD**

RESUMO

MDD (Model-Driven Development) é uma metodologia de desenvolvimento de software baseada em modelos que tem lentamente vindo a ganhar mais relevância nas últimas décadas. No entanto, existem ainda vários obstáculos que se opõem a uma maior adesão a estas metodologias e abordagens **MDD**, sendo um deles o quão pouco avançadas algumas destas ferramentas ainda são.

Uma das principais fraquezas das ferramentas existentes é a dificuldade de fazer *debug* de modelos e sistemas gerados a partir desses modelos. Como é dito no mundo do desenvolvimento de software, testar e fazer *debug* são duas atividades fundamentais na produção de software de qualidade.

Neste trabalho, propomos um *debugger* para modelos, com o qual será possível fazer *debug* de modelos e aplicações geradas a partir desses modelos, mantendo sempre a abstração de raciocínio ao nível dos modelos. O *debugger* será integrado numa ferramenta open-source de modelação e simulação, na qual será possível modelar statecharts e diagramas de classes, através do formalismo **SCCD** (State Chart extensible Markup Language), que combina ambos.

Desta forma, implementámos uma ferramenta de modelação open-source que utiliza um formalismo com forte poder de modelação, combinando funcionalidades de debugging e simulação, sendo este um importante passo no sentido de melhorar as ferramentas **MDD** e de as tornar mais utilizáveis.

Palavras-chave: *Model-Driven Development*, *Debugger*, *Statecharts*, Diagramas de Classe, Nível do Modelo, **SCCD**

CONTENTS

List of Figures	xi
List of Tables	xiii
Acronyms	xvi
1 Introduction	1
1.1 Context and Motivation	1
1.2 Objectives and Expected Contributions	2
1.3 Document Structure	3
2 Background	5
2.1 Model-Driven Engineering, Development and Architecture	5
2.2 Models and Metamodels	5
2.3 UML	6
2.3.1 Statecharts	6
2.3.2 Statecharts - Semantic Dimensions	8
2.3.3 Class Diagrams	9
2.4 Model-Driven Testing and Debugging	10
2.5 Feature Models	10
2.6 Model Transformations	11
3 Related Work	13
3.1 Feature Model - Statechart Modelling and Simulation Tools	13
3.2 Simulation Tools	17
3.2.1 Itemis Create	17
3.2.2 Sparx Enterprise Architect	17
3.2.3 Umple	18
3.2.4 MATLAB Stateflow	18
3.2.5 SCCD	18

3.2.6	Other Tools	19
3.3	Test Cases	19
3.3.1	Case One: Basic Statechart Features	19
3.3.2	Case Two: Multi-State Machine Modelling	21
3.3.3	Case Three: Dynamic Structure Modelling	21
3.4	Tool Comparison	22
3.5	SCCD - In Depth	23
3.6	Debugging in Model-Driven Development	28
4	SCCD Implementation	32
4.1	SCCD - Overall Architecture	32
4.2	SCCD - Compiler	34
4.2.1	Compiler Arguments	34
4.2.2	Platforms	35
4.2.3	Compilation Phases	36
4.3	SCCD - Runtime	43
4.4	Generated Code	44
4.5	Execution Flow	45
5	Debugger Implementation	48
5.1	Requirements for the Solution	48
5.2	Implementation Approach	49
5.3	Implemented Features	50
5.4	Changes to the Generated Code	57
5.5	Execution Flow	66
5.6	Usage Scenarios	68
6	Evaluation	75
6.1	Evaluation Planning	75
6.1.1	What is the evaluation's goal?	75
6.1.2	Who are the experimental units?	75
6.1.3	What were the experimental materials used?	76
6.1.4	What tasks do the testers have to perform?	78
6.1.5	What were the hypotheses, parameters and variables?	81
6.1.6	What evaluation design was used?	81
6.2	Execution	81
6.2.1	Procedure	81
6.2.2	Deviations	81
6.3	Analysis	82
6.3.1	Quantitative Analysis	82
6.3.2	Qualitative Analysis	87
6.4	Discussion of Results	88

6.4.1	Evaluation of Results and Implications	88
6.4.2	Threats to Validity	90
6.4.3	Inferences	90
6.4.4	Lessons Learned	91
7	Conclusion	92
7.1	Overview	92
7.2	Contributions	93
7.3	Future Work	93
7.3.1	Improvements	93
7.3.2	New Features	94
	Bibliography	96
	Annexes	
I	Tests	101

LIST OF FIGURES

1.1	Architecture of the SCCD Modelling, Simulation, Code Synthesis and Debug- ging Environment	3
2.1	Model-Driven Initiatives (taken from [2])	6
2.2	Example of a Statechart Diagram (taken from [48])	8
2.3	Example of a Class Diagram (taken from [9])	10
2.4	Model Transformation Terminology (taken from [40] and adapted from [52])	12
3.1	Statechart Modelling and Simulation Feature Model	13
3.2	Basic Statechart Features Feature Model	16
3.3	Debugging and Testing Mechanisms Feature Model	17
3.4	Test Case One: Basic Statechart Features	20
3.5	Test Case Two: Multi-State Machine Modelling	21
3.6	Test Case Three: Dynamic Structure Modelling	22
3.7	SCCD Metamodel	27
3.8	Model Debugging: Standard Approach (left) vs. Platform Independent Ap- proach (right) (adapted from [5])	29
3.9	Mappings between model layers from design to debugging (taken from [20])	30
3.10	Classic vs Model-level Debugging (taken from [24])	31
4.1	SCCD - Logical View as a State Diagram	32
4.2	SCCD - Process View as an Activity Diagram	33
4.3	SCCD - Development View as a Component Diagram	34
4.4	Generic Code Construction in Detail	39
4.5	Statechart Model Example	41
5.1	Statechart with Debugging Mechanisms Example	50
5.2	Output after entering <i>State_A</i>	67
5.3	Output after entering <i>State_Debug</i>	67
5.4	Help Menu	68
5.5	Basic Counter	68

5.6	Counter with Composite States	69
5.7	Counter with History State	69
5.8	Counter with Multiple States	70
5.9	Counter with Parallel States	71
5.10	Counter with Scripts	72
5.11	Phone	73
5.12	User Tests Task 5	74
6.1	SUS standard questionnaire; image taken from [7]	77
6.2	NASA-TLX standard questionnaire; image taken from [27]	78
6.3	Test Model 1	79
6.4	Test Model 2	80
6.5	Demographic Statistics	83
6.6	Average Ease in Understanding each Feature	84
6.7	Average Usefulness Score per Simulation Mode	85
6.8	Average Trace Log Score Evolution from Task 1 to Task 4	85
6.9	Answers to Task 5 Questions	86
6.10	Average Usefulness Score per Context Usage	86

LIST OF TABLES

3.1	Statechart Basic Features Tool Comparison	23
3.2	Statechart Modelling Advanced Features Tool Comparison Part 1	23
3.3	Statechart Modelling Advanced Features Tool Comparison Part 2	23
6.1	Goal-Question-Metric	75
6.2	Test Model 2 Detailed Event Chain	81
6.3	User Test Usability and Task Load Scores	82
6.4	Precision Statistics per Task	84
6.5	Recall Statistics per Task	84

LIST OF LISTINGS

4.1	Threads Platform to run an example model	35
4.2	Eventloop Platform to run an example model	36
4.3	Generic Code Construction Example	38
4.4	Generated Code Example	38
4.5	Statechart Structure Construction Code Example	41
4.6	Object Manager Association Management Example	43
4.7	Enter and Exit Functions Example	46
4.8	Output Of Example Model when Transition A to B occurs	46
4.9	Output Of Example Model when Transition A to D occurs	47
5.1	Timers Added in Different Branches	53
5.2	Breakpoint Specification File	54
5.3	Tracing Log Example	57
5.4	State Enter Function <i>State_A</i>	59
5.5	State Enter Function <i>State_A</i> First Branch	60
5.6	State Enter Function <i>State_A</i> Second Branch	61
5.7	State Enter Function <i>State_Debug</i>	61
5.8	State Enter Function <i>State_Final</i>	62
5.9	State Exit Function <i>State_A</i> (1)	62
5.10	State Exit Function <i>State_A</i> (2)	64
5.11	Process Time Transitions Function	64
5.12	Guard Function Example	66
6.1	Task 5 Trace Log Excerpt	89
I.1	Basic Counter	101
I.2	Counter with Composite States	101
I.3	Counter with History State	103
I.4	Counter with Multiple States	104
I.5	Counter with Parallel States	105
I.6	Counter with Scripts	106
I.7	Phone	107

I.8 User Tests Task 5	108
---------------------------------	-----

ACRONYMS

API	Application Programming Interface (<i>p. 38</i>)
AST	Abstract Syntax Tree (<i>p. 37</i>)
BSML	Big-Step Modelling Language (<i>p. 8</i>)
FODA	Feature-Oriented Domain Analysis (<i>p. 10</i>)
IDE	Integrated Development Environment (<i>pp. 2, 15, 16</i>)
MDA	Model-Driven Architecture (<i>pp. 5, 17</i>)
MDD	Model-Driven Development (<i>pp. vi, vii, 1–3, 5, 28, 30, 31</i>)
MDE	Model-Driven Engineering (<i>pp. 2, 5, 28</i>)
NSST	Nova School of Science and Technology (<i>p. 2</i>)
OMG	Object Management Group (<i>pp. 5, 6</i>)
PIM	Platform Independent Model (<i>p. 17</i>)
RTES	Real-Time Embedded Systems (<i>pp. 28, 30</i>)
SCCD	SCXML extended with Class Diagrams (<i>pp. vi, vii, xi, 2–4, 13, 18, 22, 23, 26, 27, 32–37, 40, 44, 48, 49, 54, 56, 68, 75, 81, 87, 89–93</i>)
SCXML	State Chart extensible Markup Language (<i>pp. vi, xvi, 2, 18, 68–71, 92</i>)
SUS	System Usability Scale (<i>pp. 76, 82, 87–91</i>)
TLX	Task Load Index (<i>pp. 76, 77, 82, 87, 88, 90</i>)
UML	Unified Modeling Language (<i>pp. 5, 6, 9, 18</i>)

INTRODUCTION

This chapter is an introduction to this project, in which we motivate the need for a model debugger, addressing the problem and providing some context to it. The expected contributions of this work are also presented, and the chapter finishes with the document structure.

1.1 Context and Motivation

Software systems grow in complexity every day, making the job of software engineers harder. Models allow us to understand a complex problem more easily [50], alongside its potential solutions through its high abstraction levels. [MDD](#) (Model-Driven Development) is an approach to software development in which high-level models are fundamental [49]. Several benefits would come with a broader adoption of these methodologies, such as development automation and speed-up and an overall better understandability of systems. A wider usage of these development methodologies could potentially enforce some standards across the global community, in which models would be used and understood by developers and engineers from diverse fields of study. However, it is not straightforward to create these standards because, for them to be used by larger communities, they have to possess more generic characteristics, making them less useful. So, to reach the ideal abstraction levels, more modelling languages must be created to attempt to solve domain-specific problems. When solving a problem, we may divide its complexity into 2 parts: essential complexity and accidental complexity. The essential complexity is inherent to the problem itself, so it is not possible to reduce it. However, choosing the correct modelling language may help us reduce the accidental complexity [4, 8].

Unfortunately, several reasons still stand in the way of [MDD](#) to be a more widely adopted approach in the software development field. Some of these reasons include (1) the fact there are already other methodologies currently implemented that work, and it could be troublesome to change the whole landscape completely, or (2) even uncertainties regarding the correctness in the construction of models [41].

But there is a particular reason for this refusal of a wider adoption of the [MDD](#), which will be addressed in this project: the immaturity of [MDD](#) tools, especially when compared with the traditional [Integrated Development Environment \(IDE\)](#)s. This is still palpable and visible in several tools, proving to be an obstacle that must be overcome. Therefore, it is important to develop these tools further, making their workflows simpler ensuring the correctness of models, and attempting to make them more appealing and usable overall. This is why this work focuses on the development of a debugger for a modelling environment. This modelling environment allows the modelling of systems using the [SCCD](#) ([SCXML](#) extended with Class Diagrams) formalism, which encapsulates both statecharts and class diagrams [44].

1.2 Objectives and Expected Contributions

This work is part of an international project called [SCCD Modelling, Simulation, Code Synthesis and Debugging Environment](#), a collaboration between [Nova School of Science and Technology \(NSST\)](#) and Universiteit Antwerpen, the group that developed [Model-Driven Engineering \(MDE\)](#) tools such as the [AToMPM](#) [57], which will result in a tool with modelling, debugging, simulation, and code generation features, using the [SCCD](#) formalism which will be explained in detail in sections 3.2.5 and 3.5, in which its capacities will be further analysed and highlighted.

The developed tool supports the following usage flow, according to the existing components in Figure 1.1: a model is created using the [SCCD Formalism](#), and functional code may be generated from it, making use of the *Modelling Environment* and the *Code Synthesis* components. The generated code may then be run, resulting in a working application. This application may be debugged in the *Debugger* component (yellow-coloured in the figure since it is the focus of this dissertation). The debugging process is visible at the model level while simulating the model's execution using the *Simulation Environment* component, making it easier to reason about the system's overall functioning and find the defects of the model and the system. Using the *Collaboration* component, the tool will also support collaboration mechanisms. All the interactions between the user and the tool should be done using the *User Interface* component.

The entire environment will eventually be bootstrapped into a single working tool, and an evaluation of the usability of the tool is also planned to be made as future work, even though it is outside of the scope of this particular dissertation. There currently already exists a demo for collaborative blended modelling available online [31], an extension of the previous work of Joeri Exelmans et al. [14].

To put the tool to test, it can be interesting to model some well-known modelling problems such as the Bouncing Balls [30] or the Rail-Car system [25]. Since these are famous modelling problems, it is fairly easy to find already-built models that may be considered correct. They are also important since they model systems using physical equations and dynamic structure modelling (objects may be created or deleted during

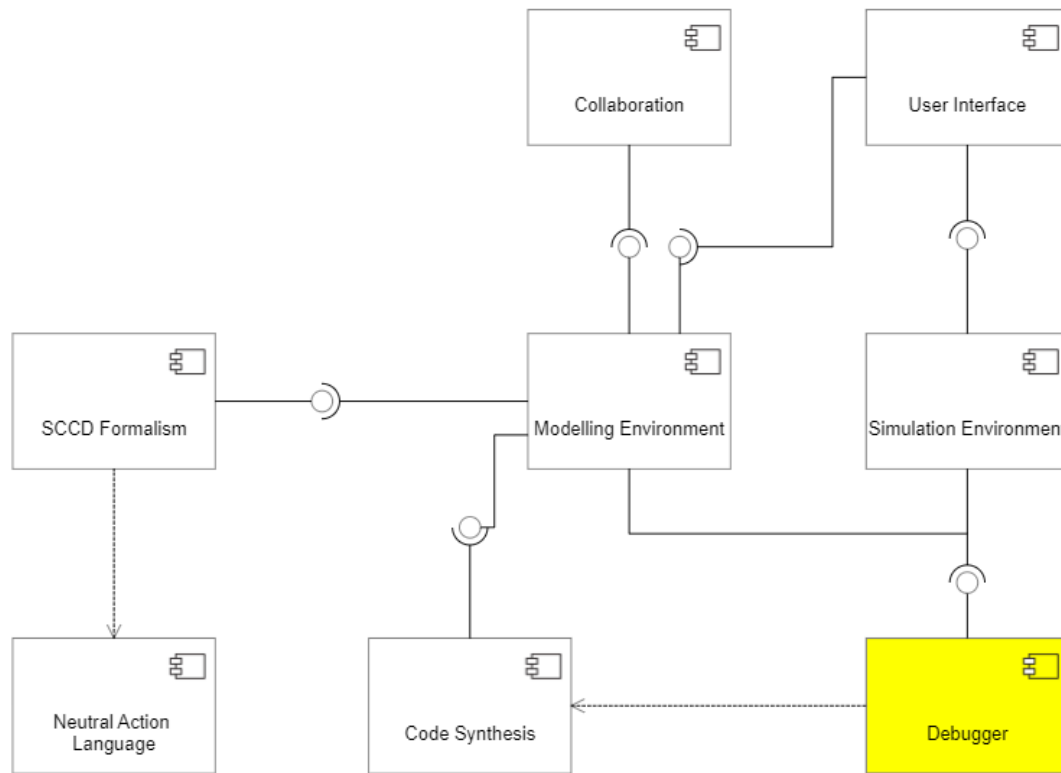


Figure 1.1: Architecture of the [SCCD](#) Modelling, Simulation, Code Synthesis and Debugging Environment

the execution) [56]. Ultimately, it would be interesting to model the [SCCD](#) modelling environment itself, but this already constitutes a more difficult challenge on the modeller side.

This tool is an open-source project, meaning it can serve as a basis for further improvements in [MDD](#) tools and future research in this growing field. Another advantage of our implementation and working methodology is that each tool component (such as the debugger, the modelling environment and so on) is independent, creating much-desired modularity, as each component may be updated without changing the entire implementation.

1.3 Document Structure

Chapter 1 - [Introduction](#): This is an introductory chapter in which the problems and motivations for this work are discussed, as well as a brief discussion about the proposed solutions.

Chapter 2 - [Background](#): This chapter provides the reader with some background concepts they should know before reading the remainder of this document.

Chapter 3 - [Related Work](#): This chapter discusses several tools and technologies related to

this work and positions this project within the current state-of-the-art landscape.

Chapter 4 - [SCCD Implementation](#): This chapter contains an in-depth explanation of the [SCCD](#)'s current implementation.

Chapter 5 - [Debugger Implementation](#): This chapter provides all the details regarding the new debugging features added to [SCCD](#) and their implementation.

Chapter 6 - [Evaluation](#): This chapter discusses all details regarding the evaluation that was made of the debugger, including user tests procedure and result analysis and discussion.

Chapter 7 - [Conclusion](#): This chapter discusses the conclusions that can be drawn from this project and outlines future work that can be implemented on top of this tool.

BACKGROUND

Before diving deeper into the subject, we should start by going over some basic concepts, such as [MDD](#) and some of its derivatives as well as going through some [Unified Modeling Language \(UML\)](#) diagrams that will be particularly important throughout this dissertation, such as statecharts and class diagrams.

2.1 Model-Driven Engineering, Development and Architecture

[MDE](#) is a software development methodology focused on creating models regarding the problems at hand. It is strongly connected to standards such as the [UML](#). As for [MDD](#) [3], it is a purely development-related subset of [MDE](#) that has grown worldwide as the programming languages abstraction levels have continuously risen. It allows developers to model the functionalities needed in a system and its overall architecture instead of pointing out every detail of its implementation.

A good example to explain [MDD](#) can be seen in how compilers handle issues like object allocation nowadays, which had to be manually programmed quite some years ago. Raising these abstraction levels lets us automate and look over these complex tasks, allowing us to try and solve more complex programming problems.

As for [Model-Driven Architecture \(MDA\)](#), it is the particular vision of [MDD](#) by [Object Management Group \(OMG\)](#), subject to its own standards, making it yet again a subset of [MDD](#). Figure 2.1 [2] shows the layered vision of the three previously mentioned model-driven initiatives.

2.2 Models and Metamodels

A model is an abstraction that makes it possible to make predictions or inferences about reality or a given System Under Study. A model should be based on an original phenomenon, it should be a selection of its properties, and it should be usable in its place with respect to some purpose.

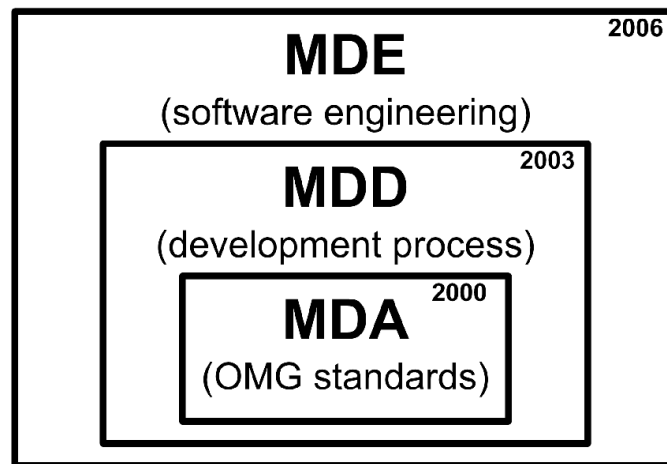


Figure 2.1: Model-Driven Initiatives (taken from [2])

The main motivations to build models are constructing systems, explaining systems to customers, code generation and simulation, among a few others. There are 2 types of models: token models and type models [35]. Token models focus on capturing more singular and concrete aspects of the original phenomenon, while type models take a more generic approach, capturing the universal aspects of the original phenomenon.

A metamodel is, by definition, a model of models. But it must be a type model of a type model because no transitive relationships may exist to reach a two-level detachment of the original phenomenon [34].

2.3 UML

The **UML** is a modelling notation language developed by Rational Software to standardise software design. It was based on 3 different notations: Booch, OMT and OOSE, combining concepts of all 3. It is a graphical language for visualising, specifying, constructing and documenting the artefacts of a software-intensive system. It offers a standard way to write a system's blueprints, as stated in [9]. It was adopted as a standard in 1997 when it was brought into **OMG**. A standard modelling way is essential to produce higher quality models and improve communication and understanding in team projects. **UML** is the response to the rise of increasingly complex systems. **UML** allows modelling through several different types of diagrams, but we are particularly interested in statechart diagrams and class diagrams in this project. Its most recent version is 2.5.1 [21].

2.3.1 Statecharts

Statechart diagrams are behaviour diagrams which were created in response to a major problem in the specification and design of large and complex reactive systems [26]. A reactive system is characterised by being mainly event-driven, reacting to internal and

external stimuli. Many examples of this type of system can be found around us, from telephones to web applications' user interfaces. The specification of reactive systems requires a clear and rigorous behavioural description to serve as the backbone of development, and statecharts provide a fitting formalism to specify this reactive behaviour. They are built using the following elements:

- **(1) State** - A state represents the mode a system is currently in. A state can raise events inside the system and perform computations. Only one state may be active at each time instance of the system's execution, except for orthogonal regions (which will be explained in further detail later in this section).
- **(2) Entry and Exit States** - The entry and exit states are simply the system's initial and final active states. They can also be placed inside composite states and orthogonal regions.
- **(3) History State** - A history state is a state which may only exist inside a composite state or an orthogonal region. It memorises the last active state before exiting the composite state or orthogonal region. When the system enters that region again, the first active state will be the last active state memorised by the history state. There are two types of history states: the deep history state, which remembers the deepest active state(s) and the shallow history state, which only remembers the immediate child's state.
- **(4) Composite State** - A composite state contains a set of sub-states, which can be regular states or composite states. These composite states allow the modeller to group behaviours and states that logically belong together.
- **(5) Transitions** - A transition connects 2 states. It may trigger if certain conditions are met (time conditions, event conditions, etc.), leading to a state change. A transition may also have no conditions, which means the source state will execute its tasks and immediately trigger the transition to the next state when it finishes performing its computations.
- **(6) Guards** - Guards are simply the names given to transitions that need to be under certain conditions to be triggered.
- **(7) Orthogonal Regions** - States can be combined orthogonally in orthogonal regions. An orthogonal region is a parallel region in which more than one state may be active simultaneously when that region is entered.

It is possible to observe an example of a statechart diagram with all these elements in Figure 2.2.

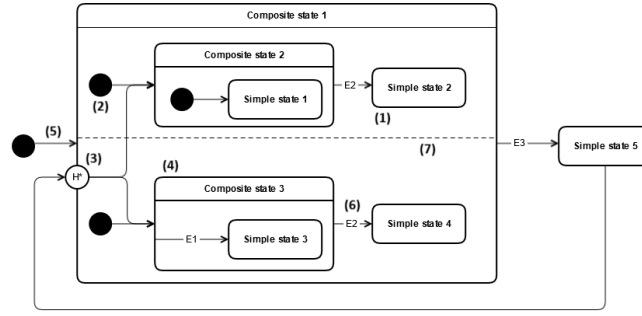


Figure 2.2: Example of a Statechart Diagram (taken from [48])

2.3.2 Statecharts - Semantic Dimensions

Statecharts may not follow the exact same standards regarding semantic decisions since the formalism does not have any precisely defined semantics [11, 12, 13]. Before identifying the different dimensions, we should note that statecharts may be considered a **Big-Step Modelling Language (BSML)**, meaning its semantics are controlled by several big-steps consisting of smaller computation steps. The different types of steps are defined based on the algorithm defining the simulation of a statechart, written by Simon Van Mierlo in [43]. We may then classify steps as:

- **Time-Steps** - A time-step executes one step of the system, which changes its simulated time value and consists of several big steps.
- **Big-Steps** - A big-step begins when an external event is received or a timeout occurs, and it consists of zero or more small steps. Since events may arrive at the same time, several big-steps may occur at the same time instant, forming a time-step.
- **Small-Steps** - A small-step consists of the execution of exactly one transition execution.
- **Combo-Steps** - Small-steps are grouped into combo-steps, so a combo-step executes small-steps until no transition is enabled by the current event from the environment or locally raised events in previous small-steps.

We may now take a look at the dimensions which may be considered (the most relevant ones are presented here), each one with different possible options:

- **Big-/Combo-Step Maximality** - This dimension controls the number of transitions that may be taken in a big-step (or in a combo-step). The possible options are:
 - **Take Many** - There is no real constraint, the big-step ends when no more transitions are enabled.
 - **Take One** - Only orthogonal (or concurrent) transitions may be taken within the same big step.

- **Syntactic** - This is basically the same as **Take One**, but transitions to states that are syntactically marked as *unstable* do not count and allow for transitions to be taken within the same step. A big-step becomes stable when a control state, which is syntactically designated to end the big-step is entered.
- **Input/Internal Event Lifeline** - This dimension specifies when and for how long, during a big-step, input and internal events are active. An input event may be **Present in First Small-Step**, **Present in First Combo-Step** or **Present in Whole** (in the entire big-step). An internal event may be **Present in Next Small-Step**, **Present in Next Combo-Step** and **Present in Remainder**.
- **Memory Protocol** - This dimension controls the versions of the variables that are read. The possible options for this dimension are: **Big-Step**, **Combo-Step** and **Small-Step**, which means values are read as they were at the start of the current step (big, combo or small).
- **Priority** - The priority dimension defines a priority ordering of transitions. During execution, if there is more than one transition that may be executed next, the transition with the highest priority will be executed. If there is no transition with the highest priority, then the executed transition is selected non-deterministically. There are some cases in which 2 transitions may be simultaneously enabled.
- **Concurrency** - Concurrency allows several transitions to be taken within the same small-step, as long as these are commutative (no priority needs to be defined between them).

2.3.3 Class Diagrams

Class Diagrams are yet another standard in the [UML](#). Unlike statechart diagrams, class diagrams are static structure diagrams. They describe the structure of a system by showing its classes, attributes, operations and the relationships between objects. A class diagram is a graphic view of the static structural model [9]. They are built using the following elements:

- **Classes** - A class describes a set of objects with similar structure, behaviour and relationships. A class may contain any number of attributes and methods (from 0 up) and may relate itself with any other classes. An object is an instance of a class, with its attributes initialised with concrete values.
- **Attributes** - An attribute represents a characteristic of the class it is in and can be of any data type (from Integer to another custom class type).
- **Operations** - An operation (or method) represents an operation that an instance of a class may perform. It may receive defined parameters of any data type and it may return a value of any defined data type.

- **Relationships** - A relationship connects two different classes to each other, stating the number of objects associated with the class to which it is related and indicating a name for the role of each class in that relationship.

It is possible to observe an example of a class diagram with all these elements in Figure 2.3.

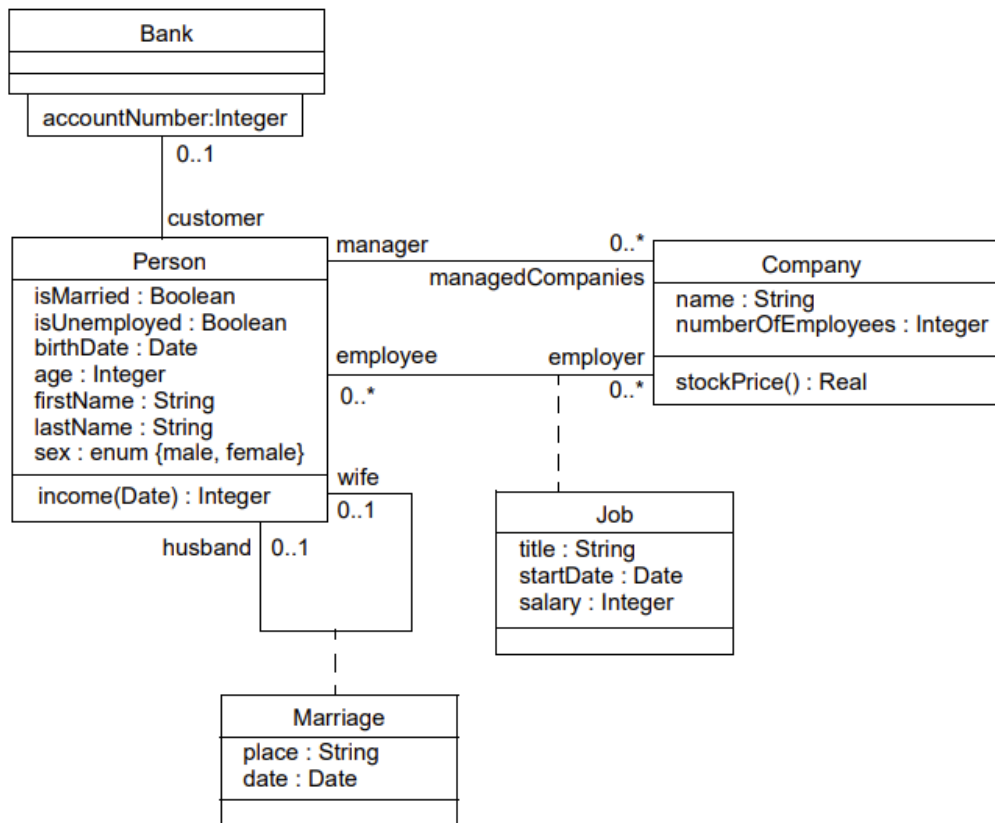


Figure 2.3: Example of a Class Diagram (taken from [9])

2.4 Model-Driven Testing and Debugging

Model-Driven Testing and Debugging are directly correlated to Model-Driven Simulation. Due to the increasing complexity of modern-day software systems, the necessity of early testing has risen in the development process. Models are usually abstractions of systems currently being modelled that have not been built yet and they provide a way to test and debug these systems without having to build them. This subject is explored more in-depth in section 3.6, exploring state-of-the-art model-driven debugging.

2.5 Feature Models

Feature Models were first introduced in the [Feature-Oriented Domain Analysis \(FODA\)](#) as an attempt to describe product requirements. Feature models include as many features

as feasible, as they represent the standard features of a family of systems in the domain and the relationships between them. The structural relationship is hierarchically built and contains mandatory, optional, and even alternative features or groups of features. The feature model serves as a communication method between users and developers as it is easily understandable for both. To the users, it shows all the available features they can have and choose for their product and to the developers, it shows a logical software architecture. Understanding feature models will be quite important since a feature model will be presented in section 3.1 with important features for modelling and simulation tools.

2.6 Model Transformations

A Model Transformation is the automatic manipulation of input models to produce output models which conform to a specification and have precise intents [40]. There are several approaches to perform model transformations, such as model-to-text and model-to-model. Model transformations may also be classified according to their intent. There are several categories inside which a few intents are grouped. The refinement category is particularly important for this work because it includes synthesis within itself. Synthesis refers to refinement where the output is an executable artefact expressed in a well-defined language format. It may refer to a model-to-code approach, which is one of the goals of the final product of this work (converting a statechart into a target language such as JavaScript or Python).

We may see in Figure 2.4 how model transformations are structured. This figure tells us that a model transformation receives as input a model (the input model) which conforms to a certain meta-model (the source meta-model) and outputs a model as well which conforms to another meta-model (the target meta-model), by executing a transformation specification that conforms to its transformation language. Both the source and the target meta-models conform to the same meta-model, making the latter a meta-meta-model. The transformation specification is defined at the level of both meta-models [40], while its execution operates at the model level.

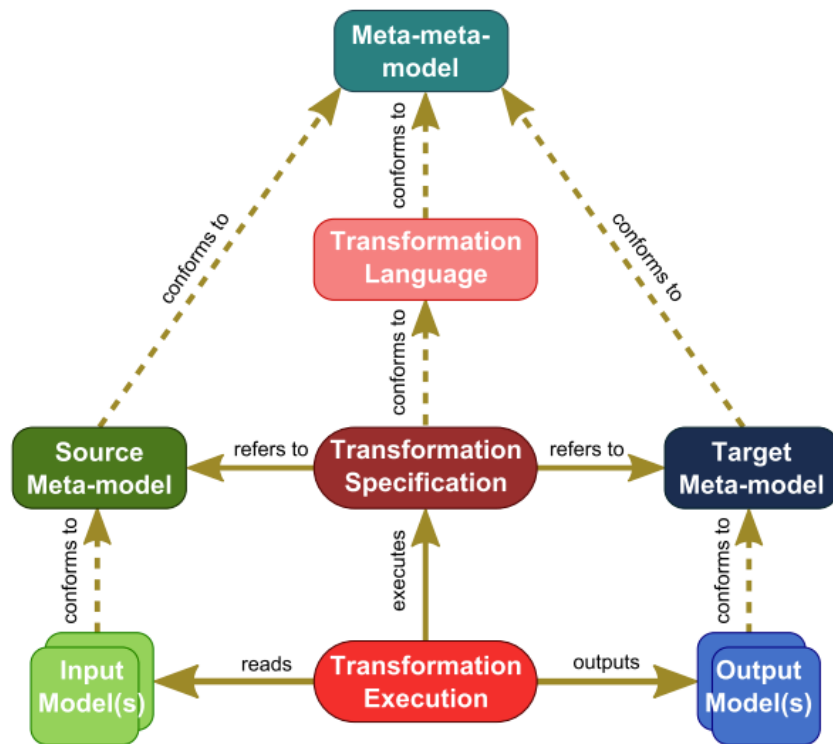


Figure 2.4: Model Transformation Terminology (taken from [40] and adapted from [52])

RELATED WORK

This chapter is structured in the following manner: in section 3.1, a feature model with useful and relevant features for statechart modelling and simulation tools is presented, focusing mainly on debugging features. Then, in section 3.2, several modelling and simulation tools are presented, analysed and tested with the test cases presented in section 3.3, including an overall comparison between all of them in section 3.4. After presenting the available tools, the SCCD formalism is presented in section 3.5 and a deeper dive into debugging in model-driven development is made and presented in section 3.6.

3.1 Feature Model - Statechart Modelling and Simulation Tools

In Figure 3.1, it is possible to see the feature model with the most relevant and desired features in a statechart modelling and simulation tool, whether mandatory or optional. Some of these features were inspired by [45].

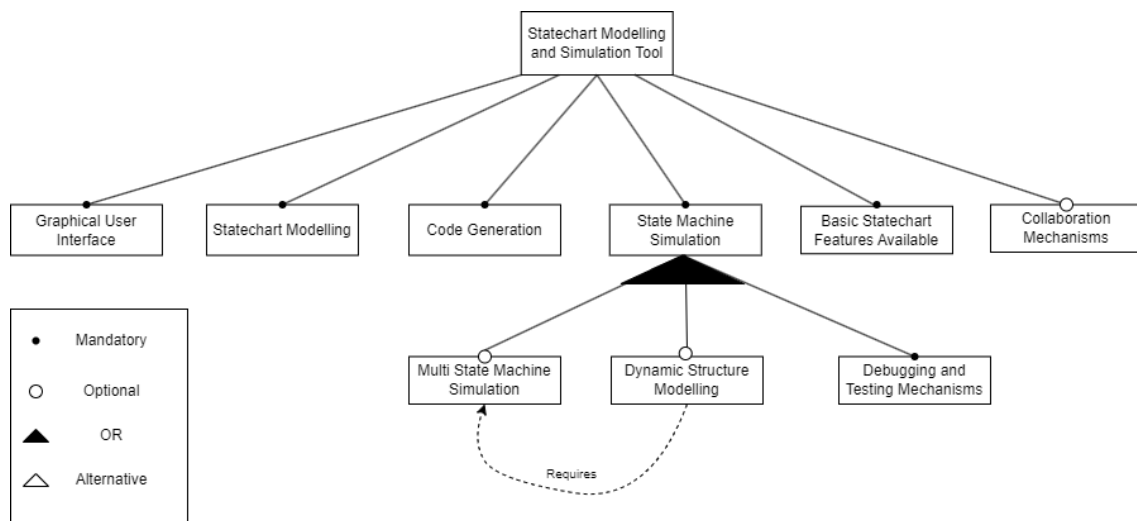


Figure 3.1: Statechart Modelling and Simulation Feature Model

The features included in the model are the following:

- **Graphical User Interface** - An important feature when modelling a visual model is to have a visual user interface. This interface should show the user the model being built, allow them to edit it visually, and reflect any change the user makes.
- **Statechart Modelling** - As we are dealing with modelling tools, being able to model statecharts is an essential feature which must be present in a modelling tool.
- **Code Generation** - Code generation means that, given a statechart model, the software can generate code according to the model's behaviour. This feature may vary depending on the software utilised because the user may want to generate code for different platforms or languages (from Java to Python).
- **State Machine Simulation** - This feature is one of the main objectives of this work. Its goal is to simulate a statechart and produce the output of its flow, given certain parameters that may be required.
- **Multi State Machine Simulation** - The term Multi-State Machine refers to multiple state machine objects that function independently of each other but can also communicate between them to form a more complex system. The multi-state machine system must be able to be simulated. The advantage of having multi-state machine interactions is that each state machine is simpler than having one single, very complex state machine, and the simulation applies to the real world, where each component exists independently.
- **Dynamic Structure Modelling** - Dynamic Structure Modelling may be defined as the creation of new state machines when required during the execution of a statechart simulation and its deletion.
- **Basic Statechart Features Available** - This feature refers to having the most basic elements used in statecharts available to model our diagrams, all of which are described in more detail in section 2.3.1. It should be noted that the analysed tools may include mechanisms which allow them to perform the roles of the required features but may not be marked as such if they do not respect the standard formalism. A separate small feature model was made with a list of all the basic elements. It is described in Figure 3.2.
- **Collaboration Mechanisms** - There are several possible ways in which collaboration mechanisms might be used. There may exist a shared repository where models are stored and from which they may be pulled or pushed (the same way git repositories work), or there might exist some live model sharing or screen sharing feature, allowing concurrent live modelling.
- **Debugging and Testing Mechanisms** - A debugging mechanism is extremely useful for a formalism like statecharts. The classic debugger functionalities provided in

programming IDEs may be similarly applied to diagrams like statecharts. A separate small feature model was made with a list of all the possible debugging mechanisms considered. The choice of these mechanisms was made based on Simon Van Mierlo's PhD thesis [43]. It is described in Figure 3.3. The following items describe those mechanisms in detail:

- **As-Fast-as-Possible Simulation** - In this simulation mode, as the name may suggest, there is no correlation between simulated time and real-time. Any reference to real-time will be ignored and run “as-fast-as-possible” as the hardware can manage until the end condition is satisfied. At the end of the execution, the user may evaluate the state in which the system ended [45].
- **Real-Time Simulation** - Unlike the previous item, simulation time is synchronised with the wall-clock time in this simulation mode. However, it does not necessarily need to follow the exact real-time. It may simply maintain a linear correlation with it by applying a scale factor which maintains the system's normal functioning but may increase or reduce its overall runtime duration [45].
- **Pause** - The pause operation resembles the common debuggers as it pauses the ongoing simulation and allows the user to inspect the system's current state, enabling other debugging operations such as stepping and state modifications.
- **Steps** - We may recognise different steps. As mentioned in subsection 2.3.2, we may consider big-steps, small-steps, combo-steps and time-steps. But, from a debugging point of view, we may also identify other types of steps as an analogy with the regular code debuggers' approach. These steps were chosen based on Sadaf Mustafiz et al.'s paper about statechart simulation environments [45]. We may then classify the steps as follows:
 - * **Step Into** - *Step into* refers, in code, to the steps taken through instructions, including child functions into which the step goes. In statecharts, an analogy may be made with composite states. Stepping into means stepping into the child state.
 - * **Step Over** - *Step over* is the counterpart of *step into*. If, for example, the active state is a composite state with child states, then the *step over* will not go into the child states but will, instead, remain at its current level. The child states will still be executed as usual, but their details will be hidden from the user.
 - * **Step Out** - We may think of *step out* as a way to undo *step into*. Being at a certain level of child states, stepping out will bring the visible, active state to the level above (to the parent state of the currently active state).
- **God Event** - A god event simply allows the user to change some value of a state variable of the model's memory.

- **Event Injection** - The user may introduce an event during the model’s execution at the current time.
- **Breakpoints** - Breakpoints are a well-known debugging feature present in most IDEs. As for statecharts, a breakpoint will stop its execution at a user-defined scope, such as a particular state. We may identify three different types of breakpoints:
 - * **State** - A breakpoint may be set on a particular state, so the execution pauses when that state is reached.
 - * **Store** - A breakpoint may specify a condition that needs to be met for the execution to pause.
 - * **Time** - A breakpoint may specify a particular simulation time so that when that point in time is reached, the execution is paused.
- **Execution States** - We may also define four different execution states in our debugging environment based on [45]:
 - * **Ready** - It is simply the state in which the environment is by default when it is not in the middle of an execution.
 - * **Running** - From the moment the statechart simulation is started, the *running* state is entered. We may run a **Real-Time Simulation** or a **As-Fast-as-Possible Simulation** while in this state.
 - * **Pause** - The execution may also be paused while it is in a *running* state. When in a *pause* state, it is possible to debug through steps and inspect states and variables manually.
 - * **Stopped** - When the execution ends, whether manually or automatically, it enters the *stopped* state. A timed reset event is then triggered to reset the environment, returning it to its initial state, *ready*.

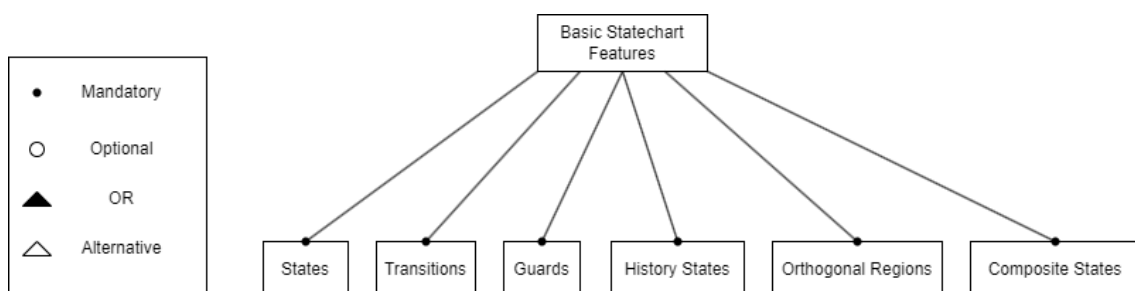


Figure 3.2: Basic Statechart Features Feature Model

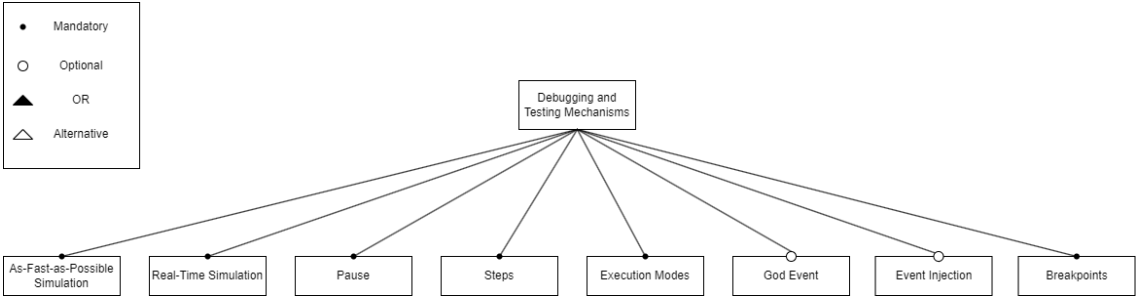


Figure 3.3: Debugging and Testing Mechanisms Feature Model

3.2 Simulation Tools

3.2.1 Itemis Create

Itemis [1], previously known as Yakindu, is a commercial statechart tool which is used to create and model statechart systems through a graphical user interface. Itemis provides most of the features mentioned in the feature model. It provides the possibility to directly simulate the system being modelled (both state machine and multi-state machine systems) as a regular simulation, as a pre-configured simulation or even with the aid of breakpoints and snapshots, which contain every information about the state of the simulation at a certain point in time.

It allows code generation from the modelled state machine system and the integration of code in models.

Itemis does not directly support dynamic structure modelling. Still, it allows the system to simulate some dynamic properties through the usage of enter and exit methods, although it is a non-optimal simulation of such dynamism. Communication between some parts of the system may also not be straightforward as event routing highly restricts the arguments, and connections are unidirectional (from parent to child) [10].

3.2.2 Sparx Enterprise Architect

Enterprise Architect [54] is an integrated modelling platform developed by Sparx Systems for professional corporate usage. It allows the users to build robust and maintainable systems while benefiting from high performance (fast load of extremely large models) and effective global collaboration mechanisms. It provides the tools to build several different types of models, including statecharts. It also includes simulation mechanisms for several types of models and provides features such as breakpoints to help control the simulations.

It provides both generation and reverse engineering of source code for many programming languages, including C, Java, and Python, among many others. It supports MDA as a single Platform Independent Model (PIM) can generate code for several different languages. Enterprise Architect also grants debugging mechanisms for models and generated code (for some of the available languages), converting executions into visual diagrams.

3.2.3 Umple

Umple [36] is an open-source modelling tool and programming language family which enables Model-Oriented Programming. It adds UML-derived abstractions, such as attributes and state machines, to object-oriented programming languages such as Java, C++ or PHP. It also allows to create state machine diagrams and class diagrams textually. Any Java, C++ or PHP projects can be adapted to use Umple, resulting in smaller and better readable code. Umple works online in a browser, as an Eclipse plugin, and as a standalone command-line Jar.

3.2.4 MATLAB Stateflow

Stateflow [42] is a modelling tool developed by MathWorks which provides its graphical language combining mainly state machine diagrams and flow charts. It is possible to design and develop many different systems using Stateflow, modelling combinatorial and sequential decision logic that can be simulated as a block within a Simulink (modelling and simulation tool also developed by MathWorks) model or executed as an object in MATLAB. It provides executing and debugging mechanisms for the created models, and it distinguishes itself for what it can do alongside the MATLAB language: Stateflow charts may use the capabilities of MATLAB, and they may be used as MATLAB objects in applications which require state machine and timing logic.

Alongside other tools developed by Mathworks, such as the previously mentioned Simulink, it allows code generation in some languages, such as C and C++.

3.2.5 SCCD

SCCD [44] is a unique tool which makes use of a formalism of its own. It links 2 well-known modelling formalisms: Statecharts and Class Diagrams, creating models which are different from the two model types previously mentioned. It provides its users with the ability to model complex, timed, autonomous, reactive, and dynamic-structure systems. SCCD has a syntax in an XML-format based on the W3C SCXML recommendation. Models may be compiled into several programming languages (Python, JavaScript and C#) and several runtime platforms implemented in those languages.

SCCD is quite different from the other tools, given that it does not actually have a modelling environment, it simply possesses a different modelling formalism which allows for the generation of code based on the provided models. In contrast, the other tools include a graphical interface and such features. To model with SCCD, one must use its XML-format syntax. However, SCCD has been compared to Itemis, which has been proven to be a more robust modelling formalism than the ones used on the other tools [10]. More about SCCD in section 3.5.

3.2.6 Other Tools

Aside from the tools listed and described above, other tools were also explored and analysed, such as Eclipse Papyrus [16], Visual Paradigm [38] or Cameo Systems Modeler [53], all modelling environments. These other tools were not analysed in detail since they did not entail different or relevant features from the other tools, as they also specialise in other types of models, particularly Visual Paradigm, which offers a wide variety of different diagrams.

3.3 Test Cases

This section will present some statechart test cases, including the diagram used and the tested features. All the models presented were built using Itemis Create.

3.3.1 Case One: Basic Statechart Features

To test the basic statechart features, which include states, transitions, guards, history states, orthogonal regions and composite states, it will be determined if it is possible to model the statechart model in Figure 3.4 using each tool.

This test case simulates the behaviour of a phone at a very simple and highly abstract level. The initial state of the phone is a sub-state in which it is powered off. Inside the *poweredOff* state, there are two possible states: *NotCharging* and *Charging*, which represent the state of the phone being plugged into its battery charger or not. Both these states possess two different events: *unplugCharger* (on the *Charging* state) or *plugCharger* (on the *NotCharging* state) and *pressPowerButton*, which will only happen if the battery is at a minimum value (defined as 10 in this model). Activating this *pressPowerButton* event successfully will move the current state to *PoweredOn*, representing the phone being powered on. Inside the *PoweredOn* state, there is an orthogonal region containing two possible concurrent states: *Charge* and *Apps*. While the *Charge* state controls the phone's energy settings the same way we have observed in the *PoweredOff* state, the *Apps* state introduces the phone's regular functionalities (reduced to a simple architecture of 2 applications A and B and a homescreen). This orthogonal region represents the state of the phone while it is charging and using application A, for example. In the *Apps* state, the initial state is the *Homescreen* if it is the first time the *PoweredOn* state is entered. Otherwise, the history state will decide the first active state. If the phone was turned off while using *App A*, then when it is turned on again, its first active state will be *App A* once again.

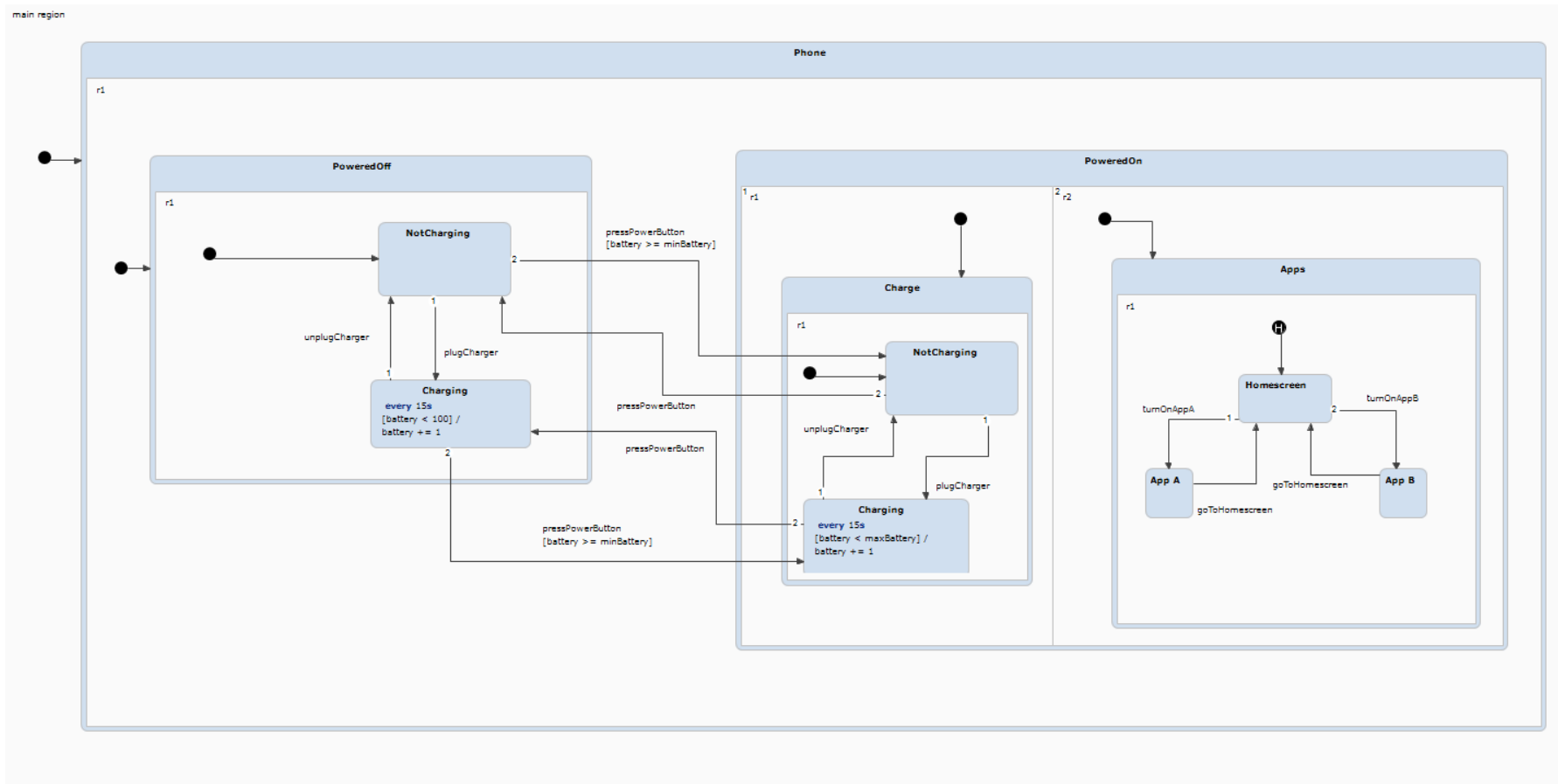


Figure 3.4: Test Case One: Basic Statechart Features

The results of the first test comparing the selected tools are displayed in Table 3.1.

3.3.2 Case Two: Multi-State Machine Modelling

This test case intends to check whether the tested tool allows for two or more different state machines to communicate with each other. This behaviour should be visible in the simulation. The built test case is very simple, and it is just proof of concept. It is visible in Figure 3.5.

This test case does not truly model any existing system. It simply shows the communication between 2 different state machines. In Figure 3.5, the left statechart only has one state with two variables: *battery*, which is a simple integer and *changer*, which is of type Changer. The Changer statechart is represented on the right-hand side of the figure, and it only has two states. The active state on the Changer statechart is changed when the event *changeState* is triggered. This may only happen through an outside source because both states of the Changer statechart are empty and do not trigger any events. The behaviour of the left statechart is the following: every second, the battery variable is incremented by 1 (it starts at 0), and when it is equal to 10, an event is raised. That event is *changeState*, which is defined on the Changer statechart. The *changer* variable, which is an active statechart (because it was entered in the beginning), will change its state every second (from *StateA* to *StateB*) because from the moment the battery variable is equal to 10, every second the *changeState* event will be raised.

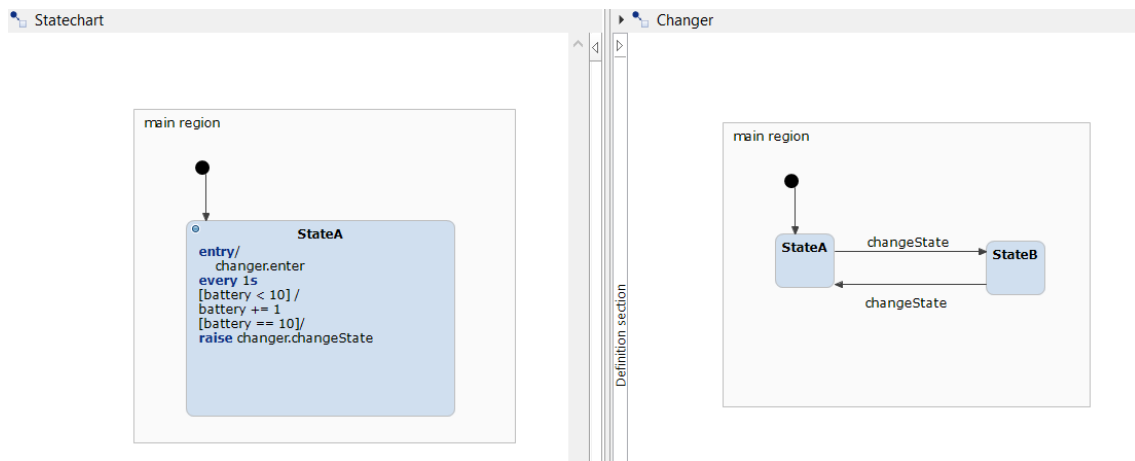


Figure 3.5: Test Case Two: Multi-State Machine Modelling

3.3.3 Case Three: Dynamic Structure Modelling

The third and last test case aims to determine whether the tested tool can dynamically create or delete other state machines as it progresses through its execution. This implicitly requires multi-state machine modelling and simulation to be available features. The test case is visible in Figure 3.6.

This is a more complex feature, and the only one Itemis truly fails to achieve. It allows for the dynamic instantiation of state machine objects, which consequently means it can simulate dynamic properties and behaviour. However, it does not truly possess the ability to dynamically create state machines, which is the objective of this feature. On the other hand, [SCCD](#) provides the ability to dynamically create new instances of an object through its object manager. These results are based on Sylvain Elias' thesis results [10].

The test case presented in Figure 3.6 does not truly represent pure dynamic structure modelling for the above reasons. However, the simulation of dynamic properties is still presented here. The test case is similar to the one in 3.3.2. The *changer* variable enters its active state using the Itemis defined primitive `enter()`, which simulates the dynamic instantiation of the variable (not its creation). After 10 seconds, the *changeState* event is raised and 1 second later, the statechart on the left-hand side of Figure 3.6 enters its stateB, where it activates the `exit()` Itemis primitive on the *changer* variable, which should truly be a deletion in a purely dynamic state. Still, it simply leaves the *changer* in an inactive state.

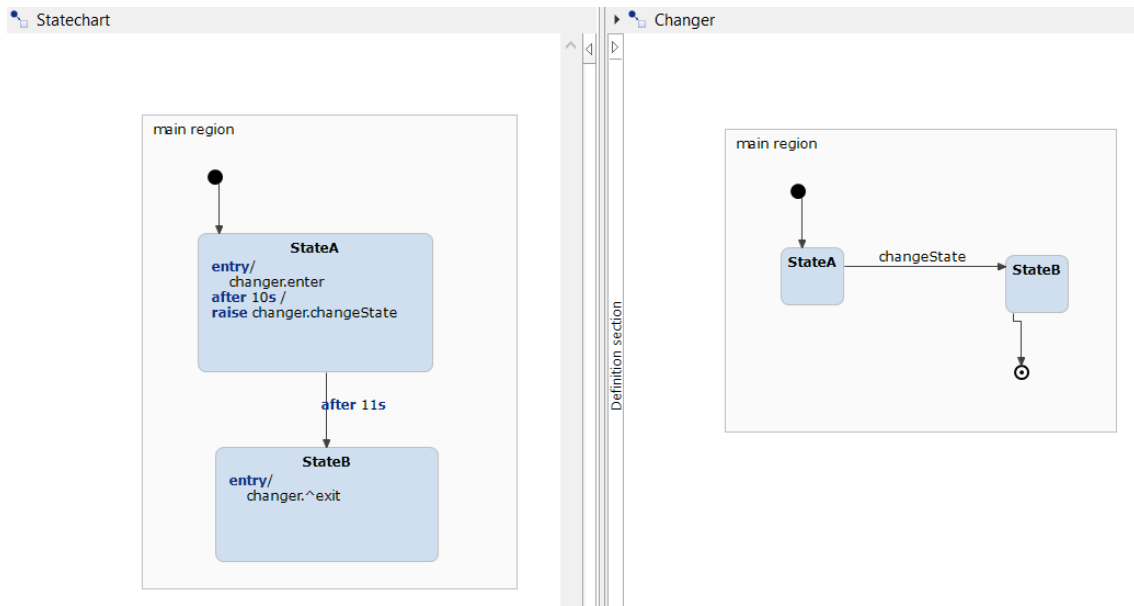


Figure 3.6: Test Case Three: Dynamic Structure Modelling

3.4 Tool Comparison

This section displays the tables comparing the previously mentioned tools regarding the selected features. It is possible to see the comparison regarding the basic features of statecharts in Table 3.1. In Tables 3.2 and 3.3, it is possible to see the overall comparison between more generic features regarding statechart modelling.

After examining the tables presented, the reader will notice that [SCCD](#) is the only formalism that can truly engage in dynamic structure modelling. However, it lacks debugging mechanisms, and this is where this project comes into play as it will focus on

Table 3.1: Statechart Basic Features Tool Comparison

Tool	States	Transitions	Guards	History States	Orthogonal Regions	Composite States
Itemis	X	X	X	X	X	X
Sparx	X	X	X	X	X	X
Umple	X	X	X	X		X
Stateflow	X	X	X	X		X
SCCD	X	X	X	X	X	X

Table 3.2: Statechart Modelling Advanced Features Tool Comparison Part 1

Tool	Graphical Interface	Statechart Modelling	Code Generation	Debugging Mechanisms	Collaboration Mechanisms
Itemis	X	X	X	X	
Sparx	X	X	X	X	
Umple	X	X	X		
Stateflow	X	X	X	X	X
SCCD		X	X	O	

Table 3.3: Statechart Modelling Advanced Features Tool Comparison Part 2

Tool	State Machine Simulation	Multi State Machine Simulation	Dynamic Structure Modelling
Itemis	X	X	
Sparx	X		
Umple			
Stateflow	X		
SCCD	X	X	X

developing these exact debugging mechanisms (marked as a circle on Table 3.2 precisely because it will be the focus of this work) setting the final bootstrapped tool apart from the other tools available. **SCCD** also provides precise and defined semantics, unlike Itemis, which means its results may differ when models are executed on different platforms.

3.5 SCCD - In Depth

Since **SCCD** will be the used formalism in this work, we will look deeper at its syntax and how to use it. An **SCCD** model consists of several classes linked between them through associations, while each class has an associated statechart which defines its behaviour at runtime. There is a default class of which an instance is created and started when the system is run. Instances have the ability to create, start, and delete instances of classes, as well as create and delete instances of associations. The following element descriptions were adapted from the official **SCCD** documentation [44].

An **SCCD** model has a few top-level elements (in its **SCCD**(XML)) notation:

- **<diagram>** : This is the top-level element of an **SCCD** model, and it has two attributes:

name, which specifies the name of the diagram and *author*, which specifies the author of the model.

- **<description>** : This element contains a description of the diagram, and it will appear as a comment at the top of the compiled file. It is a child element of **<diagram>**.
- **<top>** : This element may be used to import additional library modules which can be used by the classes modelled in the diagram. It is a child element of **<diagram>**.
- **<inport> and <outport>** : Both these elements model communication channels which can receive (through **<inport>**) or send (through **<outport>**) events to the outside world. Ports only have one attribute: *name*, which specifies their name and may be referenced by a **<transition>** (for **<inport>**) or by a **<raise>** (for **<outport>**). If the port is a child of a class, then that port is private to that class. These are child elements of **<diagram>** or **<class>**.

It also has class diagram-specific concepts:

- **<class>** : Class elements model the structure of the diagram in the shape of attributes and relationships with other classes and behaviour in the shape of methods which may change the values of attributes as well as a statechart model which will manage the runtime behaviour of the class. A class possesses three attributes: *name*, which is simply the name of the class, *default*, which will be true if this is the default class (of which one instance is created and started at the beginning of the execution of the compiled code) and *src*, which points the location of a separate XML file containing the definition of the class (if this attribute is set, the *name* cannot be set and the class cannot have children).
- **<relationships>** : This element models any number of relationships (associations or inheritances) between the diagram's parent class and other classes. It is a child element of **<class>**.
- **<association>** : This relation element links two instances at runtime, and those two instances may exchange messages over that link. It has two mandatory attributes: *name*, which defines the name of the association and *class*, which specifies the name of the target class with which that association will be established. It may also include two additional attributes: *min* and *max*, which define the minimum and maximum cardinality of the association, respectively (they default to 0 and infinity, respectively). It is a child element of **<relationships>**.
- **<inheritance>** : This relation element allows one class to inherit all methods and attributes from another class. It has two attributes: *class*, which defines the name of the target class and *priority*, which allows to set the order in which classes must be inherited in case of multiple inheritance (relations with higher priority are inherited first). It is a child element of **<relationships>**.

- **<attribute>** : This element has 2 attributes: *name*, which defines the name of the attribute and *type*, which defines the type of the attribute. It is a child element of **<class>**.
- **<constructor>** : This element is called when an object is instantiated, and it is used to initialize its attribute values. It is a child element of **<class>**.
- **<destructor>** : This element is called when an object is destroyed. It is a child element of **<class>**.
- **<method>** : This element is a piece of action code which may be called in other pieces of code in the same class. It has 2 attributes: *name*, which defines the name of the method and *type*, which defines the type of the return value (it is optional since the method may have a void return value). It is a child element of **<class>**.
- **<body>** : This element contains the action code in a chosen programming language, in which other functions may be called, and the values of variables may be changed. It is a child element of **<method>**, **<constructor>** or **<destructor>**.

Finally, it has some statechart specific concepts as well:

- **<scxml>** : This is the top-level element of the statechart, which contains the definition of its parent class. It may have the following attributes: *initial*, which specifies the initial child state of the statechart and it may be omitted, and if so, the first child in the document is the initial state; the other attributes are several different semantic dimensions discussed in 2.3.2, including *big_step_maximality*, *internal_event_lifeline*, *input_event_lifeline*, *priority* and *concurrency*, all optional attributes. It is a child element of **<class>**.
- **<state>** : This element is the basic element of a statechart. A state may be entered or exited using transitions (and may execute optional blocks in those situations). States may be, just like in statecharts, hierarchical, meaning a state may contain other states within itself. It has two attributes: *id*, which is the identifier of the state and needs to be unique within the hierarchical level the state is in and *initial*, which specifies the initial child if it is a composite state. It is a child element of **<scxml>**, **<state>** or **<parallel>**.
- **<parallel>** : The children of a parallel element run concurrently. Parallel states have one attribute: *id*, which is the identifier of the state and needs to be unique within the hierarchical level the state is in. It is a child element of **<scxml>**, **<state>** or **<parallel>**.
- **<transition>** : A transition allows the active state to be changed, and these are triggered by events or timeouts (or even by spontaneity) and may have conditions which need to be evaluated to be true for the transition to happen (guards). A

transition may have five attributes: *target*, which identifies the target state of the transition, *after*, which denotes the number of seconds that must pass before the transition is triggered after entering the parent state (this cannot happen together with *event*), *event*, which denotes the name of the event which triggers the transition (this cannot happen together with *after*), *port*, which specifies the name of the port in which the triggering event will arrive so this must be used together with *event* and cannot happen alongside *after*, and *cond*, which specifies a condition for the transition to be triggered. It is a child element of `<scxml>`, `<state>` or `<parallel>`.

- **<history>** : A history state remembers the last active configuration when its parent state is exited. If the history state goes active again, that configuration will be restored. It has two attributes: *id*, which is the identifier of the state and needs to be unique within the hierarchical level the state is in and *type*, which may be "shallow", which is the default value or "deep", both concepts explained in section 2.3.1. A history state must not have any children. It is a child element of `<state>` or `<parallel>`.
- **<onentry>** : This element is executed when entering a state. It is a child element of `<state>` or `<parallel>`.
- **<onexit>** : This element is executed when a state is exited. It is a child element of `<state>` or `<parallel>`.

There are also actions which are executed when a transition occurs. These actions may be:

- **<raise>** : This action allows for an event to be raised, allowing other entities to be notified. It may have 3 attributes: *scope*, *port* and *target*. It is a child element of `<transition>`, `<onentry>` or `<onexit>`.
- **<script>** : This action element may contain action code in the chosen programming language. It is a child element of `<transition>`, `<onentry>` or `<onexit>`.
- **<log>** : A log allows for a string to be logged. It is a child element of `<transition>`, `<onentry>` or `<onexit>`.
- **<parameter>** : There are also parameters, which are not actions but provide parameter values. It has 3 attributes: *name*, *type* and *default*. It is a child element of `<transition>`, `<raise>`, `<method>` or `<constructor>`.

Some presented elements may only take certain elements as children (and a certain amount of each), but they have not been fully depicted here for simplicity reasons. A metamodel describing the SCCD structure is represented in Figure 3.7. There is also an Object Manager that is in charge of the management of objects and links while the application is running. It may create, start, delete, associate or dissociate instances. The current implementation of the SCCD tool will be discussed in detail in chapter 4.

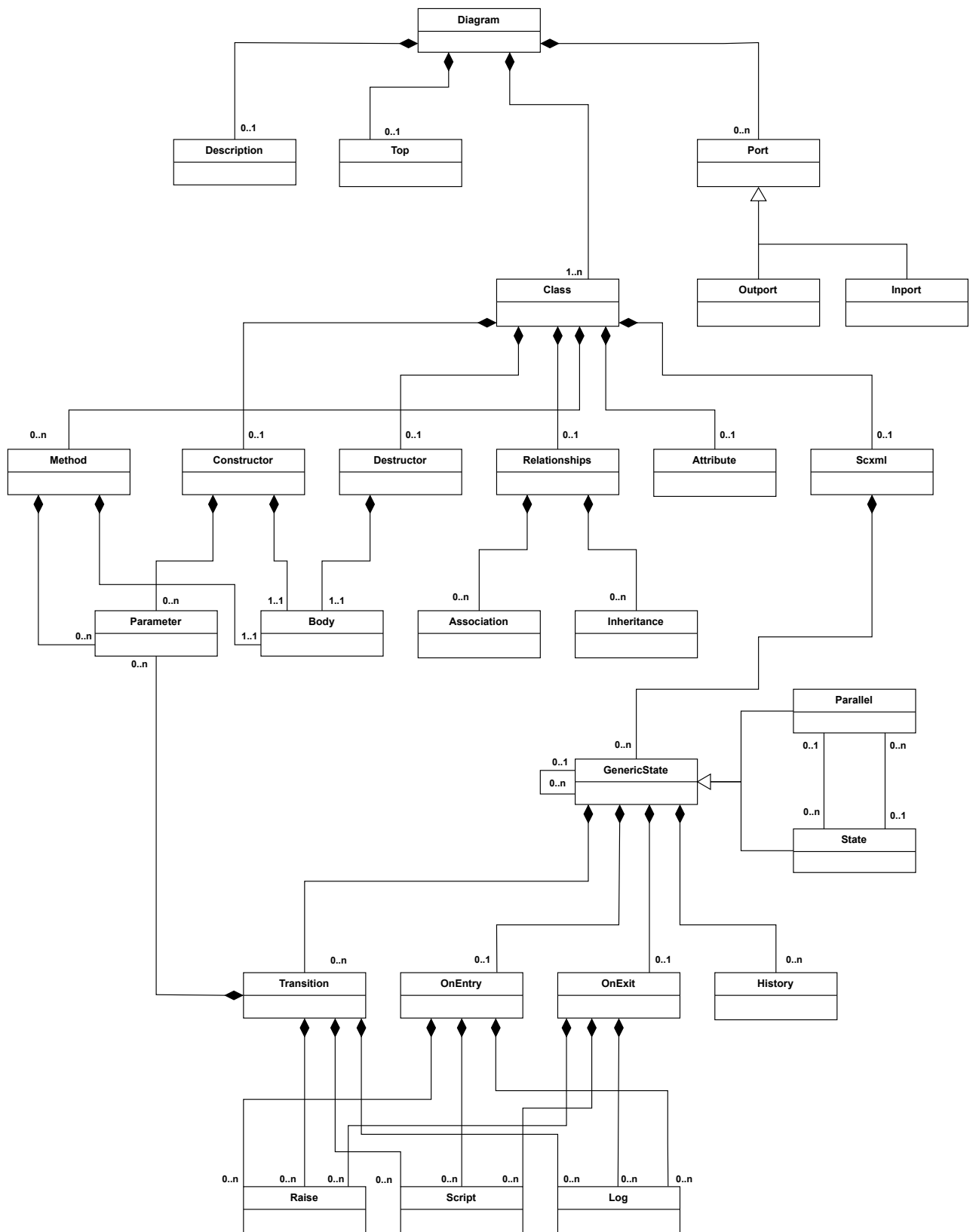


Figure 3.7: SCCD Metamodel

3.6 Debugging in Model-Driven Development

Regarding debugging in [MDD](#), several researches and approaches to the subject have been made, coming from different investigations. Our focus here will be on research, which includes debugging statecharts or state machine diagrams, as that is the focus of this project. Most research papers focused on using [MDD](#) to develop embedded systems, particularly [Real-Time Embedded Systems \(RTES\)](#). To provide some context, real-time systems may be defined as systems that depend on logical computational results and the time at which those results were produced. In contrast, embedded systems usually consist of microprocessor-based computer hardware systems with software that has the goal of performing a dedicated function, either as an independent system or as a part of a larger system [29].

Model Debugging Approaches

In 2017, Mojtaba Bagherzadeh et al. [5] proposed a novel approach to perform model-level debugging of models of [RTES](#). The implemented approach has been named MDebugger, and it is built around Eclipse Papyrus-RT [17] (a variation of Eclipse Papyrus dedicated to real-time systems), which is based on the UML-RT approach [51] (which comprises a set of constructs suitable to model real-time systems), making use of its modelling and code generation capabilities. It also relies on Epsilon tools [18] for model instrumentation. In this paper, the authors claim most proposals are to either perform model-level debugging through a model interpreter, which would require the implementation of an interpreter for the modelling language, the action language and the runtime services' library, or to maintain traceability information between the model and the code, leveraging an existing program debugger. The latter approach is highly dependent on the target language, as implementing model-based debugging features on top of an existing program debugger will require the integration of the model debugger with the different program debuggers for the languages in which the code is generated. Other mentioned limitations are the lack of reusability (as a dependency to a specific [MDE](#) tool may be added) and the semantic gap between model and code, which makes translating debugging information difficult.

The approach suggested in this paper attempts to avoid these limitations by relying on a platform-independent model instrumentation process. A Model-to-Model transformation is used to create a version of the initial model supporting debugging activities, added through model transformation rules defined for each construct of the modelling language. This will allow for the generation of applications with debugging services themselves without instrumenting the code. The generated code is already debuggable, so adding additional debugging symbols to the binary files is unnecessary. This proposal is, therefore, more general as it is less dependent on code generators. As model instrumentation is also done at the model level, the obtained debugger is easier to port from one tool to another. It is possible to see a comparison between the regular approach and this new approach in

Figure 3.8, with the novel approach located on the right-hand side.

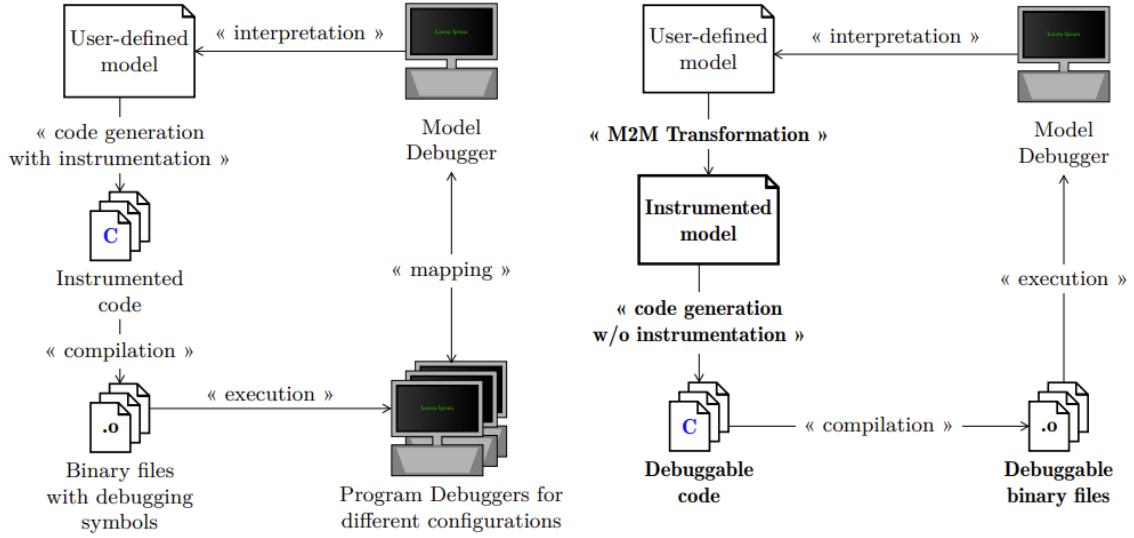


Figure 3.8: Model Debugging: Standard Approach (left) vs. Platform Independent Approach (right) (adapted from [5])

Other interesting techniques have been used and will be presented, some of which inspired the previous approach. We will take a look at some of them.

Liang Guo et al. [22] have presented a different procedure to debug statechart models. This method involves integrating established software debugging methods, such as program slicing on the generated code and relating the sliced program to model-level. The slice produced at the code level is mapped back to the model level. This paper poses an interesting question: "If the debugging of a piece of model-driven software reveals an "unexpected program behaviour" how do we reflect it at the model level?". The answer provided refers to maintaining associations between model elements and code built during code generation and exploitation of these associations to highlight model elements responsible for unexpected behaviour. The chosen technique, slicing, allows the programmer to identify what values in a specific location are affected when the program is executed with a certain input. Code is generated and then subjected to dynamic slicing, and the resultant slice is mapped back to model-level, according to the given associations.

Kebin Zeng et al. [58] proposed a graphical model debugger framework for embedded systems, yet another model debugger to allow developers to sit on a higher abstraction level, focusing on model-level. In the development of embedded systems, system complexity leads to an increased error rate in the modelling process, which is why this framework aims to detect design model errors at runtime. An initial prototype of the framework was developed on the Eclipse platform.

Philipp Graf et al. [20] proposed an architecture to debug models executed on target systems or in dedicated rapid-prototyping environments.

The process is described in Figure 3.9. Software design generates a model converted

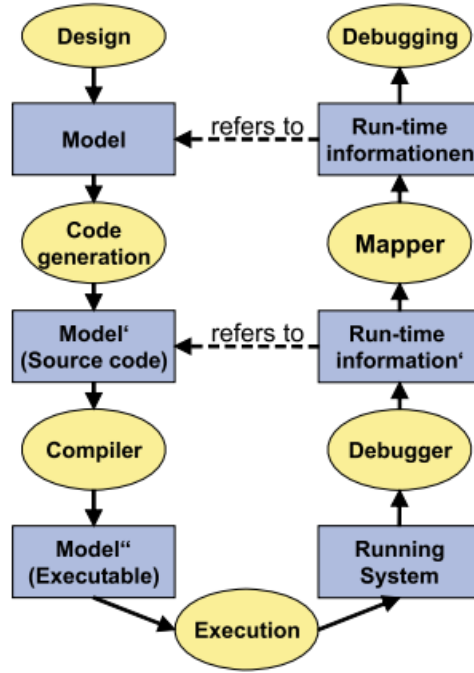


Figure 3.9: Mappings between model layers from design to debugging (taken from [20])

into source code in a high-level language (usually C/C++ in embedded software) and then compiled into an executable binary. The debugger is tasked with extracting runtime information from the executed binary and sending this information back up through the abstraction layers to model-level. The architecture described in this paper also allows the definition of various debugging perspectives and views independent of the execution platform, maximizing reusability inside the architecture.

Wolfgang Haberl et al. [24] present another approach to debugging *RTES* by capturing runtime data on the target platform and mapping it back to model-level. Debugging may then be performed at the model level by visualising input data, like feedback from the target platform's environment.

The paper claims *MDD* is one of the means to engage against the complexity that comes with designing *RTES* and takes COLA (COmponent LAnguage) [33] as a tool designed for this end. COLA is a modelling language for designing distributed, safety-critical, real-time systems, providing constructs to model a system along different layers of abstraction. The proposed approach allows for capturing actual runtime data for inputs and outputs of real-time tasks executed on the target platform (as well as their internal states). With the corresponding mappings, debugging may be performed at the model level, avoiding the manual edition of the generated code. It is also possible to specify which components will be debugged, limiting the amount of captured data. The overall procedure is facilitated by COLA's modelling and code generation features, coupled with an execution platform scheme. The import of captured data into the simulator allows for a convenient display of the data as values are shown next to their corresponding model elements. The acquired

input values may be reused for future simulations. A comparison between the classic debugging approach and the presented approach may be seen in Figure 3.10, resembling previously mentioned debugging approaches.

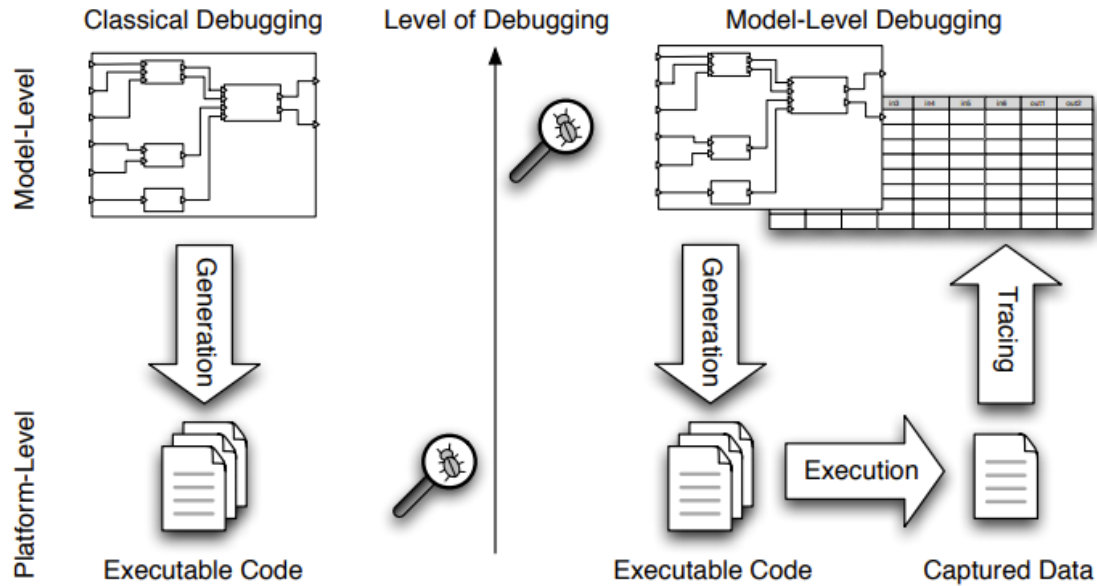


Figure 3.10: Classic vs Model-level Debugging (taken from [24])

Yet another interesting remark is made in this paper regarding [MDD](#). When generating code, if we use a certified code generator, it is assumed the generated code retains the model semantics correctly. Therefore, any manual code modifications of the generated code must be avoided to preserve its correctness. All these changes must be carried out at the model level.

What should we retain?

As the complexity of systems keeps rising, higher abstraction levels in system design should follow. Every research attempts to solve the same problem and strives to do so at the model level. The initial design is done at the model level, and the respective application is generated. The debugging of this generated application should be carried out at the model level, where the design is made. From model transformations to code slicing, all efforts point to the performance of systems' testing and debugging at a high abstraction level to provide the developers with the least amount of complexity levels possible.

SCCD IMPLEMENTATION

In this chapter, we discuss the existing implementation of the [SCCD](#) tool in detail.

4.1 SCCD - Overall Architecture

The implementation is divided into the compiler for the generated files and the runtime code to control the behaviour of the statechart during the execution of the model. As has been previously mentioned, [SCCD](#) can generate code for Python, JavaScript and C#. For this reason, there exists some code in the compiler which is shared by all 3 languages, and then some code to generate language-specific code for each one of the languages. The runtime behaviour is also defined separately for each language. To describe the implementation, architecture and functioning of [SCCD](#), a few diagrams will be presented, based on the “4+1” View Model of Software Architecture [32], which is composed of 4 different views and some usage scenarios.

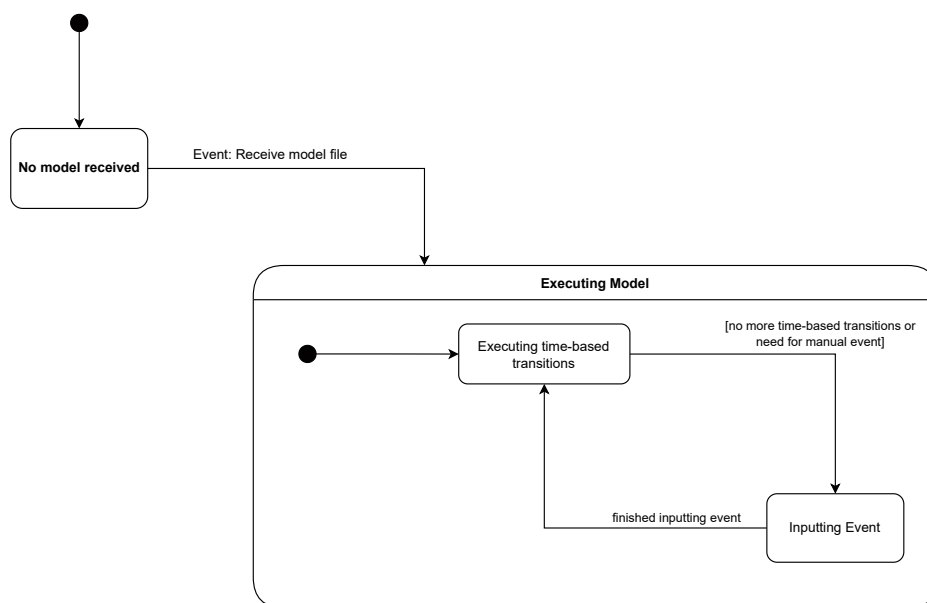


Figure 4.1: SCCD - Logical View as a State Diagram

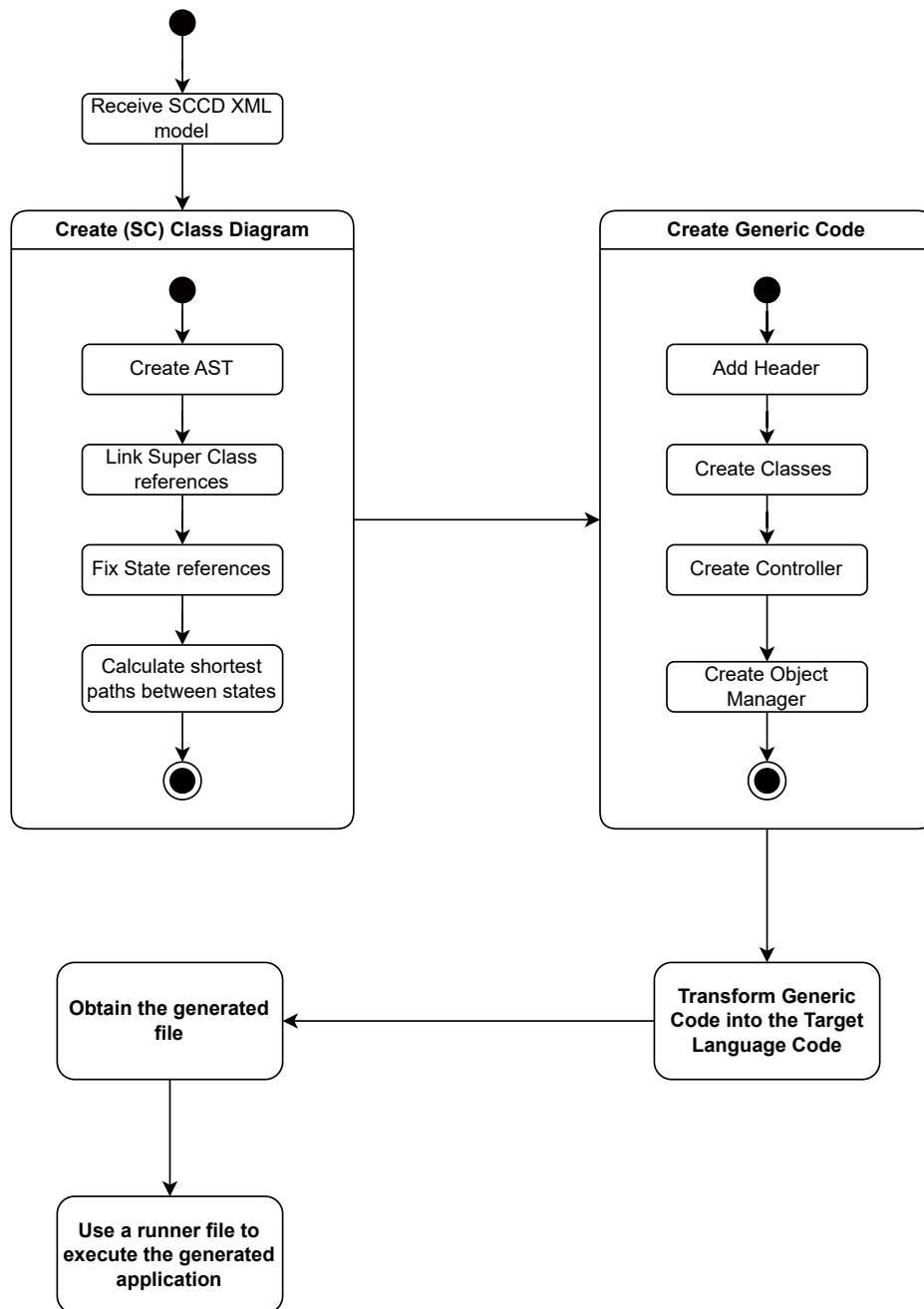


Figure 4.2: SCCD - Process View as an Activity Diagram

We can see a State Diagram representing the Logical View of SCCD in Figure 4.1, an Activity Diagram representing the Process View of SCCD in Figure 4.2 and a Component Diagram representing the Development View in Figure 4.3 (this diagram has 2 coloured arrows to help distinguish the arrows between each other, but they do not have any additional semantic meaning). The Physical View of the system is not represented here as only one physical device is present which is the machine where SCCD is being used. As such, there is no need to visually represent this view.

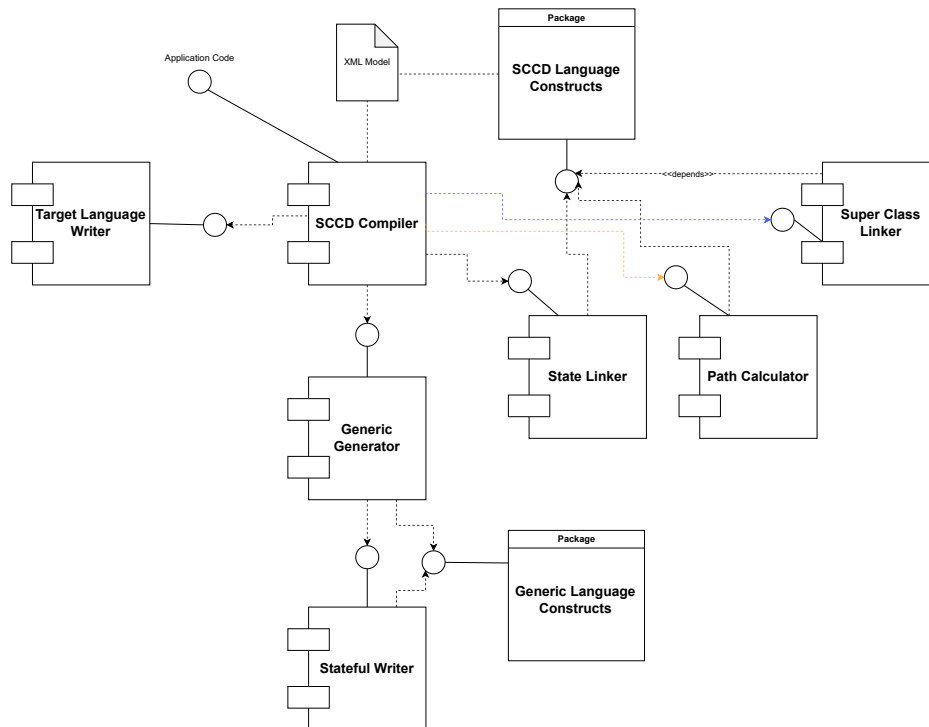


Figure 4.3: SCCD - Development View as a Component Diagram

4.2 SCCD - Compiler

4.2.1 Compiler Arguments

When running the [SCCD](#) Compiler, an XML file containing the [SCCD](#) model (details regarding the XML syntax are provided in [section 3.5](#)) must be provided alongside some optional arguments through the command line. Those arguments are the following:

- **help** - Shows a help message to the user.
- **output** - Sets the path to the generated code.
- **verbose** - Defines what output the user wants to see (options: all output; only warnings and errors; only errors; no output).
- **platform** - Sets the platform on top of which the compiled code will run (options: Threads; Eventloop; Gameloop). More on each of these in [section 4.2.2](#).
- **language** - Sets the language in which code will be generated (options: Javascript; Python).

The final usage of the compiler looks like the following expression:

```
python -m sccd.compiler.sccdc [-h HELP] [-o OUTPUT] [-v VERBOSE]
                             [-p PLATFORM] [-l LANGUAGE]
```

4.2.2 Platforms

Threads

The Threads platform is the most simple platform available. The [SCCD](#) model will be run on the main thread when the controller is started with the *Controller.start()* call (which will start all *RuntimeClassBase* instances, initializing the statechart and initializing variables such as *current_state*, *timers* and so on). If some input must be provided to the model or the output of the model's execution needs to be processed during the execution, then a separate thread should be started. A code snippet, written in Python, is presented in Listing 4.1 detailing an example usage of a model compiled for the threads platform (*target_py*) with two extra threads, each one handling a different port (input port and output port). This platform will execute the model in the command line.

Listing 4.1: Threads Platform to run an example model

```
import target_py.target as target
from sccd.runtime.statecharts_core import Event
import threading

if __name__ == '__main__':
    controller = target.Controller()

    def raw_inputter():
        while 1:
            controller.addInput(Event(raw_input(), "input", []))
    input_thread = threading.Thread(target=raw_inputter)
    input_thread.daemon = True
    input_thread.start()

    output_listener = controller.addOutputListener(["output"])
    def outputter():
        while 1:
            print output_listener.fetch(-1)
    output_thread = threading.Thread(target=outputter)
    output_thread.daemon = True
    output_thread.start()

    controller.start()
```

Eventloop

The Eventloop platform works only in combination with a UI system that allows event scheduling. For Python, the *Tkinter* package [15] is used to build the UI and for JavaScript, the web browser is used. Both of these already have an eventloop that SCCD can make use of. In this eventloop, there exists a single thread that waits for due events, which have

a timestamp for them to be triggered, and handles these events. There may not be any due event even if the event queue is not empty. In this case, the process will sleep until the next event is due. If the queue is empty, the process will sleep until it receives an input event. A code snippet, written in Python, is presented in Listing 4.2 detailing an example usage of a model compiled for the eventloop platform (*target_py*).

Listing 4.2: Eventloop Platform to run an example model

```
import Tkinter as tk
import target_py.target as target
from sccd.runtime.libs.ui import ui
from sccd.runtime.statecharts_core import Event
from sccd.runtime.tkinter_eventloop import *

if __name__ == '__main__':
    ui.window = tk.Tk()
    ui.window.withdraw()
    controller = target.Controller(TkEventLoop(ui.window))
    controller.start()
    ui.window.mainloop()
```

Gameloop

The Gameloop platform works with a game engine which calls the update function of the controller at regular intervals, advancing simulated time. Just like the Eventloop platform, it only uses a single thread.

4.2.3 Compilation Phases

There are 3 distinct phases during the compilation process which are worth being described in some detail, from the received XML file to the output file in the chosen target language. These 3 phases are described in detail below.

XML to SCCD

The first phase begins with the reception of the XML file corresponding to the model for which we want to generate code. A *ClassDiagram* object (defined on the *sccd_constructs.py* file) will be created according to the model and the XML file will be parsed on the initialisation function of the *ClassDiagram*. Python's *ElementTree* XML API is used to parse the **SCCD** model. Basic model information such as the name of the model, the author of the model, the description of the model as well as the desired language for the generated code is extracted and properly stored. All needed imports are saved as well as input and output ports to interact with the model.

Then, each class of the class diagram is processed and for each of them, a *Class* object is created. Each class will have its own relationships (associations or inheritances), attributes,

methods, constructors, destructors and, finally, an associated statechart (<scxml> element). There exists a *Statechart* object which is initialised in the *Class* object from the <scxml> element. This *Statechart* sets its root as a *StatechartNode* object which will be used to represent a state. This object has several boolean variables to define state properties. These variables help identify if the state is the root, basic, composite, history, parallel, and so on.

Each *StatechartNode* object will also parse and set its enter and exit actions, which is code that is executed when that state is entered or exited. If the state has children, the appropriate *StatechartNode* objects will be generated as the state's children. Every outgoing transition is also initialised as a *StatechartTransition*. Each transition may have a trigger event to trigger its execution, a guard condition for it to be triggered, target states, and an action to be executed when the transition occurs (just like the enter and exit actions of state), among other properties.

After the creation of the [Abstract Syntax Tree \(AST\)](#) of the model, the visitor design pattern [23] is used with the *StateLinker* object as a visitor of the created *ClassDiagram*. This will fix state references made on transition targets. States are identified hierarchically and state references are evaluated in the context of the state in which the reference occurs. So state referencing on [SCCD](#) has some specific rules that must be followed in order for the states to be referenced correctly, according to the state the reference is made in. These rules are:

- . is the state itself
- <empty string> is the root (the <scxml> element)
- .. goes up one level (to the parent state)
- a is the child with id 'a'
- a/b with a and b arbitrary state expressions evaluates state expression b in the context of the state found with state expression a.

After the state references are fixed, the optimal paths to go from one state to the other in transitions are calculated, using the *PathCalculator* object which also makes use of the visitor design pattern.

SCCD to Generic

The second phase will transform the *ClassDiagram* object obtained in the previous phase in a file containing generic code representing the behaviour of the [SCCD](#) model provided initially. There exists a class to generate the generic code called *GenericGenerator* which takes as a parameter the platform chosen by the user, which will be one of the 3 explained earlier in [4.2.2](#). The visitor design pattern is also used by the *GenericGenerator*, which will visit the previously created *ClassDiagram*.

To build the generic code, the *GenericGenerator* will use the *StatefulWriter* object as well as some generic language constructs defined in the *generic_language_constructs.py* file. The *StatefulWriter* provides an [Application Programming Interface \(API\)](#) to add code to the generic code, ranging from functions to perform stateful operations (begin and end an 'if' expression, begin and end a constructor expression, among many others) to functions that simply allow the addition of new code block entries. The generic language constructs provided are several, ranging from arithmetic expressions to raw code blocks.

An example of the usage of the *StatefulWriter* [API](#) alongside the generic language constructs is presented in [Listing 4.3](#) and the resulting expression of that code block is presented in [Listing 4.4](#). A more detailed correspondence of the generator's code and the generated code is made on [Figure 4.4](#).

Listing 4.3: Generic Code Construction Example

```
state_name = "/state_A"
state_index = 1
clazz = "State"
index_expr = GLC.MapIndexedExpression(
    GLC.SelfProperty("states"), GLC.String(state_name)
)
state_expr = GLC.NewExpression(clazz,
    [str(state_index), GLC.String(state_name), GLC.SelfExpression()]
)
self.writer.addAssignment(index_expr, state_expr)
```

Listing 4.4: Generated Code Example

```
self.states["/state_A"] = State(1, "/state_A", self)
```

In [Figure 4.4](#), the resulting expression is the one in [Listing 4.4](#). This expression is an example of the creation of a *State* object in the generated code and its assignment to a position on a dictionary containing the states of the statechart of the provided model.

To generate this code, we need the 3 variables defined on the top of [Figure 4.4](#). To create the state, we need its name and its index, defined respectively in the variables *state_name* and *state_index*. The *clazz* variable defines the type of the object we want to create using the generic language constructs. On our assignment expression, we need the left side and the right side of the assignment.

The left side represents the index of the state's position on the states' dictionary, hence it will be called *index_expr*. The language construct used to create this is the *MapIndexedExpression*, which takes 2 parameters: the mapping expression and the key expression. The mapping expression, in this case, is a self property called 'states', so we use the language construct *SelfProperty*, which takes as a parameter the name of the self property. The key expression is simply a string with the state's name, so we use the language construct *String*, which takes as a parameter the previously defined *state_name*.

Variables Definition

```
state_name = "/state_A"
state_index = 1
clazz = "State"
```

Expressions Definition

```
self.states["/state_A"]

index_expr = GLC.MapIndexedExpression(GLC.SelfProperty("states"), GLC.String(state_name))

state_expr = GLC.NewExpression(clazz, [str(state_index), GLC.String(state_name), GLC.SelfExpression()])

self.writer.addAssignment(index_expr, state_expr)

self.states["/state_A"] = State(1, "/state_A", self)
```

Final Expression

```
self.writer.addAssignment(index_expr, state_expr)

self.states["/state_A"] = State(1, "/state_A", self)
```

Figure 4.4: Generic Code Construction in Detail

As for the right side of the assignment, we use the language construct *NewExpression* to create a new object. This construct takes 2 parameters: the type of the object and an array containing the parameters for the object creation. The type of the object is defined on the variable *clazz*. The parameters needed for the state creation are passed inside an array and are the following: the index of the state, defined on the variable *state_index*, the name of the state, defined on the variable *state_name* and passed as a string using the *String* language construct and, lastly, the object inside which the *State* object is being created, which is of type *RuntimeClassBase*, an object responsible for the statechart's behaviour during runtime, which is passed using the construct *SelfExpression*.

The final expression uses the *StatefulWriter* function *addAssignment* to perform the assignment of a value, the *state_expr*, to a variable, the *index_expr*, using these as parameters.

As previously mentioned, the *GenericGenerator* is responsible for the generation of generic code representing the provided model's behaviour and follows the visitor design pattern. Through this pattern, the previously constructed structures (class diagram, classes, etc) will be visited using the methods *visit_ClassDiagram*, *visit_Class* and so on.

Each of these methods will generate different parts of the final code. The following methods may be highlighted:

- **visit_ClassDiagram** - This method is responsible for generating a header for the final file with basic information, including the model's author, name and description. It is also responsible for including the necessary imports (including mandatory imports such as the *statecharts_core* file which is responsible for the statechart's runtime behaviour as well as any optional imports, specified in the **<top>** element of the **SCCD** model). This method has a small loop to iterate over all the class diagram's classes, visiting each of them through the *visit_Class* method. This method will also generate 2 classes: *ObjectManager* and *Controller*. These classes are important during runtime, so they will be discussed further in section 4.3.
- **visit_Class** - This method will write each of the classes visited in the loop located in the *visit_ClassDiagram* method. Each class will visit its constructor, destructor and methods and it will also visit its associated statechart. This method will also write a method called *initializeStatechart* which will set the effective initial states as well as call the *initializeStatechart* method from its super class, *RuntimeClassBase*.
- **visit_Constructor** - This method is in charge of defining the constructor of its class. It writes super class construct calls and sets its semantic options according to the statechart's defined semantics, including Big Step Maximality, Internal Event Lifeline, Input Event Lifeline, Priority and Concurrency. An explanation of these semantics is made on 2.3.2. The method to set up and build the statechart structure is called and will be explained in detail further ahead. Afterwards, any user-defined attributes are initialised and any user-defined constructor is called and visited.
- **visit_Method** - This method writes any user-defined methods, adding code directly from the **SCCD** model to the generated code.
- **visit_Association** - This method will relate each class instance with other classes by creating *Association* objects with the name of the associated class and the respective minimum and maximum cardinalities.
- **visit_Statechart** - This method will write statechart logic code, according to the statechart associated with the visited class. The method present in the generated code called *build_statechart_structure* is written here. To do so, there exists the function *writeState* which is called for every state of the statechart. This creates the state object which may be a *State*, *ParallelState*, *DeepHistoryState* or a *ShallowHistoryState* and, for that object, it will set its enter and exit actions. Then, every composite state will have its children added to itself, through the *addChild* function, including the root state of the statechart represented by the empty string `""`. At the end, the *fixTree* function is called at the root state, so that every state knows which ancestor and descendant states it has. Next, the default initial states of every composite state are set, including

the root. Afterwards, the transitions of each state are created and associated with other states. For every transition with a trigger action, an *Event* object is created to represent that trigger action. If the transition has a guard condition, a function signature is given as the condition. If the transition has an associated action, this action is also created as a function and associated with it. Following the creation of transitions, enter and exit action functions are created through the *visit_EnterAction* and *visit_ExitAction* methods, respectively. After the creation of enter and exit actions for states, the transitions' action and guard functions are finally created and written. In Listing 4.5, it is possible to observe the code for this process of construction of a statechart. In this case, the statechart being built is the one on Figure 4.5.

- **visit_EnterAction** - This method writes the code related to the action itself and will then analyse the outgoing transitions of the state it is in and add timers for every transition with a time-based event so those transitions are triggered after the timer finishes.
- **visit_ExitAction** - This method removes timers related to time-based events that were previously added and will then write the code related to the action to be executed.

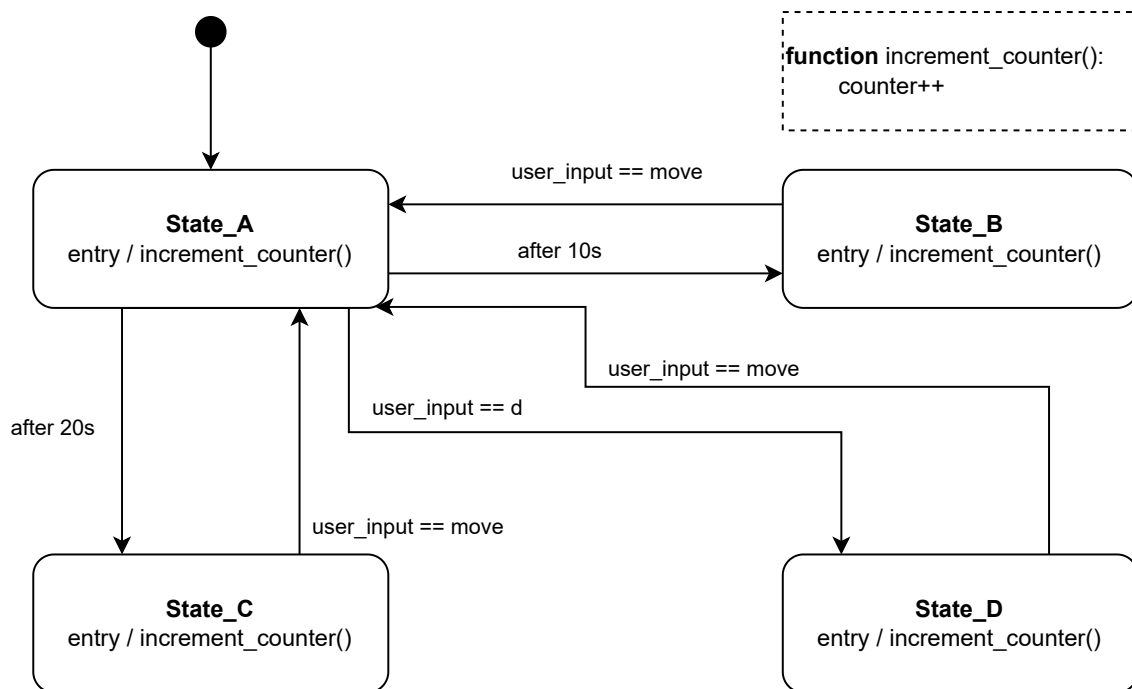


Figure 4.5: Statechart Model Example

Listing 4.5: Statechart Structure Construction Code Example

```
def build_statechart_structure(self):
    # state <root>
```

```
self . states [ "" ] = State ( 0 , "" , self )

# state /state_A
self . states [ "/state_A" ] = State ( 1 , "/state_A" , self )
self . states [ "/state_A" ]. setEnter ( self . _state_A_enter )
self . states [ "/state_A" ]. setExit ( self . _state_A_exit )

# state /state_B
self . states [ "/state_B" ] = State ( 2 , "/state_B" , self )
self . states [ "/state_B" ]. setEnter ( self . _state_B_enter )
self . states [ "/state_B" ]. setExit ( self . _state_B_exit )

# state /state_C
self . states [ "/state_C" ] = State ( 3 , "/state_C" , self )
self . states [ "/state_C" ]. setEnter ( self . _state_C_enter )
self . states [ "/state_C" ]. setExit ( self . _state_C_exit )

# state /state_D
self . states [ "/state_D" ] = State ( 4 , "/state_D" , self )
self . states [ "/state_D" ]. setEnter ( self . _state_D_enter )
self . states [ "/state_D" ]. setExit ( self . _state_D_exit )

# add children
self . states [ "" ]. addChild ( self . states [ "/state_A" ] )
self . states [ "" ]. addChild ( self . states [ "/state_B" ] )
self . states [ "" ]. addChild ( self . states [ "/state_C" ] )
self . states [ "" ]. addChild ( self . states [ "/state_D" ] )
self . states [ "" ]. fixTree ()
self . states [ "" ]. default_state = self . states [ "/state_A" ]

# transition /state_A
_state_A_0 = Transition ( self , self . states [ "/state_A" ] , [ self . states [ "/state_B" ] ] )
event_A_0 = Event ( "_0after" )
_state_A_0 . setTrigger ( event_A_0 )
self . states [ "/state_A" ]. addTransition ( _state_A_0 )

_state_A_1 = Transition ( self , self . states [ "/state_A" ] , [ self . states [ "/state_C" ] ] )
event_A_1 = Event ( "_1after" )
_state_A_1 . setTrigger ( event_A_1 )
self . states [ "/state_A" ]. addTransition ( _state_A_1 )

_state_A_2 = Transition ( self , self . states [ "/state_A" ] , [ self . states [ "/state_D" ] ] )
event_A_2 = Event ( "d" , self . getInPortName ( "input" ) )
_state_A_2 . setTrigger ( event_A_2 )
self . states [ "/state_A" ]. addTransition ( _state_A_2 )

# transition /state_B
_state_B_0 = Transition ( self , self . states [ "/state_B" ] , [ self . states [ "/state_A" ] ] )
event_B_0 = Event ( "move" , self . getInPortName ( "input" ) )
_state_B_0 . setTrigger ( event_B_0 )
self . states [ "/state_B" ]. addTransition ( _state_B_0 )

# transition /state_C
_state_C_0 = Transition ( self , self . states [ "/state_C" ] , [ self . states [ "/state_A" ] ] )
event_C_0 = Event ( "move" , self . getInPortName ( "input" ) )
_state_C_0 . setTrigger ( event_C_0 )
self . states [ "/state_C" ]. addTransition ( _state_C_0 )
```

```
# transition /state_D
_state_D_0 = Transition(self, self.states["/state_D"], [self.states["/state_A"]])
event_D_0 = Event("move", self.getInPortName("input"))
_state_D_0.setTrigger(event_D_0)
self.states["/state_D"].addTransition(_state_D_0)
```

Generic to Target

After the generation of the generic code, it is finally time to generate code for the previously chosen language. According to the chosen language, a writer will be chosen (PythonWriter or JavascriptWriter). This writer will take as an argument the output file to where it will write, thus generating the final code containing the code necessary for our model execution. This project will not modify this phase as it is unnecessary and out of its scope.

4.3 SCCD - Runtime

After the compilation phase finishes, the generated code may be run. To do this, a runner file is needed, depending on the chosen platform (from the ones mentioned in 4.2.2). In Listing 4.1 and Listing 4.2, it is possible to see how this runner file could look like in case the chosen platform is either threads or eventloop, respectively. The runner file will start the *Controller* object which is the entity responsible for starting and stopping the simulation. When the runner file is run, the *Controller*, the *ObjectManager* and every class object are initialized.

The *ObjectManager* is responsible for the management of objects and links. Class instances may communicate with the *ObjectManager* by raising events through the element **<raise>**, described in 3.5, using the *cd* scope (different scopes exist for different communication purposes, but it is not relevant to describe any other scopes). The *ObjectManager* class is generated and added to the executable code. Each class instance has a variable called *associations* which is a set containing *Association* objects parameterized by the name of the associated class as well as the minimum and the maximum cardinalities, maintaining the links between class instances.

Listing 4.6: Object Manager Association Management Example

```
class ObjectManager(ObjectManagerBase):
    def instantiate (self, ...):
        if class_name == "MainApp":
            instance = MainApp(...)
            instance.associations = {}
            instance.associations["chairs"] = Association("Chair", 0, -1)
        elif class_name == "Chair":
            instance = Chair (...)
            instance.associations = {}
            instance.associations["parent"] = Association("MainApp", 1, 1)
            instance.associations["legs"] = Association("Leg", 4, 4)
        elif class_name == "Leg":
```

```
instance = Leg (...)  
instance.associations = {}  
instance.associations["parent"] = Association("Chair", 1, 1)  
else:  
    raise Exception (...)  
return instance
```

It is possible to see in Listing 4.6 an example of a class diagram with a *MainApp* which has a number of chairs between 0 and infinity, where each *Chair* has exactly 4 legs and 1 parent which is the *MainApp* and each *Leg* has 1 parent which is a *Chair*. This code is generated in the executable file.

When the application is started, the *ObjectManager* instantiates every class and starts the associated statecharts. From this point, any instance can send events to the *ObjectManager* to control the set of executing objects. These events may have as their goal the creation or deletion of instances, the start of an instance or the association of two existing instances. The *ObjectManager* also has the responsibility of maintaining track of which objects will execute a transition, ensuring a correct scheduling between objects based on the received events. Each object has its own Statecharts executor, defined in the *RuntimeClassBase* class, all of them being synchronized in order for input and output messages between objects to be properly ensured.

In every generated file, there is a mandatory import that must always be included which is the file *statecharts_core* which contains important logic for the SCCD models to run properly. It includes several classes with statechart logic, including the following classes: *State*, *HistoryState*, *ShallowHistoryState*, *DeepHistoryState*, *ParallelState*, *Transition*, *StatechartSemantics*, *BigStepState*, *ComboStepState* and *SmallStepState*. It then includes classes for object management, including: *ObjectManagerBase*, *Event*, *Controller*, *GameLoopControllerBase*, *EventLoopControllerBase* and *ThreadsControllerBase*. Finally, responsible for the Statechart execution, we have the *RuntimeClassBase* class.

The *RuntimeClassBase* controls the execution of the Statechart. It is this class that initialises the Statechart and controls the active states and the transitions that will happen next. The statechart execution works with an *EventQueue* which contains any upcoming scheduled events (time-based transitions are inserted in this queue). For every step taken in the statechart execution, the queue is evaluated and any due event is popped and processed. A step may contain several big steps while a big step may contain several combo steps and, lastly, a combo step may contain several small steps (all step types are explained in detail in 2.3.2).

4.4 Generated Code

It is important to understand the structure of the generated code to understand how the execution works and how the debugging mechanisms will be inserted into this code.

One file is generated for the entire model. This file will contain at least 3 different classes: *Controller*, *ObjectManager* and at least one problem-related class which has the *RuntimeClassBase* as a super class. This class contains several functions, some of which we have already discussed previously. These functions are:

- **init function:** this function defines all the statechart semantics, initialises any user-defined attributes and calls the function to build the statechart structure as well as any user-defined constructor.
- **user-defined constructor:** this function contains, as its name implies, the constructor the user defined for this class.
- **user-defined destructor:** this function contains, as its name implies, the destructor the user defined for this class.
- **user-defined methods:** the user may define several class methods related to the model logic and these appear as independent functions defined inside the class.
- **build statechart structure:** this method has been explained in more detail in [4.2.3](#) and, as its name suggests, it builds the statechart structure like it is shown in [Listing 4.5](#).
- **enter/exit/transition actions:** each state has both an enter and exit action which are 2 separate methods. So, for a statechart with 3 states, there will exist 6 methods to handle every enter and exit action. Each of the enter action's methods will set the current state and add any timers if needed, as well as execute user-defined code, defined as a **<script>** in the provided XML file. Each of the exit actions will also execute user-defined code and remove the timer that triggered the exit from the state. Additionally, each transition may also have a method for itself if the user defined a **<script>** element inside the **<transition>** element in the XML file, which represents the transition's action.
- **initialize statechart:** this method will enter the default state as well as call the *initializeStatechart* function from the *RuntimeClassBase* class which will update the configuration of current states.
- **guard functions:** guard functions may also be occasionally generated depending on the model. These functions will return Boolean values which will determine whether a transition occurs or not. A transition can be associated with a guard function through the *setGuard* function available on the *Transition* object.

4.5 Execution Flow

The execution of the model starts when the runner file is executed. Let us take as an example for this explanation the model in [Figure 4.5](#), which is a statechart modelling the

behaviour of the corresponding class. The execution begins by setting the desired State-chart semantics as explained in 2.3.2 and building the statechart's structure, represented by the code in Listing 4.5, setting up all the states and their hierarchies as well as any existing transitions between them. Afterwards, user-defined attributes are initialised and user-defined constructors are called.

The execution starts at the defined default state. In the example we are following, we can see in Listing 4.5 that the default state is *State_A*. It is also possible to see in the same code listing that there are 3 outgoing transitions from *State_A*, 2 of which are time-based (so they will trigger after some timer finishes; they are marked as an *after* event with its index), while the other one is event-based (we can observe the event *d* can trigger this transition, and this event is received through an input port). So, there are 3 possible outcomes. To understand which transition will be triggered, we need to take a look at the enter and exit functions of these states.

Listing 4.7: Enter and Exit Functions Example

```
def _state_A_enter(self):
    self.current_state = self.states["/state_A"]
    self.increment_counter();
    self.addTimer(0, 10)
    self.addTimer(1, 20)

def _state_A_exit(self):
    self.removeTimer(0)
    self.removeTimer(1)

def _state_B_enter(self):
    self.current_state = self.states["/state_B"]
    self.increment_counter();

def _state_B_exit(self):
    pass
```

Let us observe the code in Listing 4.7. In this code, we can see both enter functions call the function *incrementCounter*, as represented in Figure 4.5. However, on *State_A*, we see that 2 timers are started, on indexes 0 and 1. We can observe on the code in Listing 4.5 that index 0 corresponds to the transition from *State_A* to *State_B*, while index 1 corresponds to the transition from *State_A* to *State_C*. We should note that, from a modelling point of view, it is pointless to have this second transition as it is impossible for it to be triggered. It is used here simply for explanation purposes. The *addTimer* function takes 2 arguments: the index of the timer and the number of seconds for the timer to finish. This means the transition to *State_C* will never be triggered as its timer is bigger than the timer that triggers the transition to *State_B*. This leaves the execution with 2 possibilities: 10 seconds pass and the transition to *State_B* triggers (output visible on Listing 4.8) or an input event arrives and triggers the transition from *State_A* to *State_D* (output visible on Listing 4.9, where *d* is the input event inserted by the user).

Listing 4.8: Output Of Example Model when Transition A to B occurs

```
("counter:", 1)
("counter:", 2)
```

Listing 4.9: Output Of Example Model when Transition A to D occurs

```
("counter:", 1)
d (input by user)
("counter:", 2)
```

The rest of the execution continues in a similar fashion. When a state is entered, its enter function is called and, when it is exited, its exit function is called.

DEBUGGER IMPLEMENTATION

This chapter discusses the debugging mechanisms' implementation on the existing [SCCD](#) tool. We define the solution's requirements and present the chosen implementation approach, the several features provided, and how they were implemented. The tool's implementation is fully disclosed and available for use in the following GitHub repository <https://github.com/FranciscoSimaol1/sccd-debug>.

5.1 Requirements for the Solution

To properly proceed with the construction of a model-level debugger for [SCCD](#) models, it is essential to define its requirements. As such, we may define the following requirements for our debugger, according to the debugging mechanisms mentioned in the defined features of our [Feature Model](#) (visible in Figure 3.3):

- **Execution States** - Our debugger must be able to switch between the following execution states: **Ready**, **Running**, **Pause** and **Stopped**.
- **Steps** - Our debugger must be able to provide a stepping feature, making it possible to skip transitions at will.
- **Breakpoints** - When using our debugger, the execution must stop on the defined breakpoints. These may be defined when a certain state is reached, a variable reaches some specified threshold value, or a certain simulation time threshold is reached.
- **Simulation Modes** - When our debugger is running, we provide the following simulation modes: **As-Fast-as-Possible Simulation** and **Real-Time Simulation** (and **Scaled Real-Time Simulation**).
- **Event Injection** - Our debugger must allow for the injection of events during the execution of the simulation.
- **State Inspection** - When the debugger is paused in a state, we must be able to inspect it, seeing the values of any existing variables.

5.2 Implementation Approach

Several approaches may be followed to implement debugging mechanisms in **SCCD**. As previously discussed in chapter 4, the current **SCCD** implementation has several compilation phases and objects dedicated to handling the model's execution during runtime. This means there may exist several places where debugging mechanisms can be inserted. 2 main pathways were deemed possible:

- **Runtime** - It would be possible to control debugging mechanisms from the runtime dedicated objects, possibly creating and inserting debug events (such as breakpoints or steps) directly into the event queues in the *RuntimeClassBase*. This would also mean we would have direct access to the different step primitives which are defined there (Small-steps, Combo-steps, and Big-steps, explained in section 2.3.2), which would make it possible to directly use them and allow the user to utilise them when interacting with the model in execution.
- **Model Instrumentation** - Model instrumentation is a concept that stands for changing the model directly. In this case, we could change the model by directly adding debugging mechanisms to it. There are several points in time during the compilation phase in which this may be done: during the parsing phase of the XML model into *Class* and *Statechart* objects or during the code generation phase.

Chosen Approach

The chosen approach was **Model Instrumentation** as this made it fairly intuitive to add debugging mechanisms to the tool, as opposed to changing the runtime behaviour, and this instrumentation was done during the code generation phase (mainly modifying and incrementing the *generic_generator.py*), in an attempt to strain away from glueing the model itself with the debugging mechanisms. The main idea behind the chosen model instrumentation approach was to essentially create a new state called *State Debug* in each statechart of the model (the total number of statecharts will depend on the number of classes), which every state has a transition to and from. This state represents a pause in the model's execution, so it could also be called *State Paused*. In Figure 5.1, it is possible to observe the application of this model instrumentation idea to the model represented in Figure 4.5. In orange, we can observe *State Debug* and, in dashed lines, we have this state's outgoing and incoming transitions. All the incoming transitions are triggered by the **pause** event, and all the outgoing transitions are triggered by the **continue** event.

The implementation of debugging features brought some changes to the compiler arguments described in 4.2. One extra argument is added to the final expression, which gains the following shape:

```
python -m sccd.compiler.sccd [-h HELP] [-o OUTPUT] [-v VERBOSE]
                             [-p PLATFORM] [-l LANGUAGE] [-d DEBUG]
```

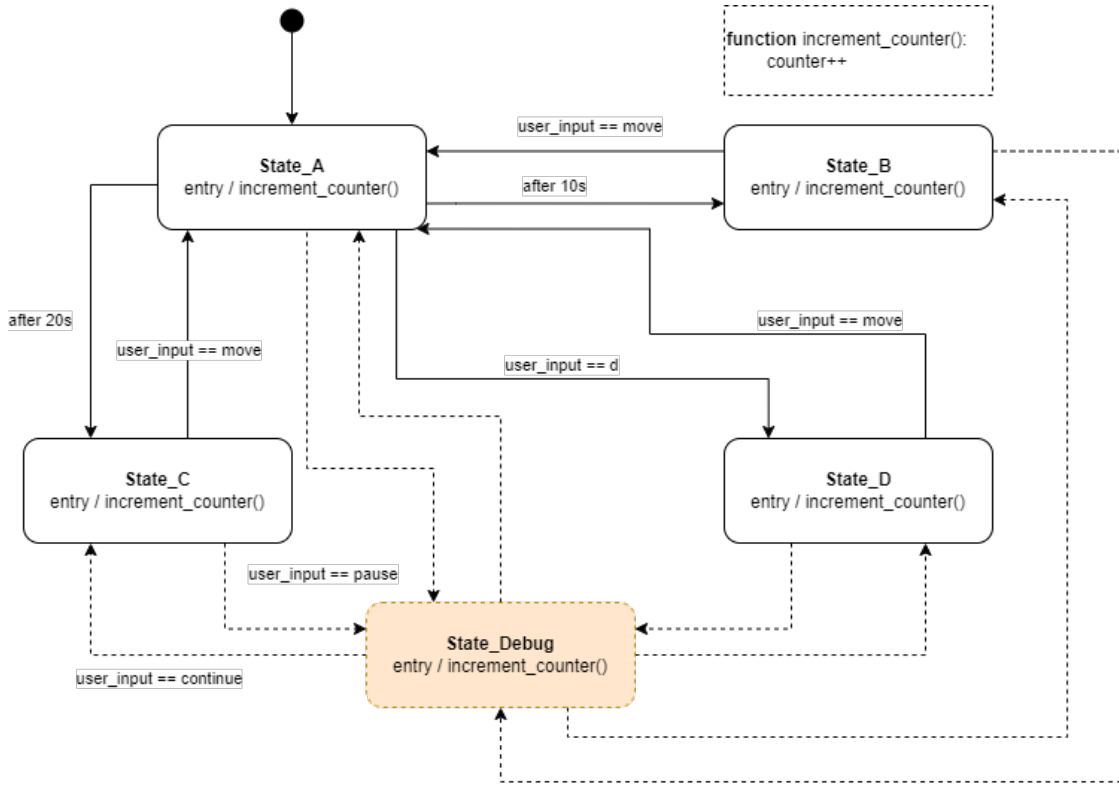


Figure 5.1: Statechart with Debugging Mechanisms Example

When d is 1, it means debug mode is on. When it is 0, the debug mode is off. It is important to mention the debugging mechanisms were exclusively added to the **Threads** platform as the **Eventloop** platform would require the presence of debugging mechanisms in the UI.

5.3 Implemented Features

Metrics Calculation

To understand some of the implemented features, it is important to understand some basic metrics used and how these are calculated.

- **Simulated Time** - This number represents the total simulation time, starting at 0 the moment the execution of the model starts. By default, it is shown in *ms*, meaning that if 10 seconds have passed since the beginning of the execution, **Simulated Time** will be 10000 (*ms*).
- **Start Time** - The **Start Time** is calculated on the enter function of each state, meaning it is updated every time a state is entered, and it is equal to the **Simulated Time** at that moment. This means we save the timestamp in which a state was entered, therefore being able to determine how long it has been since that state was entered. Thus it is also shown in *ms*.

- **Execution Time** - The following formula is used to calculate **Execution Time**:

$$executionTime = executionTime + (simulatedTime - startTime)$$

The **Execution Time** represents the total execution time excluding time spent debugging. So, it is a cumulative variable that sums its previous value with the time spent in the current state. It is calculated on the exit function of each state, meaning it is updated every time a state is exited. Since it does not account for time spent debugging, *State_Debug* is not considered as a state in this case, making it possible for this value to exclude this debug time value. It is also shown in *ms*.

- **Local Execution Time** - The following formula is used to calculate **Local Execution Time**:

$$localExecutionTime = localExecutionTime + (simulatedTime - startTime)$$

The **Local Execution Time** represents the local execution time excluding time spent debugging, meaning that it calculates the execution time for a specific state only. It starts at 0 every time a new state is entered for the first time. Let us take as an example *State_A* in 5.1. The execution starts, *State_A* is entered, and the user pauses the execution after 5 seconds (the transition to *State_B* will only trigger after spending 10 seconds on *State_A*). The **Local Execution Time** is set to 5 when *State_A* is exited. When *State_A* is entered again, **Local Execution Time** will still be 5, which will be used to calculate the leftover time until the next transition occurs. It is also shown in *ms*.

- **Cumulative Debug Time** - The **Cumulative Debug Time** represents the total time spent debugging, and it is calculated using the following formula:

$$cumulativeDebugTime = simulatedTime - executionTime$$

The total simulation time is subtracted by the total execution time which does not include time spent debugging, meaning the value that results from this subtraction is the total time spent debugging. It is also shown in *ms*.

Simulation Modes

As mentioned in 3.1, it is important to provide the user with some different simulation modes:

- **Real-Time Simulation** - The **Real-Time Simulation** is the default mode, performing the simulation as expected with no changes to any transitions that are triggered by timers.
- **Scaled Real-Time Simulation** - The **Scaled Real-Time Simulation** is an alternative to the **Real-Time Simulation**, as it uses a scale factor to change any existing timers' speed.

- **As-Fast-as-Possible Simulation** - The **As-Fast-as-Possible Simulation** uses a scale factor with the value of infinity, reducing every timer to 0, performing the transitions that the shortest timer would trigger, so the meaning of the model remains consistent.

If debug mode is on, choosing one of the above-mentioned simulation modes is possible. This is what the expression to run the runner file with the desired simulation mode looks like:

```
python runner.py [-s SIMULATION_TYPE] [-f SCALE_FACTOR]
```

The **simulation type**, **s**, may be:

- **0** - Corresponds to the default mode, **Real-Time Simulation**, where the **scale factor** is 1.
- **1** - Corresponds to the **Scaled Real-Time Simulation**. By default, the **scale factor** is 1, but it takes the value of the second argument **f** if it is specified.
- **2** - Corresponds to the **As-Fast-as-Possible Simulation**. The **scale factor** has the value of infinity, hence making the timer 0.

The calculation of the resulting timer value is the following:

$$scaledTimerValue = originalTimerValue / scaleFactor$$

So, to provide an example, if the user wants to run their runner file *runner.py* with the **Scaled Real-Time Simulation**, using a **scale factor** of 2, the expression would look like:

```
python runner.py -s 1 -f 2
```

Play/Pause

This is the most basic but essential feature in a model debugger. The **Play/Pause** feature pauses and restarts the simulation at the exact point in which it stopped. As for the implementation, the core idea is to look at playing and pausing as events. There exist 2 events that use the *input* port: **pause** and **continue**.

The **pause** event can be triggered at any point during the execution and at any state. When this event occurs through the *input* port (user's input), a transition will instantly occur from the current state to *State_Debug*, pausing all timers beforehand. On *State_Debug*, all the user-defined variables are printed alongside their current value.

The **continue** event can only be triggered when inside *State_Debug*. When this event occurs through the *input* port (user's input), a transition will instantly occur from *State_Debug* back to the state it was in, restarting the timers at the exact point they were left at.

From an implementation point of view, a few aspects must be considered for this to work according to the description above. Particular care was taken with parallel states. To

correctly handle parallel states, some *Queue* objects are used: a *calculationsQueue* and an *activeStatesQueue*. Now, it is important to understand that the enter functions of each state are divided into 2 branches and that the branch that is entered is chosen according to a boolean flag variable which indicates whether it is the first time entering that state or not. If it is not the first time, it means the transition that brought the execution to the current state came from *State_Debug*. Of course, if this state has been entered before and exited normally and is entered again, it will be considered as the first time it is entered, as long as the previous state was not *State_Debug*.

The first branch is the one chosen when the previous state was not *State_Debug*. Here, the metric **Local Execution Time** is set to 0 as the state is being entered for the first time, and the current state is queued into the *activeStatesQueue*. The state's user-defined enter script code is run, and all the timers for future transitions are started.

The second branch is chosen when the previous state was *State_Debug*, meaning we are returning to the previous state of the execution. Here, the new timers are added. The main difference between the 2 branches is the values of the added timers.

Let us observe the Listing 5.1. In this example, there are 2 timers, one for 10 seconds and another for 20 seconds, divided by the scale factor. If the execution is paused, these timers are removed (they are always removed when the state is exited). When the execution restarts, new timers are added. However, the time value provided to each of the timers takes the initial time value and subtracts from it the time that has passed in that state: $timeLeft = fullTimerValue - timeThatHasPassed$. In the Listing 5.1, the time that has passed is divided by 1000 because the variable *localExecutionTime* is in *ms* and the timers accept their values in *s*. Therefore a normalization of the value must be done.

Listing 5.1: Timers Added in Different Branches

```
if self.firstTime == True:
    self.addTimer(0, 10 / self.scaleFactor)
    self.addTimer(1, 20 / self.scaleFactor)
    ...
else:
    self.addTimer(0, 10.0 - ((self.localExecutionTime / 1000.0) / self.scaleFactor))
    self.addTimer(1, 20.0 - ((self.localExecutionTime / 1000.0) / self.scaleFactor))
    ...
```

It should also be mentioned there is another event called the **stop** event which stops the simulation, but this will be explained further ahead in 5.3, in more detail.

Breakpoints

The breakpoints may be split into 3 different types:

- **State Breakpoints** - These breakpoints stop the execution when the specified state is entered. This is achieved with a transition to *State_Debug* that is triggered after the code of the first branch of the enter function is run (as explained in 5.3).

- **Variable Breakpoints** - This type of breakpoint works similarly to the one above, adding a condition before creating the timer. For instance, if there is a variable named *counter* and we want to stop the execution when this variable reaches the value 5, the generated code will look like:

```
if counter == 5:
    addTimer(4, 0)
```

Here the number 4 is simply the timer index, which is generated automatically and will vary according to each model's specifications. The timer value is 0, hence the transition occurs instantly.

- **Time Breakpoints** - This is the most complex breakpoint type as it has some extra concerns that must be dealt with. A **Time Breakpoint** requires the user to specify a certain timestamp for the execution to stop. This timestamp logically relates to the execution time and does not include time spent on *State_Debug*. There is an array structure that keeps track of **Time Breakpoints** that have expired, as these timers have to be removed after they are triggered so the execution may work correctly. Every breakpoint timestamp will have an associated timer index. Through this index, it is possible to determine the index that has expired in the *expiredTimestamps* array and remove the expired timers.

All transitions for every breakpoint type are created beforehand with an associated timer index and they are all stored in a map of lists for each state. The timer indexes are organized in the following manner: initially, all transitions with timers are stored; afterwards, each **Time Breakpoint** gets its own index (so 1 index per timestamp); then 1 index is associated with all **Variable Breakpoints**; finally, 1 last index is associated with all **State Breakpoints**.

To define breakpoints in Python code, for instance, we select which line of code we want the execution to stop in. Defining breakpoints in [SCCD](#) models is not as simple as just selecting which state we want to stop the execution in. It is also impossible to insert debugging-related elements like breakpoints in the XML file containing the original model. The solution found for this was for the user to specify any breakpoints in a pre-created file called *breakpoints.xml*, which contains an element called **<breakpointsList>** which will contain **<breakpoint>** elements inside of it. An example of this file is shown in [Listing 5.2](#), with an example of each breakpoint type mentioned above.

Listing 5.2: Breakpoint Specification File

```
<?xml version="1.0" ?>
<breakpointsList>
  <breakpoint state="/state_B" />
  <breakpoint timestamp="10" />
  <breakpoint variable="counter" value="5" />
</breakpointsList>
```

The *breakpoints.xml* file is read by the code generator, which will then process the breakpoints and generate them accordingly.

Steps

When dealing with code debuggers, steps are used to skip lines of code we are not interested in debugging, or to simply move along the code slowly and detect changes in variables' values and such. In our debugger, steps will allow us to move along the model's execution at our own pace, leisurely observing the evolution of the model's state. Steps are generated during runtime, and these are handled differently for different transition types:

- **Time-based Transitions** - Once we enter a state, all existing transitions triggered by a timer's completion are compared regarding their timer values. If we analyse the case in which we have 2 transitions leaving the same state, one with a timer of 10 seconds and the other with a timer of 20 seconds, we can guarantee the second one will never be triggered. So, the only step that is made available is the one with the smallest timer value (if there are 2 with the same timer value, the one with the highest priority is chosen). These comparisons are made during runtime, and the chosen step is generated as a new transition that is triggered by the input event *step*.
- **Event-based Transitions** - **Event-based Transitions**, unlike **Time-based Transitions**, may be more than 1. Alongside the step to perform the time-based transition, any events that can trigger a transition from the current state are presented to the user. So, effectively speaking, no steps are generated here, these transitions are simply shown to the user.

Any generated transition is created inside each state's enter functions. The functions responsible for creating and/or showcasing these transitions to the user are: *process_time_transitions* and *process_event_transitions*.

Tracing

A feature that was not initially planned and was not part of the requirements for a model debugger is **Tracing**. What **Tracing** does is, essentially, to record all the actions that happened during the simulation of a model. When execution stops, it is possible to see, in the end, all the events that happened (whether they were induced by the user or simply by the system itself), including their timestamps and the values of all the user-defined variables in that moment. Our tracer also records some useful information about the execution, containing some of the previously mentioned metrics in 5.3 (**Simulated Time, Execution Time and Cumulative Debug Time**).

To implement this feature, some decisions had to be made:

- **When does the execution stop?** This question is quite important since [SCCD](#) models don't include final states, and the user may want to stop the execution at any point in time. So, for the user to have the ability to stop the execution at any point in time, a **stop** event was created. This event works exactly like the **play** and **pause** events. When the user latches the **stop** event onto the simulation, the current state will be exited and a transition to *State_Final* will be triggered. This *State_Final* also works similarly to *State_Debug*.
- **Which events should be recorded? And how are they recorded?** It is important to know exactly which events we want to record, so we should think about what the most important information about a statechart's execution is. The tracer records the following events:
 - **State Enter** - A new entry is created on the tracer whenever a state is entered. This includes a regular entrance and a re-entrance. A re-entrance occurs when entering a state and returning from the paused state. These events are recorded by the strings "*state_enter stateName*" and "*state_re-enter stateName*", where *stateName* refers to the name of the state that was just entered.
 - **State Exit** - A new entry is created on the tracer whenever a state is exited. A state may be exited due to several different events, so each event is recorded with the string "*state_exit stateName - eventName*", where *stateName* refers to the name of the state that was just exited and *eventName* refers to the name of the event that triggered that exit. The associated event may be a **pause**, a **step** or an **after** event which occurs when a timer finishes. The event can also be a **stop** event, which means the execution has stopped. There may be an exit without an associated event. This occurs only when the state that was exited is a parent state of a state that was exited before (in this case, the deepest state in the hierarchy will save the event's name).

Each event is saved through the function *saveEvent*, which will take as arguments the name of the event, the total simulated time at that point (the absolute timestamp), and a list containing tuples where each tuple is composed by the name of a user-defined variable and its value at that point in time. For each event, a *TracedEvent* object is built with the arguments above-mentioned and stored in a traced events list.

- **How is the final tracing log built?** The final log is built when *State_Final* is entered. The *Controller* is stopped, and a function to save the execution's trace is called. This function will create a text file to write the tracing log to, and then, for each element in the traced events list, a new line will be added to this file. When the log is complete, the file is closed, and the system is exited. An example of a trace log of the execution of the model in [Figure 5.1](#) is shown in [Listing 5.3](#).

Listing 5.3: Tracing Log Example

```

Execution Info

Total Simulation Time: 20349.0 ms (includes Debug Time)
Execution Time: 17910.0 ms
Total Debug Time: 2439.0 ms

Events
0. Timestamp: 0; Name: state_enter: /state_A; Attributes: [counter: 1; ]
1. Timestamp: 10000; Name: state_exit: /state_A - _0after; Attributes: [counter: 1; ]
2. Timestamp: 10000; Name: state_enter: /state_B; Attributes: [counter: 2; ]
3. Timestamp: 12435; Name: state_exit: /state_B - step0; Attributes: [counter: 2; ]
4. Timestamp: 12435; Name: state_enter: /state_A; Attributes: [counter: 3; ]
5. Timestamp: 16046; Name: state_exit: /state_A - pause; Attributes: [counter: 3; ]
6. Timestamp: 18485; Name: state_re-enter: /state_A; Attributes: [counter: 3; ]
7. Timestamp: 20349; Name: state_exit: /state_A - stop; Attributes: [counter: 3; ]

```

In Listing 5.3, we have the metrics printed first and then the several events that occurred during the simulation of the model. Looking at the trace log, it is possible to conclude what happened during the execution: *State_A* was entered, raising the user-defined counter variable to 1; a timeout occurred, triggering a transition from *State_A* to *State_B*; *State_B* was entered, raising the counter variable to 2; the user made a step forward, triggering a transition from *State_B* to *State_A*; *State_A* was entered, raising the counter variable to 3; the user paused the execution at timestamp 16046 ms and then restarted it at 18485 ms (*State_A* was exited and re-entered); the user stopped the execution, with the counter having a final value of 3.

Help

This feature was also not part of the requirements for a model debugger. Still, it was included as an important feature to provide a more user-friendly experience. This is implemented in a similar way to the aforementioned *State_Debug* and *State_Final*, as *State_Help* is added to the list. Every state except for *State_Final* has a transition to this state, so the **help** event that triggers any of these transitions may be sent at any time during a simulation. *State_Help* has an enter function which displays the available commands and some relevant information to the user. It mentions information about the **pause** event, the **continue** event, the **stop** event, the **step** event, other possible events the user may send during the simulation, the different simulation modes and how to change them and how to add a breakpoint. After providing this information, a timer with a duration of 0 seconds is added, so the execution automatically returns to the previous state after printing the aforementioned relevant information.

5.4 Changes to the Generated Code

We will now take a look at the changes made to the generated code when debugging mechanisms are added. A comparison will be made with the description of the generated

code of the previous chapter 4.4. Just like it is mentioned in this chapter, there is a single file containing the execution code for the model, containing at least 3 different classes: *Controller*, *ObjectManager* and at least one problem-related class which has the *RuntimeClassBase* as its super class. The *Controller* and the *ObjectManager* are unaffected by the addition of debugging mechanisms. All changes are made in the problem-related class. The structure of this class remains mostly the same, with deeper changes made in each individual function. We will take a look at these changes, including some newly added functions:

- **init function:** this function works as it did before, but it now initializes some class variables (including some of the previously mentioned metric variables), queues to handle the active states and time calculations, and lists to store traced events for the trace log and expired timestamps for breakpoints. Here, the function that will handle the command line arguments for the simulation type and speed scale factor is also called. It is the *setSimulationSpeed* function.
- **set simulation speed function:** this function parses the command line arguments to set the simulation type and the scale factor the user selected. The first argument refers to the simulation type, whereas the second one decodes the scale factor value. If the selected simulation type is the **Real-time Simulation**, the scale factor class variable is set to 1. If it is the **Scaled Real-time Simulation**, the second argument is required, and the scale factor is set to this argument's value. If it is the **As-fast-as-possible Simulation**, then the scale is set to an infinite value, which will make every timer have a final value of 0, making it instantaneous.
- **build_statechart_structure:** this function suffers some changes when adding debugging mechanisms. Besides all the model's original states and transitions, *State_Debug* and *State_Final* are also created, alongside all outgoing and incoming transitions from these states. This includes all breakpoint transitions which are incoming transitions to *State_Debug*. There are also 3 events that are created here: **pause**, **continue**, and **stop** and are then associated with the appropriate transitions. For debugging control purposes, some transition dictionaries are created to store specific transitions per each state. These dictionaries are:
 - **timedTransitions** - the transitions stored here are all of the transitions which are triggered by timers. This is useful when generating step transitions for the shortest timer values.
 - **eventTransitions** - the transitions stored here are all of the transitions triggered by some existing event. This is also useful when showing to the user transitions that are triggered by events.
 - **createdTransitions** - the transitions stored here are all of the transitions that were generated during runtime for stepping purposes. If a transition is stored here, it will not be generated again.

- **pauseTransitions** - this dictionary contains all of the transitions which go from any state to *State_Debug*. This is very important to determine whether the cause of an exit of a certain state is due to a **pause** event or not. If we know this, we can know if a state is being entered for the first time or if it is returning from a paused state. A better explanation for this will be provided further down. It also contains all the transitions that exit *State_Debug*, including transitions that have as targets multiple states (concurrent states). These transitions have several different combinations of possible concurrently active states.
- **stopTransitions** - this dictionary contains all of the transitions which go from any state to *State_Final*.
- **timeBreakpointTransitions** - this dictionary contains all of the transitions corresponding to breakpoints placed on specific timestamps. By storing these transitions, we can determine whether the reason for a certain state's exit is due to a breakpoint or not. If it is, we can consider that breakpoint as expired.
- **genBreakpointTransitions** - this dictionary contains all of the transitions corresponding to breakpoints not placed on timestamps. So it contains breakpoints placed on specific states or variable values. Just like the **timeBreakpointTransitions**, this dictionary allows us to determine whether the reason for a certain state's exit is due to a breakpoint or not.
- **helpTransitions** - this dictionary contains all of the transitions from any regular state to *State_Help*.

All of these dictionaries are put together on state exit functions to determine the event responsible for the exit. This is important to display the appropriate event names in the tracing logs.

- **state enter functions:** enter functions suffer some changes with debugging mechanisms added. We may identify 4 different state enter functions:
 - **Regular State Enter Function** - this is the most common state enter function. Every state has an enter function with this format, except for *State_Debug*, *State_Final* and *State_Help*. Let us take as an example *State_A* in the model in Figure 5.1.

Listing 5.4: State Enter Function *State_A*

```
def _state_A_enter(self):
    self.current_state = self.states["/state_A"]
    self.startTime = self.getSimulatedTime()

    while (not self.didCalcs.empty()):
        self.didCalcs.get()

    if self.firstTime == True:
        (First Branch Code)
```

```

else:
    (Second Branch Code)

```

In Listing 5.4, we can observe the main code structure for a regular state enter function. The current state and the starting time of *State_A*'s activity are set. Afterwards, the *didCalcs* queue is emptied. This queue is important to handle metrics calculations when there are multiple active states (parallel states) as it will only allow one of the active states to perform the calculations of those metrics so they will not stack values on top of each other (for example, with two active states, without this queue, the execution time would be doubled as each of them would calculate that value separately). Subsequently, as mentioned in the description of the **Play/Pause** feature in 5.3, two branches exist in this function. We will now take a look at each of them.

Listing 5.5: State Enter Function *State_A* First Branch

```

self.localExecutionTime = 0.0
self.active_states.put(self.current_state)

self.increment_counter();

self.print_internal_state("/state_A")
allAttTuples = []
allAttTuples.append(["counter", self.counter])
self.saveEvent("state_enter:␣/state_A", self.getSimulatedTime(), allAttTuples)

timers = []
if simulation_type != "As-fast-as-possible_Simulation":
    self.addTimer(0, 10 / self.scaleFactor)
    timers.append(10)
    self.addTimer(1, 20 / self.scaleFactor)
    timers.append(20)
    self.process_time_transitions(timers, "/state_A")
else:
    self.addTimer(0, 10.0 / self.scaleFactor)
    self.process_event_transitions("/state_A")

```

In Listing 5.5, we can see the first branch which is the code executed when that state is entered normally (it is starting its execution, it is not coming from a paused state). On this branch, the local execution time (time that passes while we are in the current state) is initialised at 0, and the current state is queued into the active states queue. Following this, the user-defined scripts are executed. In this example, the *increment_counter* function is called. This function simply increments the user-defined counter variable. Afterwards, the function *print_internal_state* is called. It simply prints the user-defined variables at that point in time and that is why it is called after the user-defined scripts are executed (so all the changes in the system's state are visible). Next up, all of the user-defined variables are appended to a list of tuples where each tuple represents the variable's name alongside its value at that point of the execution.

Then, an event is saved, containing its name, the absolute timestamp of the simulation, and the list of all the values of the variables in the system.

Afterwards, there is a distinction between **As-fast-as-possible Simulations** and other simulation types. If the simulation type is **Real-Time Simulation** or **Scaled Real-Time Simulation**, then all the timers to trigger transitions should be added, and an additional stepping transition should be created for the transition with the shortest timer by calling the *process_time_transitions* function. However, if the simulation type is **As-fast-as-possible Simulation**, the only timer that is added is the shortest timer. This means that only the transition with the shortest timer will occur. In this case, since the scale factor is infinite, then this transition occurs instantly. We also do not want to generate any stepping transitions for the user, as it makes no sense to do this when all transitions with timers are executed instantly. After handling the time-based transitions, all the available events are presented to the user, alongside the targets of the transitions triggered by those events, through the function *process_event_transitions*.

Listing 5.6: State Enter Function *State_A* Second Branch

```
self.addTimer(0, 10.0 - ((self.localExecutionTime / 1000.0) / self.scaleFactor))
self.addTimer(1, 20.0 - ((self.localExecutionTime / 1000.0) / self.scaleFactor))
allAttTuples = []
allAttTuples.append(["counter", self.counter])
self.saveEvent("state_re-enter:□/state_A", self.getSimulatedTime(), allAttTuples)
```

In Listing 5.6, we can see the second branch, which is executed when we come back from a paused state, resuming the execution where it was left off. This branch is a lot simpler as the user-defined scripts have already been executed, the stepping transitions have already been created and so on. The only tasks that need to be performed here are the creation of new timers with the time left for each transition to be triggered and the storage of the event of re-entrance in the state on the trace log.

- **State_Debug Enter Function** - In Listing 5.7, we can observe the enter function of *State_Debug*. This function sets the *firstTime* variable to false because when the execution is restarted, the state we were previously in will be entered again, resuming the execution at the point where it was left off, meaning it is not the first time that state is being entered. Afterwards, the prints indicate that the debug mode was entered and show the currently active states and the values of all user-defined variables.

Listing 5.7: State Enter Function *State_Debug*

```
def _state_Debug_enter(self):
    if self.firstTime:
        self.firstTime = False
    targets = list(self.active_states.queue)
```

```

states_names = [s.name for s in targets]

print("DEBUG_MODE")
print("Current_State: {}".format(states_names))
print("counter" + ": {}".format(self.counter))

```

- **State_Final Enter Function** - In Listing 5.8, we can observe the enter function of *State_Final*. It is a short function that will stop the execution through the *Controller* class, it will call the *saveExecutionTrace* function to create a trace log for the execution and then it will finally make a system exit.

Listing 5.8: State Enter Function *State_Final*

```

def _state_Final_enter( self ):
    outputName = "executionTrace.txt"
    self.controller.stop()
    self.saveExecutionTrace(outputName)
    exit(1)

```

- **State_Help Enter Function** - As explained previously in 5.3, this function prints out a lot of relevant information to the user. It also adds a timer with a duration of 0 seconds to automatically return to the state it was previously in.
- **state exit functions:** Just like enter functions, exit functions are also drastically modified when debugging mechanisms are generated. Although, unlike state enter functions, here we can only identify 3 different state exit functions, as *State_Final* does not need an exit function, as it is never exited:
 - **Regular State Exit Function** - This is the regular format for state exit functions. Once again, let us take as an example *State_A* in the model in Figure 5.1. For simplicity, the code for this function was divided into 2 listings.

Listing 5.9: State Exit Function *State_A* (1)

```

def _state_A_exit(self):
    self.removeTimer(0)
    self.removeTimer(1)
    index = 2
    for et in self.expiredTimestamps:
        self.removeTimer(index)
        index = (index + 1)

    if self.didCalcs.empty():
        self.localExecutionTime = (self.localExecutionTime + (self.getSimulatedTime() - self.startTime))
        self.executionTime = (self.executionTime + (self.getSimulatedTime() - self.startTime))
        self.didCalcs.put(True)

    found = False
    for b in self.timeBreakpointTransitions["/state_A"]:
        if b.enabled_event != None:
            found = True
            timerIndex = int(b.enabled_event.name[1:2])
            startingIndex = 2

```

```

        self.expiredTimestamps[timerIndex - startingIndex] = True

    for b in self.genBreakpointTransitions["/state_A"]:
        if b.enabled_event != None:
            found = True

    if ((self.pauseTransitions["/state_B"].enabled_event == None)
        and (not found)) and (self.helpTransitions["/state_B"].enabled_event == None):
        self.firstTime = True
        queue = self.active_states.queue
        if queue[0] == self.states["/state_B"]:
            self.active_states.get()
        else:
            index = 0
            iteration = 0
            for e in queue:
                if self.states["/state_B"] == e:
                    index = iteration
                    iteration = (iteration + 1)
            del self.active_states.queue[index]

    (code to save an event for the final tracing log)

```

The first part of the code is shown in Listing 5.9. We saw that 2 timers for transitions were added for this state's enter function. When that state is exited, its timers must be removed, otherwise they will remain active. This must also be done for timers that are related to timestamp breakpoints, but these are only removed when they are triggered for the first time, as the timestamps concern absolute time, meaning they can last through several state iterations.

After removing all the timers, the execution time variables are calculated. They may only be calculated once, hence the presence of an *if* statement. This prevents multiple calculations when multiple states are active concurrently.

Subsequently, all breakpoint transitions are reviewed to check if the state is being exited due to a breakpoint. This is done by checking each breakpoint transition for its *enabled_event* variable, as this variable will contain the event name if the breakpoint was activated. If it is, the local variable *found* will be set to true. For the timestamp breakpoints, if their timer finished and caused the state exit, their index will be marked as expired, so the timer is removed later. The pause and help transitions for this state are also checked to see if we are moving to *State_Debug* or *State_Help*. If none of these conditions is true, then it means the execution is not being paused, therefore the state can be removed from the *active_states* queue and the *firstTime* variable becomes *True* again. However, a state may only remove itself from the queue if it is the state being exited. If the head of the queue is not the current state, then the queue must be searched for the correct state to be removed. Initially, the queue was seemingly the adequate data structure to use here as we handled concurrent objects. However, given that in some situations, the object to be removed is

not the head of the queue, this may warn us that a queue is unnecessary and a regular list would be more adequate.

Listing 5.10: State Exit Function *State_A* (2)

```

allTransitions = []
allTransitions.extend(self.timedTransitions["/state_A"])
allTransitions.extend(self.eventTransitions["/state_A"])
allTransitions.extend(self.timeBreakpointTransitions["/state_A"])
allTransitions.extend(self.genBreakpointTransitions["/state_A"])
allTransitions.extend(self.createdTransitions["/state_A"])
allTransitions.append(self.stopTransitions["/state_A"])
allTransitions.append(self.pauseTransitions["/state_A"])
allTransitions.append(self.helpTransitions["/state_A"])
event = "exit:_/state_A"
for tr in allTransitions:
    if not (tr.enabled_event == None):
        event = (event + ("_" + tr.enabled_event.name))
allAttTuples = []
allAttTuples.append(["counter", self.counter])
self.saveEvent(event, self.getSimulatedTime(), allAttTuples)

```

The second part of the code, shown in Listing 5.10, only has one purpose. This purpose is to save the correct event for the final tracing log. To know why the state is being exited, all that state's transitions are checked to see which one has a currently *enabled_event*, just like it was done for all transitions leading to *State_Debug* previously.

- **State_Debug Exit Function** - This function is very simple and does not need to be shown as it simply calculates the *cumulativeDebugTime* variable using the formula described in 5.3.
- **State_Help Exit Function** - This function removes the timer added in *State_Help*'s enter function which expired automatically.
- **process_time_transitions**: This function is called in every state's enter function in which the state has outgoing transitions that are triggered by timers.

Listing 5.11: Process Time Transitions Function

```

def process_time_transitions(self, timers, state_name):
    iteration = 0
    chosen = None
    lowest = timers[0]
    for t in self.timedTransitions[state_name]:
        if lowest >= timers[iteration]:
            lowest = timers[iteration]
            chosen = t
        iteration = iteration + 1
    if iteration > 0:
        temp = Transition(self, chosen.source, chosen.targets)
        temp.setTrigger(Event("step", self.getInPortName("input")))
        temp.setAction(chosen.action)
        temp.setGuard(chosen.guard)
        if not self.listContains(self.createdTransitions[state_name], temp):

```

```
self.createdTransitions[state_name].append(temp)
chosen.source.addTransition(temp)
attrs = [s.name for s in chosen.targets]
print (...)
```

Its task is to generate a transition for the shortest timer out of the outgoing transitions from that state because this is the only transition that can happen. This generated transition is provided as a possible step for the user, which is why it can be triggered by the event **step**. Looking at its implementation, there exists a loop to determine the transition with the shortest timer. A transition is then generated with the same source, targets, action and guard as the one with the shortest timer. The difference is that this one is not triggered by that timer and can only be triggered with the **step** event.

- **process_event_transitions:** This function is called in every state's enter function, in which the state has outgoing transitions that are triggered by events. It is not worth displaying the code of this function as its task is simply to print the available transitions triggered by events, which means no new transitions are generated.
- **print_internal_state:** This function is called in every state's enter function after every user-defined task on that state is performed. The function shows the current value of every user-defined attribute of the class, and that is why the state's tasks must be performed before this, as those variables may suffer changes to their values during those tasks.
- **save_execution_trace:** This function is called on the enter function of *State_Final*. It is responsible for the creation of the file that contains the final trace log. It will get all the relevant information described in 5.3 and write it to the file, including all the events saved in the *tracedEvents* list.
- **save_event:** This function is called every time we want to save an event for the trace log during the execution. It creates a *TracedEvent* object containing the *event_name*, the current *timestamp* of the event and a list with all the user-defined attributes' current values. It then appends this object to the *tracedEvents* list.
- **list_contains:** This function is called by the function *process_time_transitions* when creating a new transition. It is used to check if the new transition that will be created already exists so it does not create a duplicate transition. This comparison cannot be done by simply comparing the objects, as this will return incorrect results.
- **guard functions:** Guard functions for the model's transitions may be generated the same way as before. However, with debugging mechanisms on the scene, guard functions take a more relevant role. Each transition that exits *State_Debug* needs to have a guard associated with it to confirm whether the currently active state is the target of this transition or not. This happens because all these transitions

are triggered by the same event, the **continue** event. This means that all of them are triggered when this event occurs. So, for the correct transition to be the only one being triggered, guard functions are added to each transition. Since several transitions are generated for different combinations of active states that may be active concurrently, a guard function is also generated for each of these combinations. An example of these guard functions is provided in Listing 5.12.

Listing 5.12: Guard Function Example

```
def continueGuard_state_A(self, parameters):  
    return list ( self . active_states .queue) == self . states ["/state_A"]
```

It is worth mentioning that the code shown in the snippets does not correspond exactly to the generated code, as that code may be too complex to show here, and the simpler version corresponds to a better and cleaner understanding of how it works.

5.5 Execution Flow

The execution of the model starts when the runner file is executed. Let us take as an example for this explanation the model in Figure 5.1 and put together all of the functions we have seen in the previous section 5.4. The execution begins the same way as in section 4.5, initialising any variables (including metric variables or data structures used during the simulation). It then calls the *setSimulationSpeed* function to set the correct simulation mode and speed scale factor for the execution and it builds the statechart structure, including *State_Debug*, *State_Final* and *State_Help*, their transitions and the data structures needed to store these objects. Subsequently, user-defined attributes are initialised, and user-defined constructors are called.

The execution starts at the defined default state, which is, in this example, *State_A*. Unlike in an execution without debugging mechanisms, here there are not 3 outgoing transitions from *State_A*, but 6. The same 3 transitions still remain: 2 time-based transitions (from *State_A* to *State_B* with a duration of 10 seconds and from *State_A* to *State_C* with a duration of 20 seconds) and 1 event-based transition (from *State_A* to *State_D*, triggered by the event **d**). However, 3 new transitions are added, all event-based: a transition from *State_A* to *State_Debug*, triggered by the event **pause**, a transition from *State_A* to *State_Final*, triggered by the event **stop** and a transition from *State_A* to *State_Help*, triggered by the event **help**. We will follow an execution flow which ends with the **stop** event.

Once *State_A* is entered, it will enter the first code branch of its enter function, shown in Listing 5.5, where it is placed in the *active_states* queue. The user-defined tasks are then performed. In this case, it is a simple function to increment a counter, *increment_counter*, setting the user-defined *counter* variable to 1. Afterwards, user-defined variable names and values are printed. Then, timers are added according to the type of simulation selected and stepping transitions are generated for time-based transitions. All available

event-based transitions are displayed. At this point of the execution, the output looks as shown in Figure 5.2.

```
Real-time Simulation
Scale Factor: 1.0
Type help to see the available commands.

Entered /state_A
counter: 1
Available Transition Options:
[time-based] type step to skip the transition to ['/state_B'] which has a duration of 10 seconds and the guard condition None
[event-based] type d to perform the transition to ['/state_D'] with the guard condition None
[ /state_A ] >
```

Figure 5.2: Output after entering *State_A*

Now, for this execution, 5 seconds pass, and the user sends the input event **pause** to the system. This triggers the transition to *State_Debug*. When exiting *State_A*, its exit function is called. In this function, the previously set timers are removed. Calculations for execution times are made, which will later be used to calculate new timers when the execution returns to *State_A*. Every debugging transition is searched to understand if any of them has been activated, whether it is a breakpoint transition or a regular pause transition. In this case, the pause transition from *State_A* to *State_Debug* has an enabled event, meaning it is the one that has been activated. Because of this, *State_A* is not removed from the *active_states* queue. *State_Debug* is entered. In its enter function, the boolean variable *firstTime* will be set to **False**, meaning that when *State_A* is entered again, it will go to its second code branch. Then, the debug prints are made, including the currently active states and every user-defined attribute's names and current values. At this point, the output looks as shown in Figure 5.3.

```
Real-time Simulation
Scale Factor: 1.0
Type help to see the available commands.

Entered /state_A
counter: 1
Available Transition Options:
[time-based] type step to skip the transition to ['/state_B'] which has a duration of 10 seconds and the guard condition None
[event-based] type d to perform the transition to ['/state_D'] with the guard condition None
[ /state_A ] > pause
DEBUG MODE
Current States: ['/state_A']
counter: 1
[/state_Debug] >
```

Figure 5.3: Output after entering *State_Debug*

When the user wishes to return to the execution, they send the input event **continue** to the system. *State_Debug* is exited, and its exit function is called, in which the enabled event of the pause transition that was triggered before is set to **None**. *State_A* is entered again, entering its second code branch, shown in Listing 5.6. As 5 seconds had passed before the **pause** event, new timers are now added with the remaining time left (for example, the transition from *State_A* to *State_B* has a timer of 10 seconds associated to it; as 5 seconds passed, then the new timer will have a value $10 - 5 = 5$ seconds).

Subsequently, 5 more seconds pass and the transition to *State_B* is triggered. *State_A* is exited and this time, *State_A* is removed from the *active_states* queue in its exit function. *State_B* is entered, and its enter function is called.

The execution follows this pattern until the user sends the input event **stop** to the system. The currently active state is exited, and *State_Final* is entered, stopping the controller and saving the execution trace. It then finishes with an *exit* call. At any point during the execution, the user may look at the help menu, illustrated in Figure 5.4.

```
[ /state_A ] > help
HELP - Available Commands:
1. pause - Pauses the execution.
2. continue - Continue the execution if it is paused.
3. stop - If there exists a time-based transition, this command will skip it.
4. stop - Stops the execution completely and saves a trace with information about the simulation.
5. Possible events to simulate are displayed at the arrival of each state if they are available.
6. To change the Simulation Type and its Scale Factor, use the flags -s and -f, respectively, when executing the generated file.
7. The Simulation Type, -s may have the following values: 0 = Real-Time Simulation; 1 = Scaled Real-Time Simulation; 2 = As-fast-as-possible Simulation.
8. When using the Scaled Real-Time Simulation, a Scale Factor, -f may be added. Its value may be any number > 0.
9. To add a breakpoint, edit the breakpoints.xml file directly.
[ /state_A ] >
```

Figure 5.4: Help Menu

5.6 Usage Scenarios

This tool was continuously tested and used during the development as new features were gradually implemented. Different models were used for different purposes, such as trying new features or testing the compatibility with certain statechart elements. Most of those models are based on the same model, the *Basic Counter*, represented in Listing I.1 and Figure 5.5, to which they add some complexity. Now, each model will be shown as an image, and its purpose will be explained, while each model’s SCXML format will be available in Appendix I.

The Basic Counter, shown in Figure 5.5, was used to explore the existing code base of SCCD and implement the Play/Pause feature. It tested some basic state chart elements and was used mostly as a proof of concept. Its SCXML version is available in the Listing I.1.

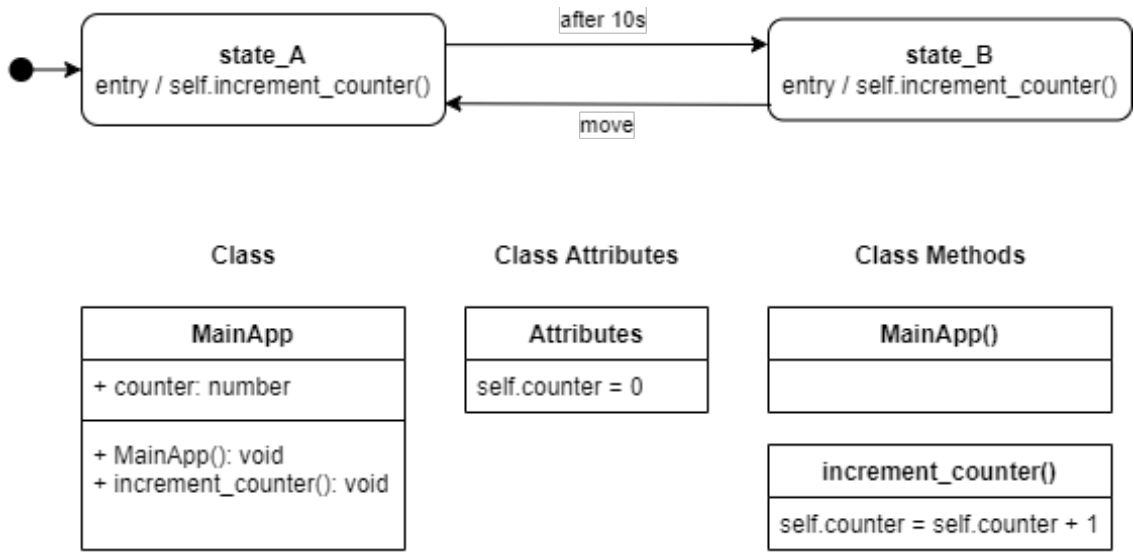


Figure 5.5: Basic Counter

The Counter with Composite States, shown in Figure 5.6, was used to test and implement any features that required special attention with composite states, considering

parent and child states may have different entry and exit functions. Its [SCXML](#) version is available in the Listing [I.2](#).

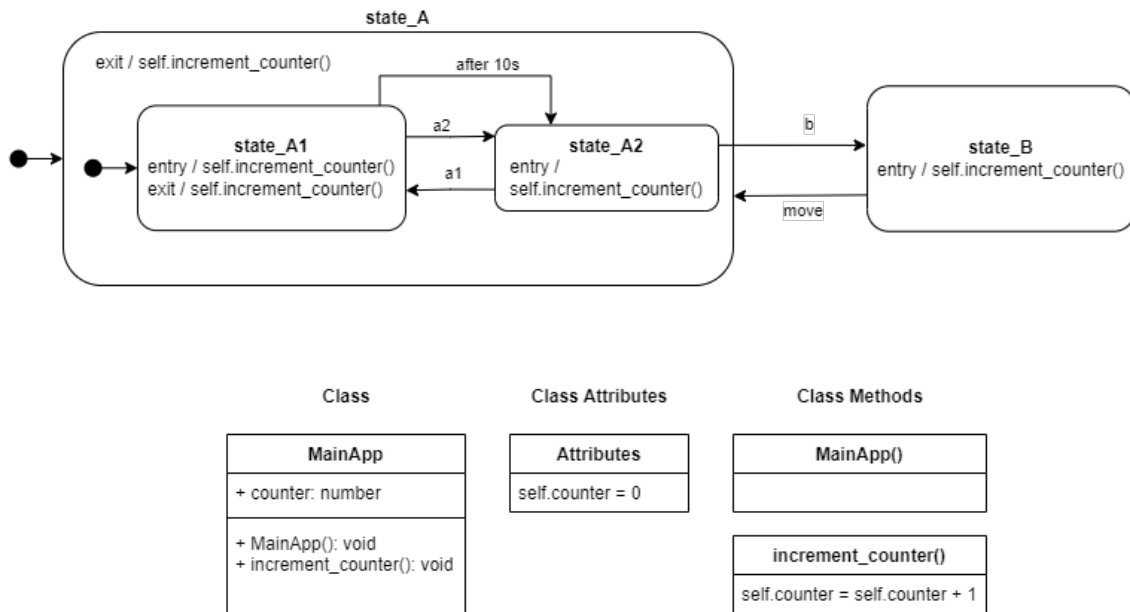


Figure 5.6: Counter with Composite States

The Counter with History State, shown in Figure [5.7](#), was used to test the behaviour of history states with the debugger. Its [SCXML](#) version is available in the Listing [I.3](#).

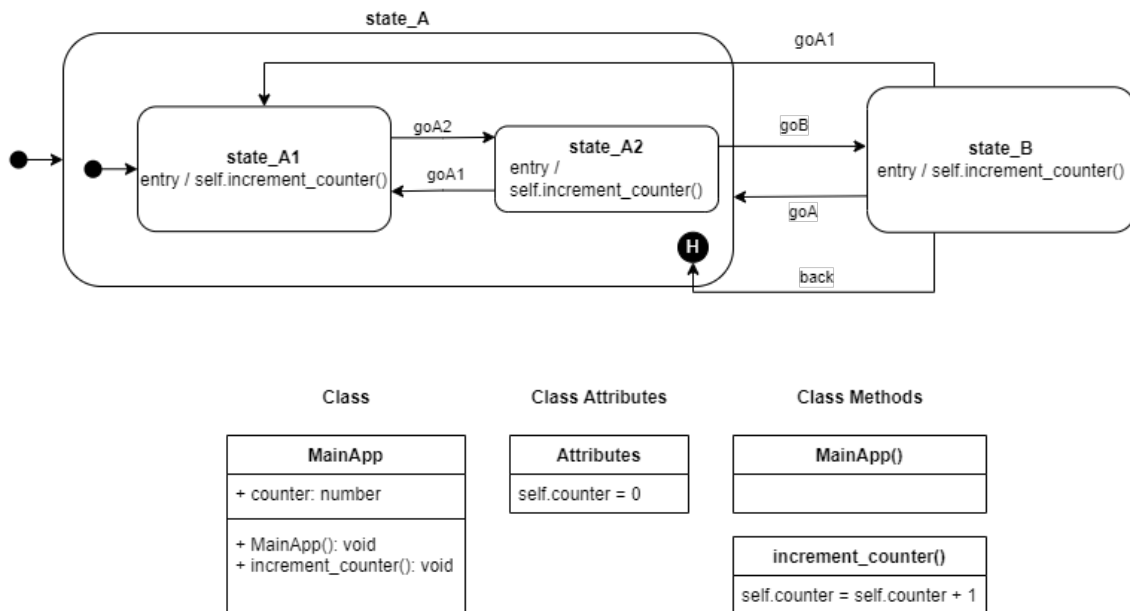


Figure 5.7: Counter with History State

The Counter with Multiple States, shown in Figure [5.8](#), was used to test and implement most features as it is a model with both event and time-based transitions and with more than 2 states so it allowed to test different execution complexities while making it easy to understand what was happening. Its [SCXML](#) version is available in the Listing [I.4](#).

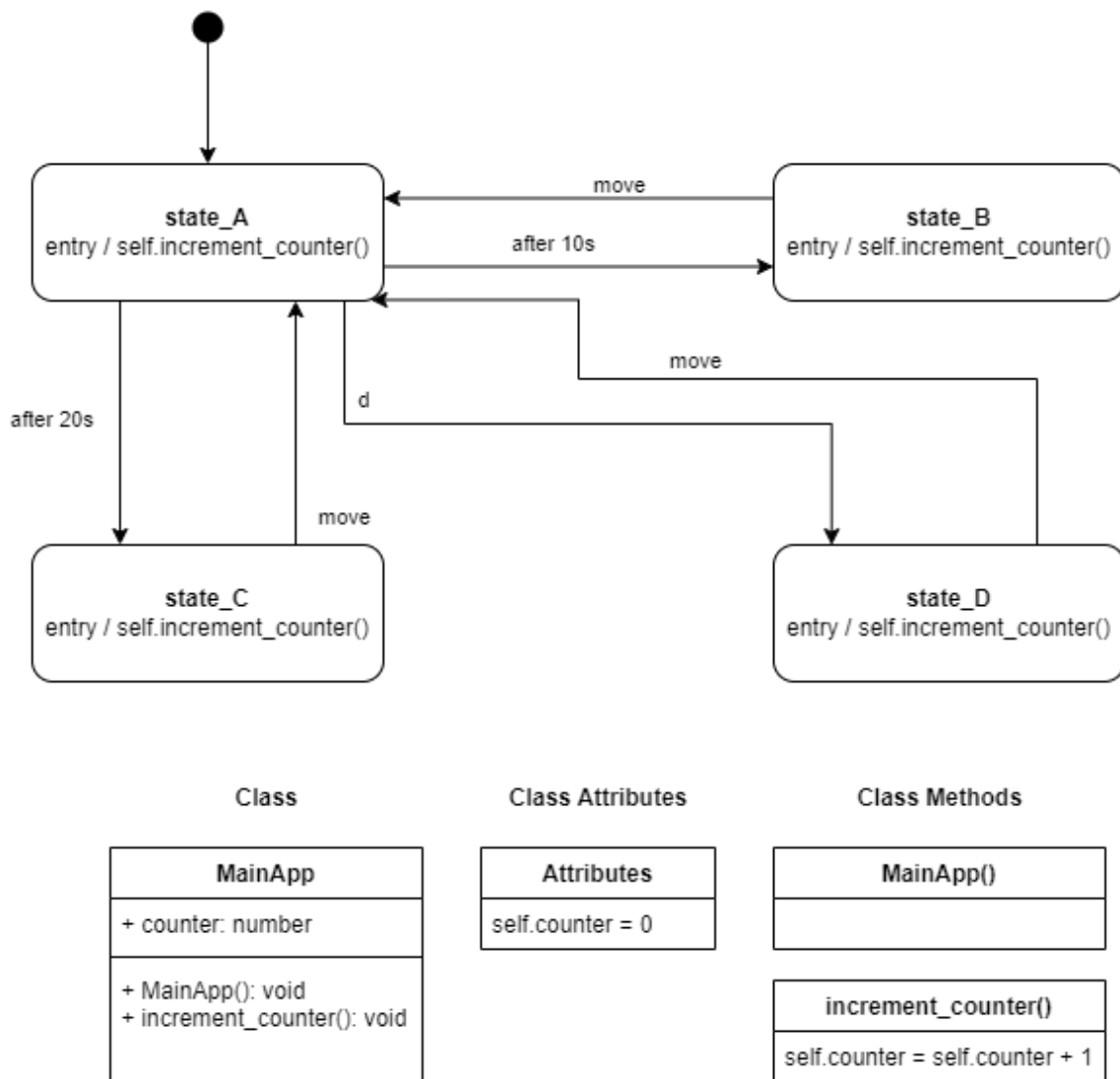


Figure 5.8: Counter with Multiple States

The Counter with Parallel States, shown in Figure 5.9, was used to change any implementation aspects that had not considered parallel states. The implementation suffered some big changes due to tests made with this model, as the debugger did not work for parallel states initially. Its SCXML version is available in the Listing I.5.

The Counter with Scripts, shown in Figure 5.10, was used to check whether scripts added to entry and exit functions and transitions were effectively being reflected in the execution state after the addition of the debugger's implementation. Its SCXML version is available in the Listing I.6.

The Phone, shown in Figure 5.11, was used for user tests and to combine different statechart elements, increasing the complexity level and putting the debugger to test in a more chaotic environment. This model was based on the model used to test other statechart simulation tools, represented in Figure 3.4. Its SCXML version is available in the Listing I.7.

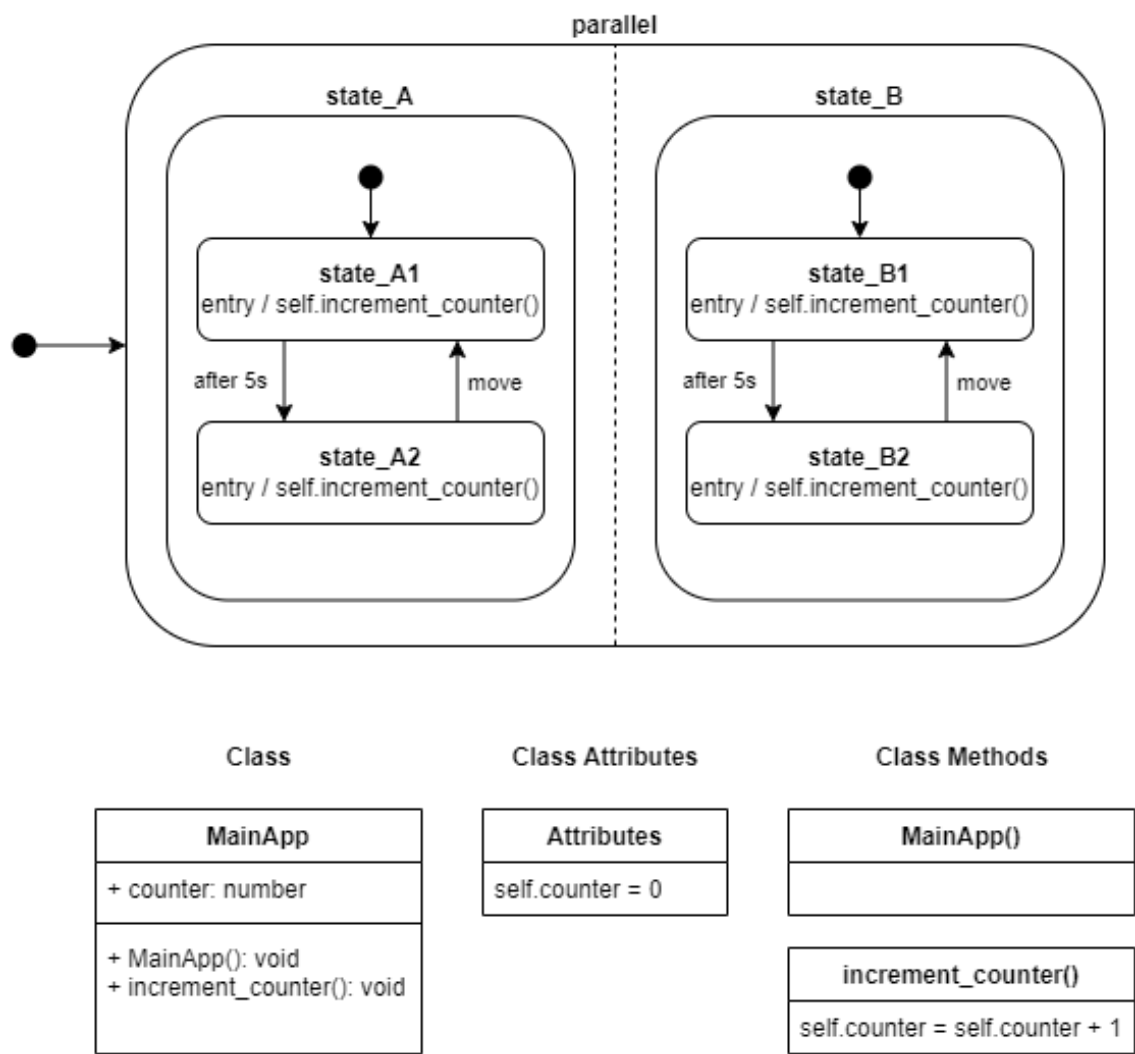


Figure 5.9: Counter with Parallel States

The Task5 model, shown in Figure 5.12, was also used for user tests, and it is a model that can be quite educational and is often used when teaching students about state machines and statecharts. Its [SCXML](#) version is available in the Listing 1.8.

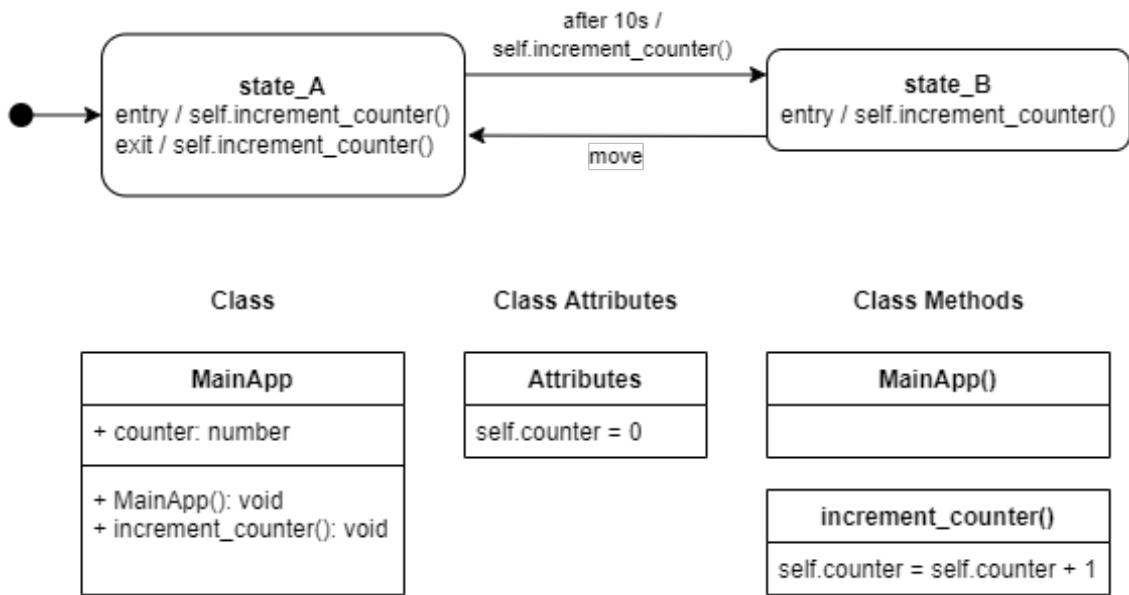
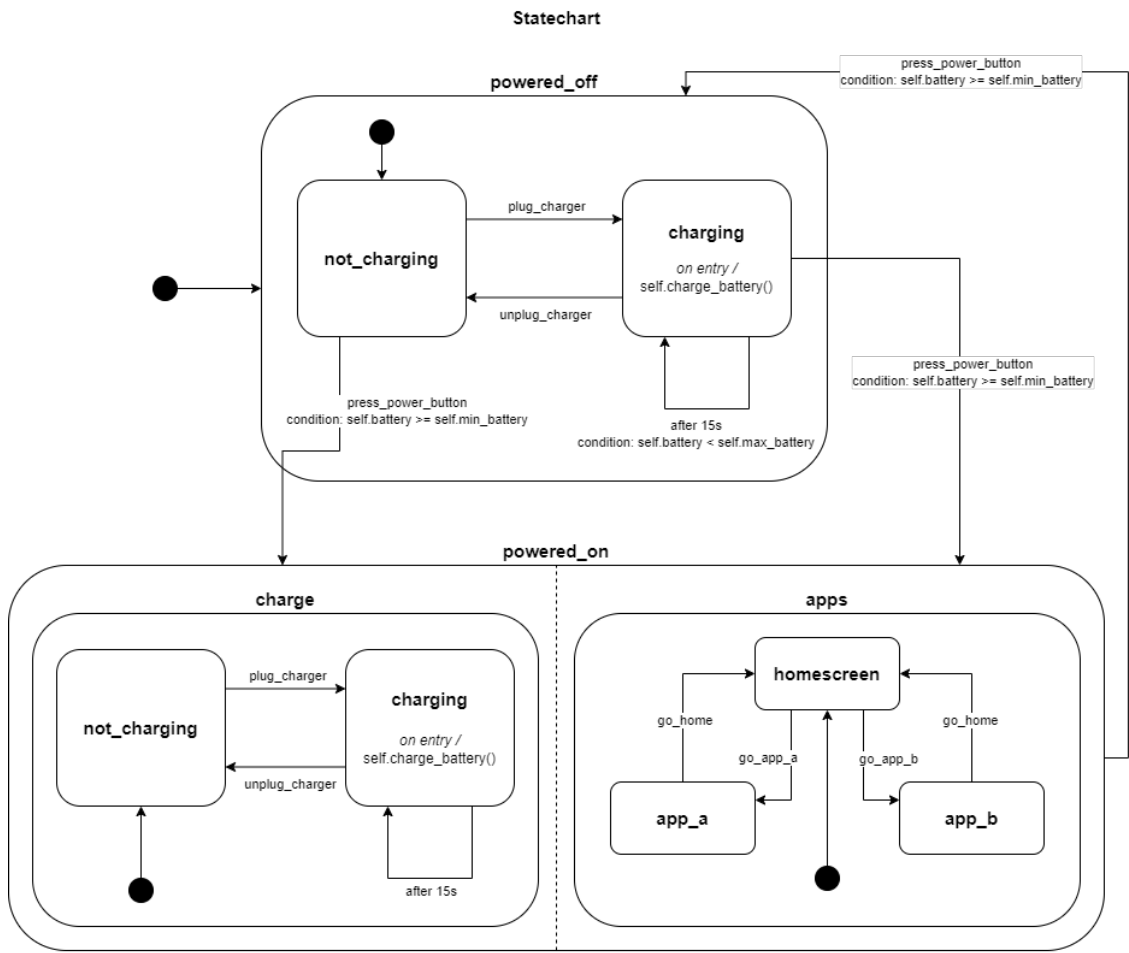


Figure 5.10: Counter with Scripts



(The dashed line represents the division between 2 parallel regions)

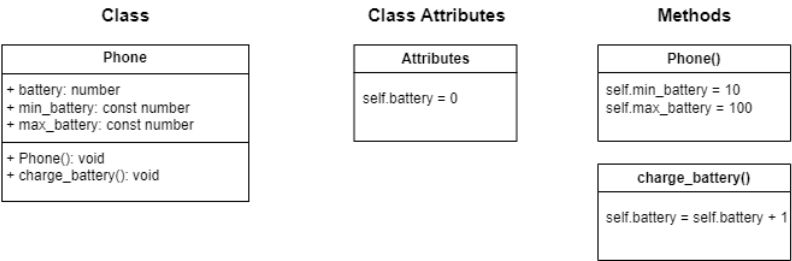


Figure 5.11: Phone

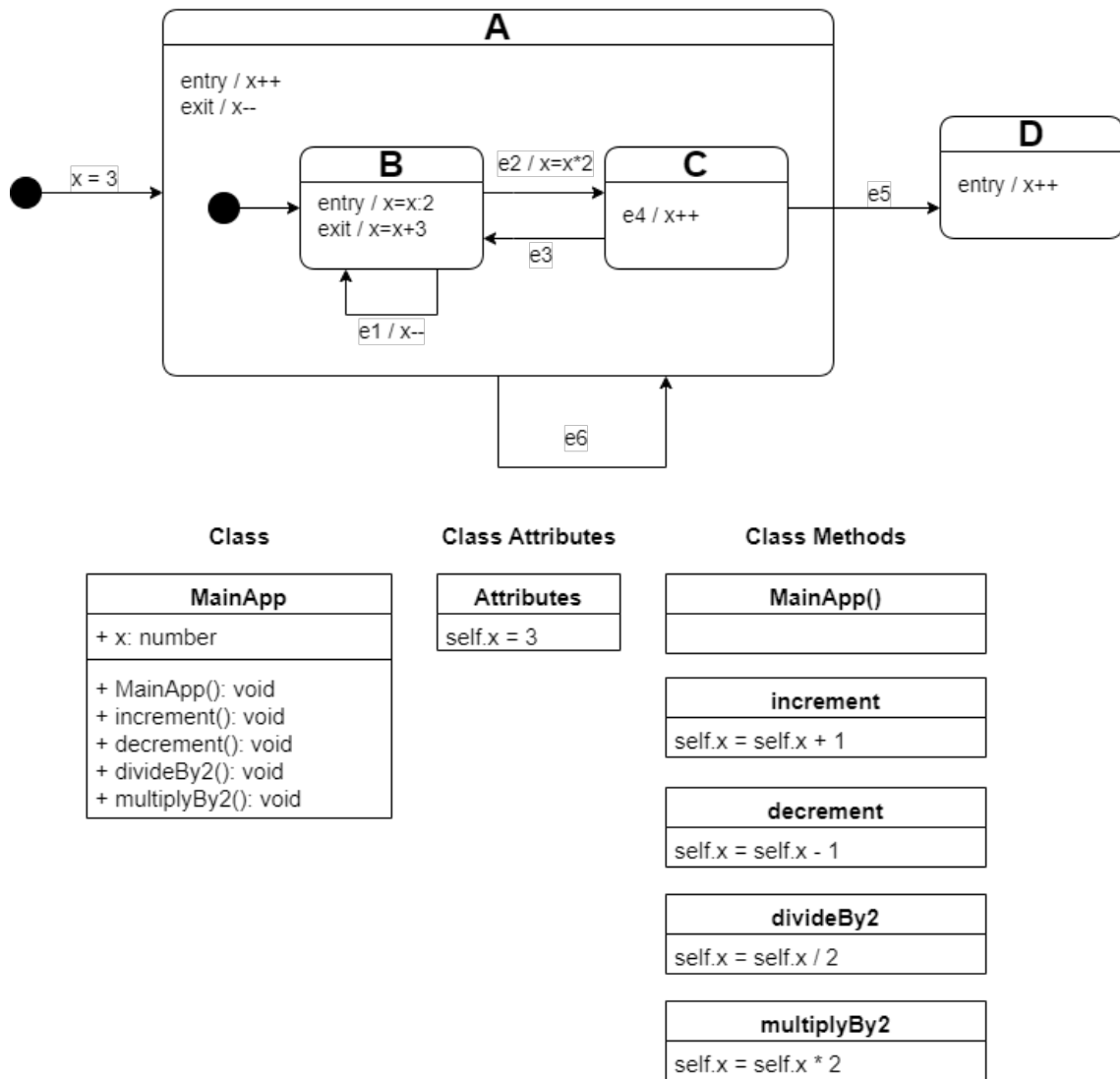


Figure 5.12: User Tests Task 5

EVALUATION

In this chapter, we will discuss how the implemented solution was tested and evaluated and what conclusions we may draw from the results of those tests.

6.1 Evaluation Planning

To perform a controlled testing evaluation, it is advised to build an evaluation plan. As such, some questions should be answered from the start:

6.1.1 What is the evaluation's goal?

To define the concrete goals of this evaluation, we will use the *Goal-Question-Metric Approach* [6].

Table 6.1: Goal-Question-Metric

Evaluation Goal	Question	Metric
SCCD Debugger's Usability	How usable is the SCCD Debugger?	SUS score
SCCD Debugger's Task Load	How much effort does a user need to use the SCCD Debugger?	NASA-TLX score
SCCD Debugger's Usage Precision	With how much precision can a user utilise the SCCD Debugger?	Precision score
SCCD Debugger's Usage Recall	With how much recall can a user utilise the SCCD Debugger?	Recall score

6.1.2 Who are the experimental units?

The testers should be Computer Science students who are completing or have already obtained their degrees. They are expected to have some prior knowledge of Statecharts and Class Diagrams.

6.1.3 What were the experimental materials used?

As for the materials used during the evaluation, several can be mentioned. The tester is provided with a document fully represented in Appendix I. This document contains some information regarding the tool that will be tested, including a general view of how it works and its objective. It also informs the tester about the objective of the evaluation as well as its conditions. The document explains what features of the tool will be tested and provides the tester with a small installation guide. All the tasks that must be executed using the tool are also described in this document.

Alongside this document, the tester is also handed an online form containing some questions for them to answer. This form is fully represented in Appendix I. The sets of questions asked are the following:

- **Demographic Questions** - There is a set of demographic questions about the tester, enquiring about their age, gender, number of monitors used to perform the evaluation, highest obtained academic degree (or ongoing), academic background, experience with Statecharts, Class Diagrams and Statechart simulation tools.
- **System Usability Questions** - This set of questions is based on the standard [System Usability Scale \(SUS\)](#) [7]. This scale is used to objectively evaluate usability by creating a standard way to easily and cheaply assess a system's usability, as this is a quality without absolute and concrete measures. The [SUS](#) is composed of 10 statements which the tester may grade from 1 to 5 (where 1 means the tester strongly disagrees with the statement and 5 means the tester strongly agrees with the statement). To calculate the [SUS](#) score, we must first sum the score contributions from each statement. Each statement's score contribution will range from 0 to 4. For statements 1, 3, 5, 7 and 9, the score contribution is the scale position minus 1. For items 2, 4, 6, 8 and 10, the contribution is 5 minus the scale position. Then, we must multiply the sum of the scores by 2.5 to obtain the final usability score, which will range from 0 to 100, where a higher score corresponds to theoretically higher usability. A picture depicting the standard questionnaire used by [SUS](#) is presented in Figure 6.1.
- **Task Load Questions** - This set of questions aims to evaluate the workload of the tasks given to the tester. For this, a standard evaluation method is also used. We use the [NASA-Task Load Index \(TLX\)](#) [28], which evaluates the tester on 6 different sub-scales: mental demand, physical demand, temporal demand, performance, effort and frustration. The tester may evaluate each of these 6 dimensions from 1 to 100, where 100 is the worst theoretical value we may consider for each dimension. Originally, from these 100, the enquired tester may select 20 different values, where the interval between each value is 5 (5, 10, 15, and so on until 100). Just like [SUS](#), [NASA-TLX](#) also calculates a final score. This score is based on the mean of the values of the 6 dimensions, and this mean may be weighted or unweighted. If it

	Strongly disagree				Strongly agree
1. I think that I would like to use this system frequently	☐	☐	☐	☐	☐
	1	2	3	4	5
2. I found the system unnecessarily complex	☐	☐	☐	☐	☐
	1	2	3	4	5
3. I thought the system was easy to use	☐	☐	☐	☐	☐
	1	2	3	4	5
4. I think that I would need the support of a technical person to be able to use this system	☐	☐	☐	☐	☐
	1	2	3	4	5
5. I found the various functions in this system were well integrated	☐	☐	☐	☐	☐
	1	2	3	4	5
6. I thought there was too much inconsistency in this system	☐	☐	☐	☐	☐
	1	2	3	4	5
7. I would imagine that most people would learn to use this system very quickly	☐	☐	☐	☐	☐
	1	2	3	4	5
8. I found the system very cumbersome to use	☐	☐	☐	☐	☐
	1	2	3	4	5
9. I felt very confident using the system	☐	☐	☐	☐	☐
	1	2	3	4	5
10. I needed to learn a lot of things before I could get going with this system	☐	☐	☐	☐	☐
	1	2	3	4	5

Figure 6.1: SUS standard questionnaire; image taken from [7]

is weighted, the weights are calculated by presenting 15 pair combinations of the 6 dimensions where the tester must select the one that contributes the most to the task's workload. The number of times each dimension is selected is their weight. In this specific evaluation, we calculate an unweighted mean, and the tester may only select values from 1 to 10. The values are then converted to the original NASA-TLX scale. A picture depicting the standard questionnaire used and a sample question for each dimension is presented in Figure 6.2.

- **Task and Tool Questions** - The tester is presented with a set of tasks to perform

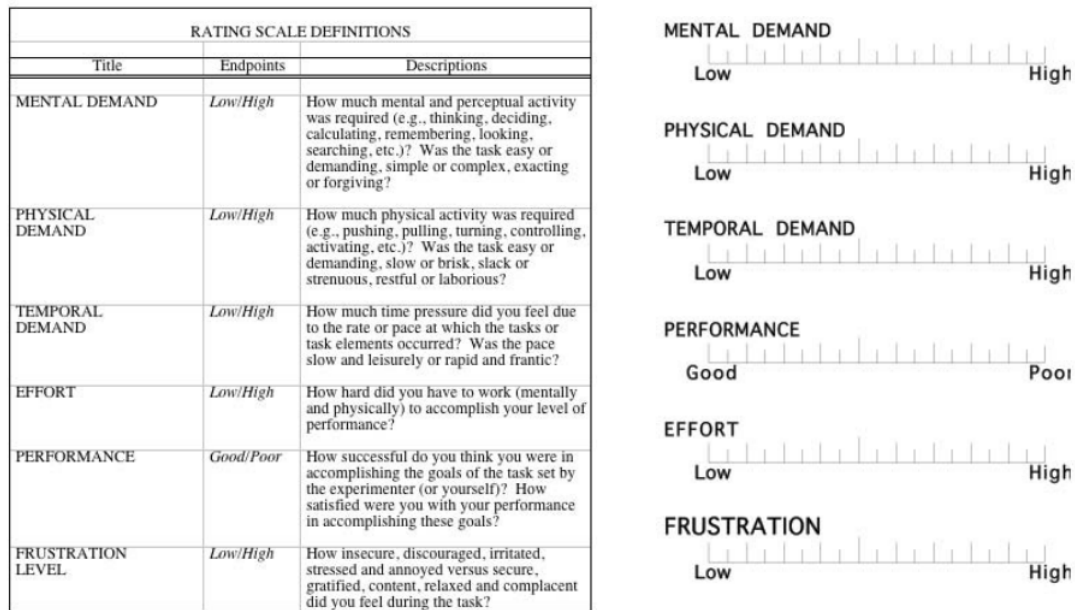


Figure 6.2: NASA-TLX standard questionnaire; image taken from [27]

using the tool, and for each task, a few questions are asked regarding the tool. These questions enquire the tester about the clarity of the help menu, the produced execution trace log and the overall interface. The tester is also asked about how easy it is to perform some of the tasks, how useful the tester thinks some features can be, and about potential new features or feature improvements that can be made in the current implementation. Some small questions about each task are also made to see if the tester understood the tool and task correctly.

The whole evaluation is performed on a laptop with both the tester and the researcher in an online meeting. The tester performs the tasks while the researcher observes and records the whole experience.

Precision and recall will also be measured using the recordings of each user test. Precision is the fraction of correctly performed steps among all steps performed by the user, whereas recall is the fraction of correctly performed steps among the necessary steps to complete a task.

6.1.4 What tasks do the testers have to perform?

The testers must perform 5 different tasks, where each task aims to test different features or situations. These tasks are:

- **Task 1** - This task provides the tester with the model depicted in Figure 6.3 which will be used from tasks 1 to 4.

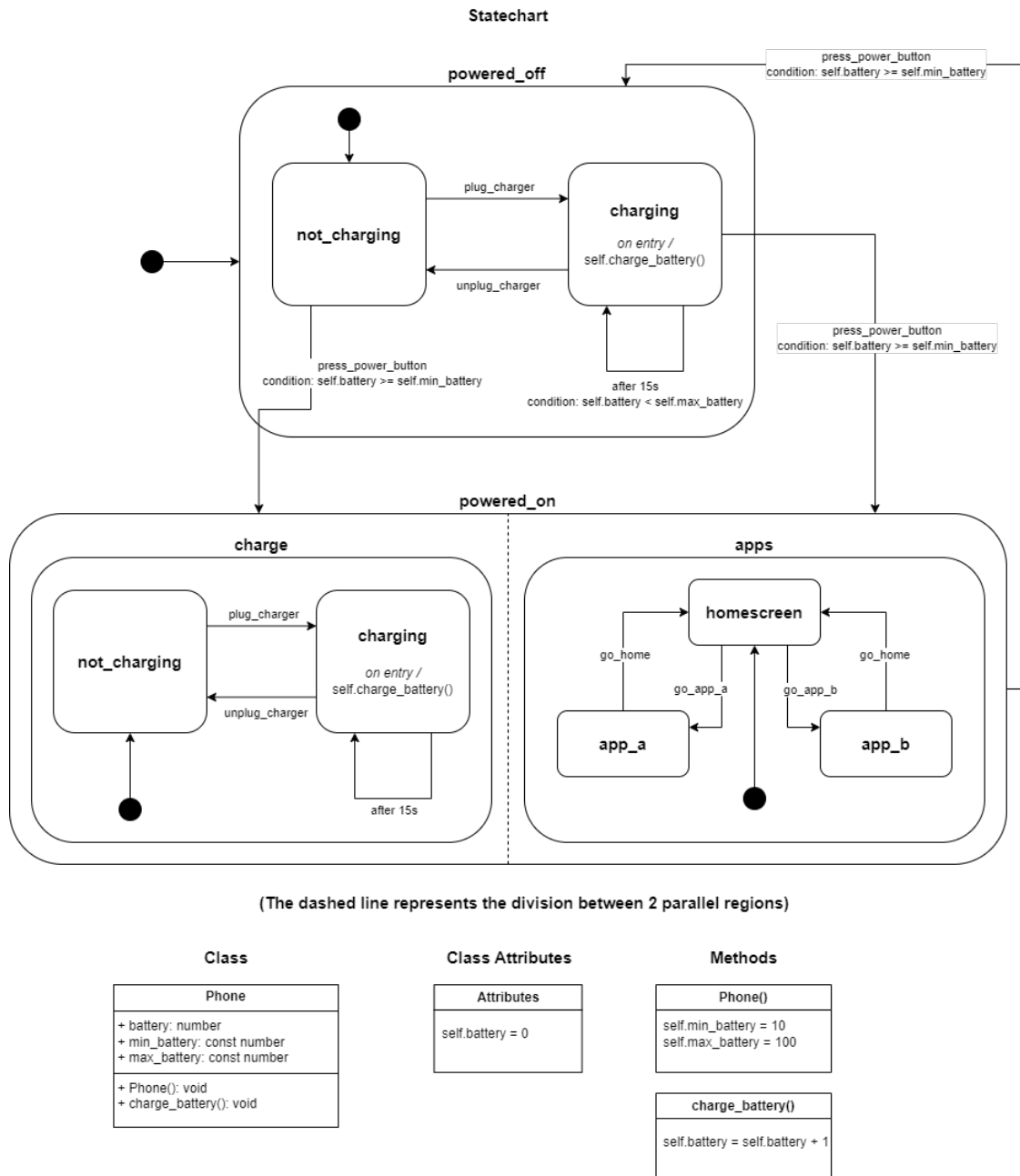


Figure 6.3: Test Model 1

This task, alongside tasks 2 and 3, gives more detailed instructions to make the tester more comfortable with the tool. The tester is instructed to generate code for the given model and start the simulation using that code. The tester is advised to open the help menu and observe the available commands. The tester is commanded to send in the *plug_charger* event to move on to a different state where time-based transitions will happen. This way, the tester has contact with both transition types that may occur. Then, the tester is instructed to pause, unpaue and stop the simulation so they can understand how some of the most fundamental features work. To finish,

the tester is instructed to open and read the execution trace log.

- **Task 2** - This task introduces the different simulation modes available and requests the tester to use the **Scaled Real-Time Simulation** and make the simulation 2x faster. The tester is instructed to send the *plug_charger* event once again so they may observe the execution of time-based transitions happening faster than before. To finish, the tester is once again instructed to open and read the execution trace log.
- **Task 3** - This task introduces breakpoints and steps. The tester is asked to change back the simulation mode to **Real-Time Simulation** and to place a breakpoint on the */powered_off/charging* state. As the tester arrives at this state, the execution pauses due to the breakpoint. Then, the tester is instructed to use the step feature and skip transitions using steps. As the tester tries the step on this state, they may notice the execution will pause every time as the transition being simulated with the step is from the */powered_off/charging* state with the breakpoint back to itself, as it is possible to observe in Figure 6.3.
- **Task 4** - This task requests the tester to use the **As-fast-as-possible Simulation** mode to fully charge the phone of the model and open *app A*. Instructions are more generic, as at this point, the tester should be more familiar with the tool, and we can now test them properly.
- **Task 5** - This task uses the model shown in Figure 6.4.

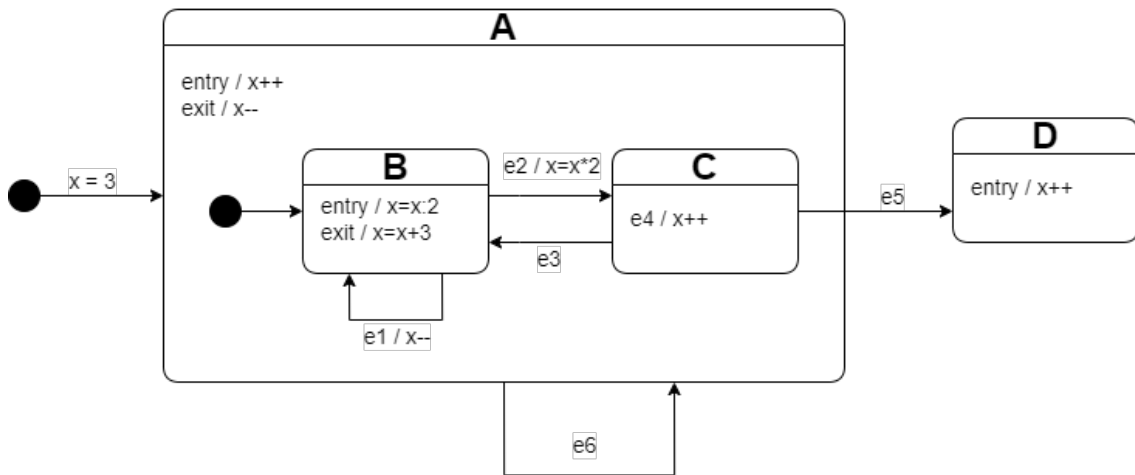


Figure 6.4: Test Model 2

The tester is then asked to answer some questions about the model using the tool. The questions are:

- What is the value of x after the occurrence of the event chain **e2, e3, e2, e4, e5** ? (to which the answer is 17)
- What was the value of x during the occurrence of the event chain of the previous question right after event **e3** ? (to which the answer is 5)

Table 6.2: Test Model 2 Detailed Event Chain

Event	State	Action	Value of x
		start	3
	A	entry of A	4
	A/B	entry of B	2
e2	A/C	exit of B $x = x * 2$	5 10
e3	A/B	entry of B	5
e2	A/C	exit of B $x = x * 2$	8 16
e4		$x++$ in C	17
e5		exit of A	16
	D	entry of D	17

Every value of x during the execution of the event chain is shown in Table 6.2.

6.1.5 What were the hypotheses, parameters and variables?

The single parameter that was used was the SCCD Debugger itself.

6.1.6 What evaluation design was used?

The design was non-experimental, as the participants only tested the SCCD Debugger, and no comparison was made with other tools.

6.2 Execution

6.2.1 Procedure

The evaluation procedure had the following structure: each tester joined an online meeting and started a screen share session. Initially, the tester read the document provided as a PDF in the Appendix I until they reached the tasks. The researcher could answer questions regarding the tool and its installation. After reading the document, the testers could proceed to the online form to fill in their demographic information and start task 1. From here, the researcher could only step in if there were doubts about the models or if there was an unforeseen issue. The online form collected all the information about the participants as well as their feedback about the tool and answers to the task questions or usability and task load questionnaires. It is fully represented in Appendix I. To build these forms, *Google Forms* [19] was used.

6.2.2 Deviations

The whole evaluation phase went smoothly, according to expectations, except for the first tester, who served as a pilot for the evaluation and encountered an unexpected bug:

breakpoints were not working as intended. This bug was instantly fixed, and everything else occurred as expected.

6.3 Analysis

We may divide the analysis of the obtained data into 2 different groups: quantitative analysis, which will focus on describing the numerical data using charts and some general statistics, and qualitative analysis, which will focus on describing the written or voiced feedback given during the user tests.

6.3.1 Quantitative Analysis

6.3.1.1 Descriptive Statistics

In this section, we will take a look at some general statistics and charts regarding the data collected during the user tests.

The demographic information retrieved regarding the participants in the evaluation is represented in Figure 6.5, containing the following sub-figures: Figure 6.5a - Age, Figure 6.5b - Gender, Figure 6.5c - Number of Monitors used in the Test, Figure 6.5d - Experience with Statecharts and Figure 6.5e - Experience with Class Diagrams. A total of 10 testers participated in this user testing evaluation.

The participants were all in the process of concluding their Master's Degree in Computer Science or had already obtained their degree in that field. All of their experience with Statecharts and Class Diagrams was exclusively derived from academic usage. None of them had prior experience with Statechart simulation tools.

In Table 6.3, we can observe several different statistics retrieved from the usability and task load questionnaires. All the values on the table were calculated based on the 10 scores of each participant for each of the tests.

Table 6.3: User Test Usability and Task Load Scores

Test	Mean	Std Deviation	Min	25%	50%	75%	Max
SUS	76.5	16.6	45.0	65.0	80.0	89.4	97.5
NASA-TLX	24.8	7.3	15.0	19.2	24.2	29.6	38.3

In Table 6.4, we can observe different precision statistics of the participants regarding each of the 5 tasks they performed, whereas, in Table 6.5, we may observe different recall statistics of the participants regarding the same 5 tasks. These values were calculated using the recordings made of the user tests and, for each task, defining the necessary steps for its successful completion and what steps could be considered errors and, therefore, decrease precision. Consulting the help menu, for example, was not considered a mistake. Most mistakes were derived from using commands that did not exist or inappropriately using the ones that did exist.

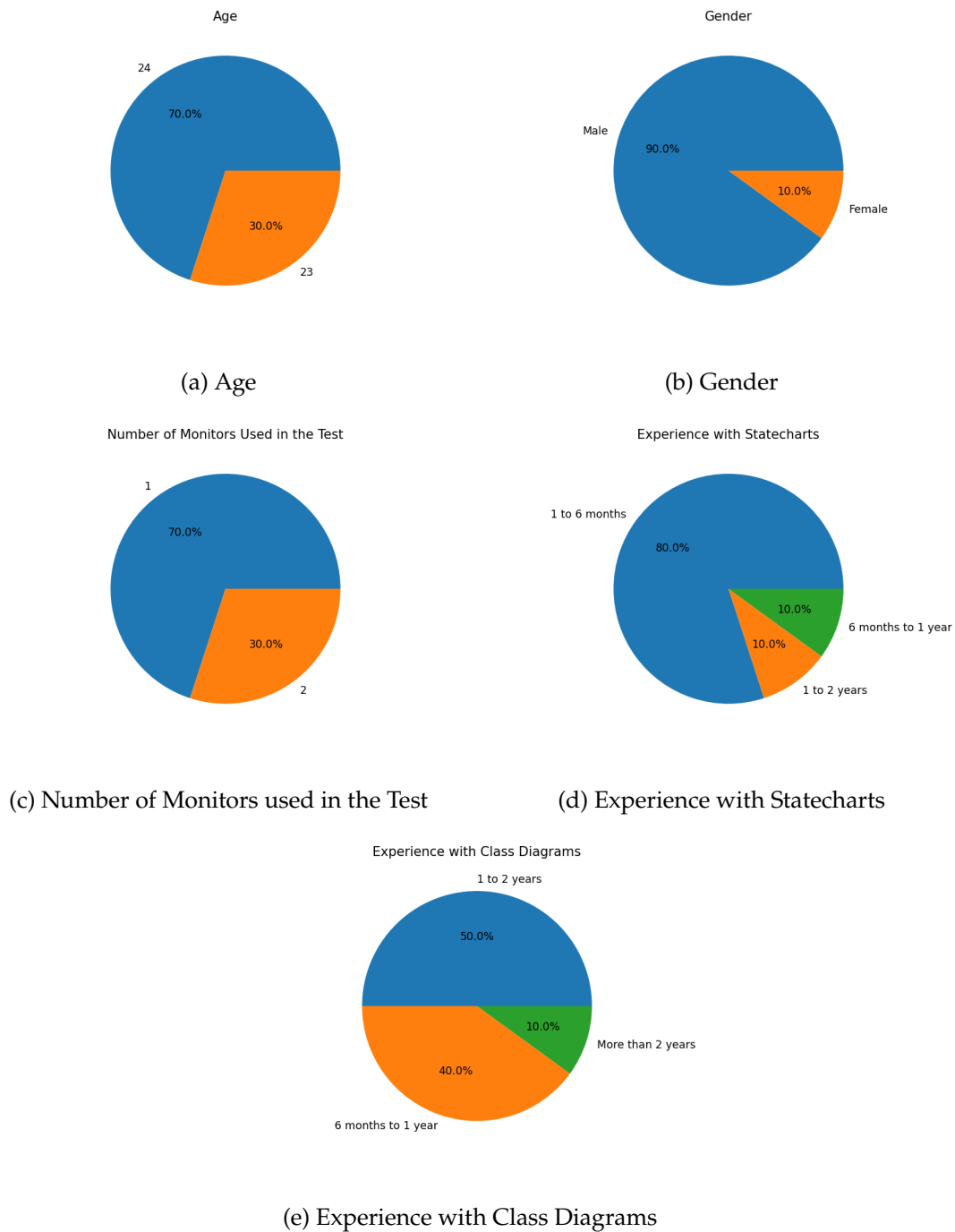


Figure 6.5: Demographic Statistics

In Figure 6.6, we may observe a mean of the given scores as to how easy it was to understand and use each of the 5 mentioned features. From this bar chart, we may conclude that most testers thought that changing command line arguments to change the simulation mode was quite simple. However, placing breakpoints and using steps were the least easy features to use and understand, corresponding to the observations and feedback obtained during the tests. All of the scores still boast a quite high value.

Table 6.4: Precision Statistics per Task

Task Number	Mean	Std Deviation	Min	25%	50%	75%	Max
Task 1	97.89	4.46	88.89	100.00	100.00	100.00	100.00
Task 2	98.33	5.27	83.33	100.00	100.00	100.00	100.00
Task 3	88.25	9.74	75.00	81.82	89.45	97.50	100.00
Task 4	95.75	5.53	87.50	90.00	100.00	100.00	100.00
Task 5	98.09	4.03	90.00	100.00	100.00	100.00	100.00

Table 6.5: Recall Statistics per Task

Task Number	Mean	Std Deviation	Min	25%	50%	75%	Max
Task 1	98.89	3.51	88.89	100.00	100.00	100.00	100.00
Task 2	100.00	0.00	100.00	100.00	100.00	100.00	100.00
Task 3	98.89	3.51	88.89	100.00	100.00	100.00	100.00
Task 4	95.56	9.39	77.78	100.00	100.00	100.00	100.00
Task 5	99.00	3.16	90.00	100.00	100.00	100.00	100.00

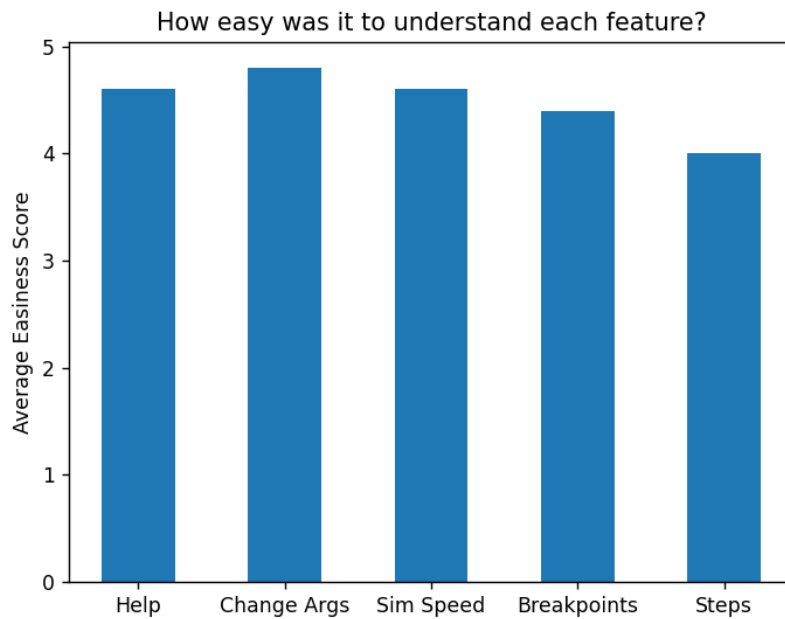


Figure 6.6: Average Ease in Understanding each Feature

On the user test forms, the testers were enquired about the usefulness of each simulation mode, excluding the default real-time simulation mode. Both modes, scaled real-time simulation and as-fast-as-possible simulation, were deemed very useful by most testers, each of them averaging the same score, as seen in Figure 6.7. Most of the testers thought the 3 provided simulation modes were enough for the tool.

In Figure 6.8, we can observe an evolution of the ease of reading and understanding the trace log of an execution. This question was asked at the end of all tasks (except for task 5) for 2 reasons: to evaluate how fast the testers would get used to reading that trace

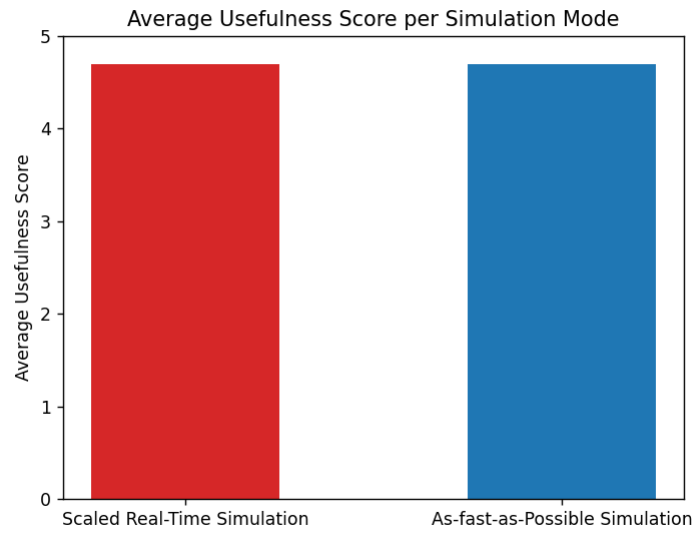


Figure 6.7: Average Usefulness Score per Simulation Mode

log and to evaluate if different operations described in the log would result in a harder comprehension.

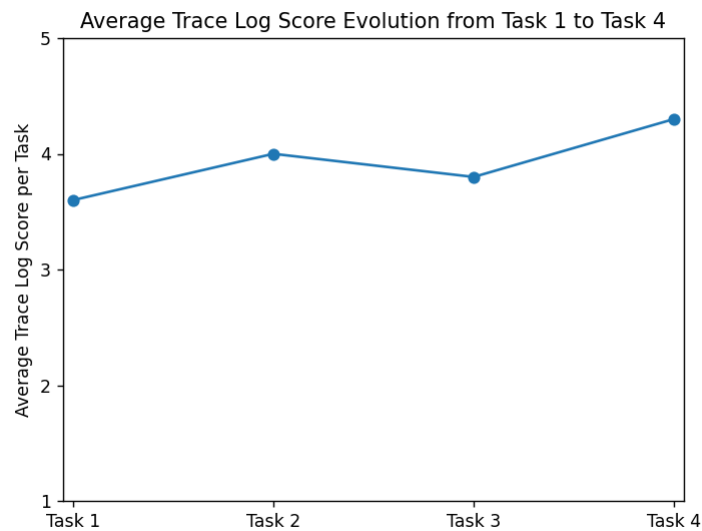


Figure 6.8: Average Trace Log Score Evolution from Task 1 to Task 4

In Figure 6.9, we can observe the answers given to the 2 questions in task 5, explained in section 6.1.4. The answer to the first question was 17, to which everyone answered correctly. However, for the second question, the correct answer was 5, but there was one wrong answer. This question asked about the value of a variable during the execution, so not the final value. The only participant who gave a wrong answer here used a different method to get to their answer. While everyone else scrolled through the terminal to check

the variable's value, this participant used the trace log to determine their answer.

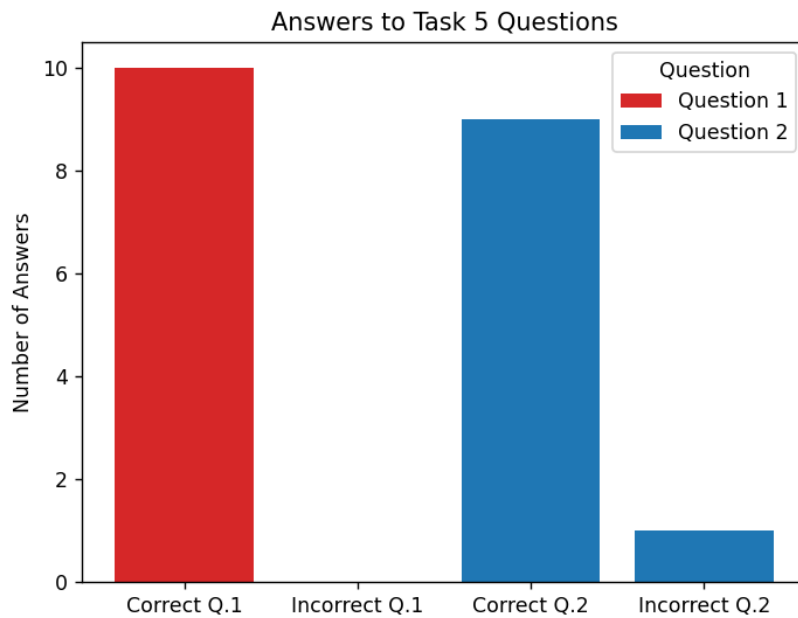


Figure 6.9: Answers to Task 5 Questions

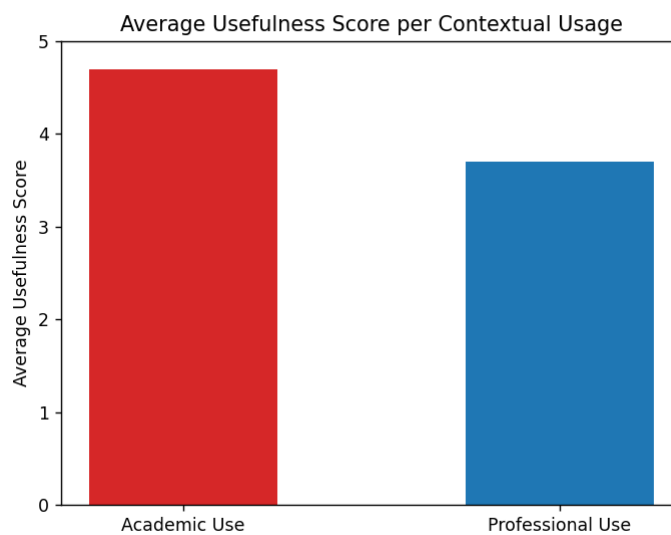


Figure 6.10: Average Usefulness Score per Context Usage

Participants were enquired about the possible usefulness of this tool according to the context in which it was used. The results shown in Figure 6.10 allow us to conclude that participants believe this tool may be more useful for students learning statecharts, for example, than if it is used in professional situations using more complex models. However, both usage contexts still boast some fairly high scores. The comparison of the scores and

also the feedback obtained during the tests just prove that the tool seems to be perceived as more useful for less complex models. It is possible to speculate as to why this may be. Perhaps, there is some unease regarding the execution of complex models at the console level as the terminal could get cluttered with many available events and parallel states and other additional elements. This may have to do with the sample of participants that were chosen to test this tool who may not have as much affinity with the console as other potential users of the tool. Regardless, it is a valid point of concern.

6.3.1.2 Data Set Preparation

The data was handled using Pandas [47], a Python Data Analysis Library. This helped select the data for each chart and perform any needed calculations, such as the [SUS](#) and the [NASA-TLX](#) scores. Matplotlib [55] was used to build all of the presented charts.

6.3.2 Qualitative Analysis

There were open questions on the forms filled by the testers, which mostly asked for their opinion and feedback on possible improvements or new features that could be added to the [SCCD](#) Debugger. This feedback will now be divided into different improvement areas:

- **Trace Log** - A lot of feedback was given about the trace log. Several participants failed to understand the representation of the events, asking for a "clearer distinction between events". Time-based events are represented by `_xafter`, where `x` is an integer equal to or higher than 0, and most testers did not understand this. Similarly, these expressions also represented breakpoints, and the testers suggested specifying the event name with "breakpoint". One tester also suggested lining up the structure of the trace log so it is clearer to read. It was also suggested to "organise information by timestamps", showing each timestamp just once and then showing every event that happened on that timestamp. The timestamps were not immediately clear for one tester, who suggested using smaller numbers or different time units to represent them, as *ms* would make the numbers too big. One tester also suggested showing the event name before the timestamp on each line. Event names are shown when exiting a state in the trace log and one tester suggested these could also be represented on the entry event of the new state. A recommendation was also made for the speed scale factor to be shown in the trace log. From a structural point of view, one tester suggested having a small explanation of the template used for each line of the trace log so the users may understand it better, and another tester suggested exploring a different way to represent this, such as spreadsheets.
- **Help Menu** - In the help menu, a common issue was the lack of a distinction between commands and program concepts. Commands were represented in a darker yellow tone, but so were some program concepts, which caused some testers to struggle to distinguish what keywords were actual commands.

- **Breakpoints** - Some testers suggested new possible breakpoint types, such as breakpoints with **and** conditions, joining two different breakpoint conditions. It was also suggested that a breakpoint with a state and a time value would be created, triggering some specified time value after arriving at the specified state. A similar suggestion was a breakpoint with a state and a number value, which would trigger after that state was entered x times, where x was the specified number. It was also suggested that breakpoints could be created through the command line as arguments when starting the simulation instead of having an XML file with them.
- **Steps** - Some new possible step types were suggested, such as the commonly known debugger steps (*Step Out*, *Step Into* and *Step Over*), stepping to a specific attribute value or a step with an x number where x would allow the user to take x consecutive steps ("batch steps"). Stepping is not currently possible in debug mode, which felt inconsistent to some testers as they suggested this to be possible.
- **Execution** - Some suggestions were made about the execution itself. For example, when the speed scale factor was changed in Task 2, some testers said they would like to have a visual perception of the time passing to tell how fast it was. One tester also mentioned that it could be useful to know attribute values when a state was being exited during the execution (so any state exit functions could be noticed on the attribute values).
- **Debug Mode** - As for the debug mode, the only suggestion was to display a message indicating that debug mode was entered due to a breakpoint if that was the case.
- **Simulation Modes** - The existing simulation modes were deemed useful and good enough by most testers, so a single suggestion for a new mode was made, which was the *Skip-to-specific Threshold*, which would start a simulation at a specified point of the execution of the model.
- **Others** - Some other suggestions were made, such as the creation of an **undo** command to undo the last given command or go back in time for a specified value of seconds. It was also suggested that every operation that can be performed outside the terminal would be explained (placing breakpoints, changing simulation modes, and so on).

6.4 Discussion of Results

6.4.1 Evaluation of Results and Implications

We may form several conjectures based on the gathered data and the analysis performed.

Starting with the usability and task load scores calculated using the [SUS](#) and [NASA-TLX](#) tests, it is possible to conclude that both the usability and task load scores are quite positive. The usability score has a mean of 76.5 out of 100 (where 100 is the most positive

score), with an outlier low score of 45, which is a relatively low score. As for the task load score, it resulted in a mean of 24.8 out of 100 (where 100 is the most negative score), which is also a positive result. Based on these results, the tool has a good usability level (above average, as the average SUS score is 68, as mentioned in [37]) and a low effort level.

As for the precision and recall results, both have very high scores, which indicates how easy it may be to get used to the tool. The worst precision rate was on task 3, with a mean of 88.25%. This was due to the **step** event being used in debug mode, which was not allowed in the implementation of the debugger when the user tests were made. This was suggested by the participants of the user tests a few times as an improvement to the tool and would surely be a very important upgrade. As a consequence, steps were the least usable feature, as can also be seen in Figure 6.6.

The trace log was a feature that received some criticism regarding its clarity and how its comprehension could be boosted with a different format or an explanation of how it is formatted. In Figure 6.8, we can see the evolution of the users' perception of the log, and we may conclude that the testers got used to it after seeing it a few times, and we may also notice that there was a small score drop in task 3. This was due to breakpoints not being correctly signalled in the trace log.

In task 5, the testers had to answer 2 questions about the model presented in that task using the SCCD Debugger. As we saw in the previous subsection, the first question, which was about the value of a variable after 5 events occurred, was answered correctly by everyone. However, the second question, which asked about the value of that same variable at a specific point of the execution, was answered correctly by 9 testers and incorrectly by 1. This question could have been answered in 2 different ways: the tester could use the terminal and scroll up to see the value of the variable right after the event; or the tester could stop the simulation and consult the trace log to see the value of the variable. The 9 correct answers were given using the first method, and the wrong answer was given using the second method. This was expected to happen, and the purpose of this question was exactly this. It came to prove the trace log still has its share of flaws. It is possible to see this situation represented in Listing 6.1.

Listing 6.1: Task 5 Trace Log Excerpt

5. Timestamp: 12754; Name: exit: /A/C – e3; Attributes: [x: 10;]
6. Timestamp: 12754; Name: entry: /A/B; Attributes: [x: 5;]

It is possible to see in this excerpt 2 distinct events represented in the trace log. *State_C* and *State_B* are both children states of *State_A*. *State_C* was exited in 5 and *State_B* was entered in 6. We are interested in the value of the variable *x* right after event **e3**. The correct answer would be 5, which would be fairly obvious when looking at the terminal execution. However, looking at this excerpt of the trace log, it would be very possible for the user to wonder between 5 and 10.

The obtained results also indicate that the help menu's colours were not extremely intuitive, as commands were not directly recognisable from simple concept words, and

this was viewed as a very useful criticism.

Some feedback was given about the debug mode not giving information to the user as to why it was entered. It may be entered because of a pause or a breakpoint, and some testers did not understand the reason for debug mode becoming active due to a breakpoint. This should be explicit to the user.

Some testers claimed they would have preferred to have a perception of the time passing during the execution and not just on the trace log. This makes a lot of sense as we might be dealing with models that require attention to the passage of time during the execution, which could be useful when debugging.

6.4.2 Threats to Validity

The validity of this evaluation may have some threats to it. First of all, the sample size of all user tests is 10, so it is a fairly small number from which we can draw definitive conclusions. However, one of the main purposes of this evaluation was to determine which features of the [SCCD](#) Debugger required improvements and what kind of improvements could be performed. As stated by Nielsen in [46], 5 users will find most of the problems in an application, meaning the sample size of our evaluation was enough to cover a huge part of the existing problems. The participants also do not have much variety among themselves, as they are all Computer Science students at the same university, which means the foundations of their knowledge in the subject are quite similar. In addition, the participants are also acquainted with the author of this project, meaning some acquaintance bias might be present.

Regarding the test itself, all testers performed the same tasks using the same tools, so there was no baseline to compare to. Some tests could have been performed using another tool, so it would have been possible to compare both tools. The conditions under which the tests were performed may have also somehow influenced the participants' performances, with a few unwanted interruptions.

There is also the matter of using usability and task load questionnaires like [SUS](#) or [NASA-TLX](#) since these tests, although commonly used standards, still try to measure something that is not entirely objective, so it is imperative to remember the values obtained in those tests are not absolute. One particular question asked in the [SUS](#) questionnaire gathered very inconsistent results. This question is "I think that I would like to use this system frequently". Some participants answered this question as if they had to work with statechart models, others did not. This resulted in some negative results which are related to statecharts themselves instead of the [SCCD](#) Debugger.

6.4.3 Inferences

About the usability of the [SCCD](#) Debugger, some preliminary inferences can be made. The current implementation provides a solid level of usability, as seen by most of the test results in the previous subsections. However, a big margin for improvement still remains,

and some user experiences may be particularly cumbersome for their users, as was the case with one of the testers whose [SUS](#) score was below 50. This could have been expected since the focus of this dissertation was effectively applied to the functionality and not as much on the usability. It is also possible that because the current implementation provides a console-based interface instead of a graphical one, the usability scores may be lower as some people are less accustomed to using this type of interface.

6.4.4 Lessons Learned

Some lessons were learned about user testing. It is very important to define the goals before performing the experiments, as the data we are interested in must be set beforehand. This will make the tests easier to analyse and more objective. The type of experiment that will be performed should also be set as this evaluation would have been more interesting if there was a baseline of comparison to test the [SCCD](#) Debugger against. For example, the users could have tried to perform simulations using the [SCCD](#) Debugger and Itemis (explored in section [3.2.1](#)).

CONCLUSION

This chapter highlights the contributions of this work and discusses possible future work to continue this project.

7.1 Overview

For this work, we defined the implementation of a debugging tool to use on [SCCD](#) models as an objective.

Since the simulation of [SCCD](#) models is quite similar to the simulation of Statecharts, Statechart modelling and simulation tools were researched and investigated to define the minimum requirements for this project as well as what improvements could be made from existing tools and what innovative and relevant features could be included in this tool.

The requirements set for our solution were: the possibility to change between different **execution states**, the possibility to change between different **simulation modes**, the ability to place **breakpoints** in different parts of the model, the ability to take **steps during** the execution of a model, the possibility to **inject external events** during the execution of a model and the ability to **inspect the state** the execution is currently in. Some other features ended up being added: the calculation of different **metrics** about the simulation, the creation of a **tracing log** containing detailed information about the simulation as well as the possibility to open a **help** menu at any point of the simulation.

It was also important to define the differential characteristics of our tool. These characteristics are heavily derived from the usage of [SCCD](#) because no other tool uses this formalism. With [SCCD](#), we have a modelling formalism which can model structure (Class Diagrams) and behaviour (Statecharts). With this formalism, it is possible to perform both Multi-State Machine Simulation and Dynamic Structure Modelling, something unseen in any other tools.

To extend the implementation of [SCCD](#), it was necessary to deeply study and understand it as it is quite complex. Before the addition of debugging mechanisms, its implementation consisted of a compiler that could receive a [SCCD](#) model in the [SCXML](#) format and would generate code representing the behaviour and structure of this model.

The generated code could then be run using an auxiliary runner file, and the result was a working simulation of the model's execution. This could be a terminal-level simulation, or it could include a user interface, depending on the chosen platform to compile the model. The execution was also controlled by other objects that were part of the runtime implementation and would remain mostly untouched with the addition of debugging mechanisms.

The aforementioned debugging features were then added as an extension of the [SCCD](#) implementation. The final tool only allowed these mechanisms to be used at the console-level.

After the implementation and individual testing of the tool, an evaluation through user tests was conducted in order to assess its qualities and possible improvements or new additions.

7.2 Contributions

With this project, we developed a console-level tool capable of model simulation and debugging and capable of performing multi-state machine simulations as well as dynamic modelling. Despite having room for improvement (which will be discussed further), in its current state, this tool can be seen as a stepping stone in modelling and simulation tools and their usability as it is the first one to allow multi-state machine simulation and dynamic modelling and combine them with debugging mechanisms.

7.3 Future Work

7.3.1 Improvements

A few features that were implemented in the debugger can be improved somehow. These improvements are derived from user tests feedback but also from personal experience as the developer.

- **Trace Log** - The trace log can be improved in several ways. Different ways to represent the log itself may be explored, such as using a table to display the data or perhaps straightening and aligning the lines of the log, in an attempt to unclutter the information and make it more readable. It is also essential to make every bit of information clear to the user, omitting irrelevant information and not displaying confusing names, as is the case with timers or breakpoints. It is also important to make the user understand the events that happened and when they happened. Aside from this, the trace log can also be extended with event raises and history states and attempt to answer other questions such as "Which states were active at x timestamp?" or "How would the timestamps look if we excluded debugging time or time leaps with steps?" (essentially showing the real passage of time without effectively going through it).

- **Breakpoints** - Currently, breakpoints can be placed in specific timestamps, states or class attribute values. However, it was suggested that breakpoints could allow the junction of different breakpoints, creating a breakpoint that only activates when all the different conditions are true (basically a breakpoint with a logical "and").
- **Help Menu** - In the help menu, a small revamp can be performed so the commands are easily distinguishable from other information.
- **Steps** - An improvement that can be made for steps is to allow these to be performed even when the simulation is paused.
- **Execution** - Come up with some way to give the user a visual perception of time passing during the execution.
- **Debug Mode** - Display a message indicating that debug mode was entered due to a breakpoint if that was the case.
- **Code Quality and Efficiency** - The current state of the code is that it effectively works, but a lot of unnecessary code is generated. So both the generator and the generated code could be improved, not only to make it more readable but also to make it easily extensible.
- **Models with Multiple Classes** - It is possible to execute models with at least 2 classes. However, these simulations will be cluttered and not user-friendly as, basically, several executions are running in the same terminal instance (1 execution per class). Each class should have its own terminal instance so each execution can be seen properly, receive separate input events, and have a separate trace log. For each class, there could also be a separate terminal to receive input events if the execution happens to make many consecutive prints, which would make it hard for the user to insert their own input.
- **Evaluation** - As it was mentioned in the previous section as a threat to the validity of the evaluation and as a lesson learned, it would be interesting to evaluate the performance of this tool against other model simulation tools or even against a manual simulation of models (for example, simulating a model on a notebook).

7.3.2 New Features

There are also some new features or possibilities that have not been explored yet but have either been suggested through user tests or simply seem like interesting additions to the current state of the tool.

- **Steps** - New step types could be implemented. However, these steps could follow a similar approach to code debuggers (*Step Out*, *Step Into* and *Step Over*) or could be specific to statechart semantics (*Time Step*, *Big Step*, *Small Step* and *Combo Step*). Steps

could also be performed using a numerical argument (for example, the user writes "step 4", and 4 steps are performed automatically).

- **Simulation** - The user may not want to perform the whole simulation, so a complete or partial automatic simulation until a certain point should be possible, with the aid of a file with the list of input events to be performed and when.
- **Undo** - The user may want to undo some performed step, so an undo command could be created.
- **User Interface** - This is possibly one of the most obvious and essential features to be added in the future. A User Interface for modelling and simulation would be very interesting to have and would make the user experience much richer and smoother.
- **Live Modelling** - It could be interesting to allow the user to create some event that would add a new state or transition to the model during the execution. This would be called a "god"event.

BIBLIOGRAPHY

- [1] I. AG. *Itemis Create*. 2018. URL: <https://www.itemis.com/en/products/itemis-create/> (visited on 2023-05-09) (cit. on p. 17).
- [2] D. Ameller. “Considering Non-Functional Requirements in Model-Driven Engineering”. PhD thesis. Universitat Politècnica de Catalunya, 2011-01 (cit. on pp. 5, 6).
- [3] C. Atkinson and T. Kühne. “Model-Driven Development: A Metamodeling Foundation”. In: *IEEE Softw.* 20.5 (2003-09), pp. 36–41. ISSN: 0740-7459. DOI: [10.1109/MS.2003.1231149](https://doi.org/10.1109/MS.2003.1231149). URL: <https://doi.org/10.1109/MS.2003.1231149> (cit. on p. 5).
- [4] C. Atkinson and T. Kühne. “Reducing accidental complexity in domain models”. In: *Software and Systems Modeling* 7 (2008-07), pp. 345–359. DOI: [10.1007/s10270-007-0061-0](https://doi.org/10.1007/s10270-007-0061-0) (cit. on p. 1).
- [5] M. Bagherzadeh, N. Hili, and J. Dingel. “Model-Level, Platform-Independent Debugging in the Context of the Model-Driven Development of Real-Time Systems”. In: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2017*. Paderborn, Germany: Association for Computing Machinery, 2017, pp. 419–430. ISBN: 9781450351058. DOI: [10.1145/3106237.3106278](https://doi.org/10.1145/3106237.3106278). URL: <https://doi.org/10.1145/3106237.3106278> (cit. on pp. 28, 29).
- [6] V. R. Basili, G. Caldiera, and H. D. Rombach. “The goal question metric approach”. In: *Encyclopedia of software engineering* (1994), pp. 528–532 (cit. on p. 75).
- [7] J. Brooke. “SUS: A quick and dirty usability scale”. In: *Usability Eval. Ind.* 189 (1995-11) (cit. on pp. 76, 77).
- [8] F. P. Brooks. “No Silver Bullet: Essence and Accidents of Software Engineering”. In: *IEEE computer* 20.4 (1987), pp. 10–19 (cit. on p. 1).
- [9] R. S. Corporation. *OMG Unified Modeling Language Specification*. 2000. URL: <https://www.omg.org/spec/UML/1.3/PDF> (visited on 2023-04-08) (cit. on pp. 6, 9, 10).

- [10] S. Elias. “Model-Based Development of a Modelling and Simulation Environment for the Statecharts and Class Diagrams (SCCD) Formalism”. MA thesis. Universiteit Antwerpen, Belgium, 2021 (cit. on pp. 17, 18, 22).
- [11] S. Esmaeilsabzali et al. “Big-Step Semantics”. In: 2009. URL: <https://api.semanticscholar.org/CorpusID:6043928> (cit. on p. 8).
- [12] S. Esmaeilsabzali et al. “Deconstructing the semantics of big-step modelling languages”. In: *Requir. Eng.* 15 (2010-06), pp. 235–265. DOI: [10.1007/s00766-010-0102-z](https://doi.org/10.1007/s00766-010-0102-z) (cit. on p. 8).
- [13] J. Exelmans, S. Van Mierlo, and H. Vangheluwe. “A Statecharts Interpreter and Compiler with Semantic Variability”. In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings. MODELS ’22*. Montreal, Quebec, Canada: Association for Computing Machinery, 2022, pp. 722–727. ISBN: 9781450394673. DOI: [10.1145/3550356.3561569](https://doi.org/10.1145/3550356.3561569). URL: <https://doi.org/10.1145/3550356.3561569> (cit. on p. 8).
- [14] J. Exelmans et al. “Optimistic Versioning for Conflict-tolerant Collaborative Blended Modeling”. In: *STAF Workshops*. 2022. URL: <https://api.semanticscholar.org/CorpusID:253270030> (cit. on p. 2).
- [15] P. S. Foundation. *Tkinter documentation*. Accessed: November 8, 2023. 2023. URL: <https://docs.python.org/3/library/tkinter.html> (cit. on p. 35).
- [16] T. E. Foundation. *Eclipse Papyrus*. 2019. URL: <https://www.eclipse.org/papyrus/> (visited on 2023-06-19) (cit. on p. 19).
- [17] T. E. Foundation. *Eclipse Papyrus-RT*. 2017. URL: <https://projects.eclipse.org/projects/modeling.papyrus-rt> (visited on 2023-06-19) (cit. on p. 28).
- [18] T. E. Foundation. *Epsilon*. 2006. URL: <https://www.eclipse.org/epsilon/> (visited on 2023-06-19) (cit. on p. 28).
- [19] *Google Forms*. Google. URL: <https://www.google.com/forms/about/> (cit. on p. 81).
- [20] P. Graf and K. Muller-Glaser. “Dynamic Mapping of Runtime Information Models for Debugging Embedded Software”. In: *Seventeenth IEEE International Workshop on Rapid System Prototyping (RSP’06)*. 2006, pp. 3–9. DOI: [10.1109/RSP.2006.15](https://doi.org/10.1109/RSP.2006.15) (cit. on pp. 29, 30).
- [21] O. M. Group. *OMG UML Specification*. <https://www.omg.org/spec/UML/2.5.1/About-UML>. 2017 (cit. on p. 6).
- [22] L. Guo and A. Roychoudhury. “Debugging Statecharts Via Model-Code Traceability”. In: *Leveraging Applications of Formal Methods*. 2008 (cit. on p. 29).
- [23] R. Guru. *Design Patterns - Visitor*. 2023. URL: <https://refactoring.guru/design-patterns/visitor> (cit. on p. 37).

- [24] W. Haberl et al. “Model-Level Debugging of Embedded Real-Time Systems”. In: *2010 10th IEEE International Conference on Computer and Information Technology*. 2010, pp. 1887–1894. DOI: [10.1109/CIT.2010.323](https://doi.org/10.1109/CIT.2010.323) (cit. on pp. 30, 31).
- [25] D. Harel and E. Gery. “Executable object modeling with statecharts”. In: *Computer* 30.7 (1997), pp. 31–42. DOI: [10.1109/2.596624](https://doi.org/10.1109/2.596624) (cit. on p. 2).
- [26] D. Harel. “Statecharts: a visual formalism for complex systems”. In: *Science of Computer Programming* 8.3 (1987), pp. 231–274. ISSN: 0167-6423. DOI: [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9). URL: <https://www.sciencedirect.com/science/article/pii/0167642387900359> (cit. on p. 6).
- [27] S. Hart. “Nasa-task load index (Nasa-TLX); 20 years later”. In: vol. 50. 2006-10. DOI: [10.1177/154193120605000909](https://doi.org/10.1177/154193120605000909) (cit. on p. 78).
- [28] S. G. Hart and L. E. Staveland. “Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research”. In: *Human Mental Workload*. Ed. by P. A. Hancock and N. Meshkati. Vol. 52. Advances in Psychology. North-Holland, 1988, pp. 139–183. DOI: [https://doi.org/10.1016/S0166-4115\(08\)62386-9](https://doi.org/10.1016/S0166-4115(08)62386-9). URL: <https://www.sciencedirect.com/science/article/pii/S0166411508623869> (cit. on p. 76).
- [29] Heavy.AI. *Embedded Systems*. Accessed: June 21, 2024. 2024. URL: <https://www.heavy.ai/technical-glossary/embedded-systems> (cit. on p. 28).
- [30] C. Höger et al. “Modelica3D - Platform Independent Simulation Visualization”. In: 2012-11. DOI: [10.3384/ecp12076485](https://doi.org/10.3384/ecp12076485) (cit. on p. 2).
- [31] M. Jexelmans. *OnionCollab: Collaborative Research Repository*. <https://msdl.uantwerpen.be/git/jexelmans/onioncollab>. Accessed: June 26, 2023. 2023 (cit. on p. 2).
- [32] G. Kiczales et al. “Aspect-Oriented Programming”. In: *Proceedings of the European Conference on Object-Oriented Programming (ECOOP)* (1997), pp. 220–242. URL: <https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf> (cit. on p. 32).
- [33] S. Kugele et al. *COLA – The component language*. Tech. rep. 2007 (cit. on p. 30).
- [34] T. Kühne. “Matters of (Meta-) Modeling”. In: *Software & Systems Modeling* 5 (2006), pp. 369–385. URL: <https://api.semanticscholar.org/CorpusID:150431> (cit. on p. 6).
- [35] T. Kühne. “What is a Model?” In: *Language Engineering for Model-Driven Software Development*. Ed. by J. Bezivin and R. Heckel. Vol. 4101. Dagstuhl Seminar Proceedings (DagSemProc). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2005, pp. 1–10. DOI: [10.4230/DagSemProc.04101.15](https://doi.org/10.4230/DagSemProc.04101.15). URL: <https://drops.dagstuhl.de/entities/document/10.4230/DagSemProc.04101.15> (cit. on p. 6).

-
- [36] T. C. Lethbridge et al. "Umple: Model-driven development for open source and education". In: *Science of Computer Programming* 208 (2021), p. 102665. ISSN: 0167-6423. DOI: <https://doi.org/10.1016/j.scico.2021.102665>. URL: <https://www.sciencedirect.com/science/article/pii/S0167642321000587> (cit. on p. 18).
- [37] J. R. Lewis and J. Sauro. "Item benchmarks for the system usability scale." In: *Journal of Usability Studies* 13.3 (2018) (cit. on p. 89).
- [38] V. P. I. Limited. *Visual Paradigm*. 2002. URL: <https://www.visual-paradigm.com/> (visited on 2023-06-19) (cit. on p. 19).
- [39] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf> (cit. on p. ii).
- [40] L. Lúcio et al. "Model Transformation Intents and Their Properties". In: *Software & Systems Modeling* 15 (2016-07), pp. 647–684. DOI: [10.1007/s10270-014-0429-x](https://doi.org/10.1007/s10270-014-0429-x) (cit. on pp. 11, 12).
- [41] J. Ludewig. "Models in software engineering - An introduction". In: *Inform., Forsch. Entwickl.* 18 (2004-04), pp. 105–112. DOI: [10.1007/s10270-003-0020-3](https://doi.org/10.1007/s10270-003-0020-3) (cit. on p. 1).
- [42] MathWorks. *Stateflow*. 2018. URL: <https://www.mathworks.com/products/stateflow.html> (visited on 2023-05-16) (cit. on p. 18).
- [43] S. V. Mierlo. "A multi-paradigm modelling approach for engineering model debugging environments". PhD thesis. 2018. URL: <https://api.semanticscholar.org/CorpusID:57464392> (cit. on pp. 8, 15).
- [44] S. V. Mierlo et al. *SCCD: SCXML extended with class diagrams*. 2016. URL: <https://msdl.uantwerpen.be/documentation/SCCD/#sccd> (visited on 2023-05-14) (cit. on pp. 2, 18, 23).
- [45] S. Mustafiz and H. Vangheluwe. "Explicit Modelling of Statechart Simulation Environments". In: *Proceedings of the 2013 Summer Computer Simulation Conference*. SCSC '13. Toronto, Ontario, Canada: Society for Modeling & Simulation International, 2013. ISBN: 9781627482769. URL: <https://dl.acm.org/doi/10.5555/2557696.2557720> (cit. on pp. 13, 15, 16).
- [46] J. Nielsen. *Why You Only Need to Test with 5 Users*. Nielsen Norman Group. 2000. URL: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/> (cit. on p. 90).
- [47] pandas development team. *pandas: powerful data analysis tools for Python*. pandas development team. URL: <https://pandas.pydata.org/docs/> (cit. on p. 87).
- [48] S. Project. *Deep History*. Accessed: June 21, 2024. 2023. URL: https://yasmine.seadex.de/Deep_history.html (cit. on p. 8).

- [49] B. Selic. “Model-driven development: its essence and opportunities”. In: *Ninth IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC’06)*. 2006, p. 7. DOI: [10.1109/ISORC.2006.54](https://doi.org/10.1109/ISORC.2006.54) (cit. on p. 1).
- [50] B. Selic. “The pragmatics of model-driven development”. In: *IEEE Software* 20.5 (2003), pp. 19–25. DOI: [10.1109/MS.2003.1231146](https://doi.org/10.1109/MS.2003.1231146) (cit. on p. 1).
- [51] B. Selic. “Using UML for modeling complex real-time systems”. In: *Languages, Compilers, and Tools for Embedded Systems*. Ed. by F. Mueller and A. Bestavros. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 250–260. ISBN: 978-3-540-49673-1. DOI: [10.1007/BFb0057795](https://doi.org/10.1007/BFb0057795) (cit. on p. 28).
- [52] E. Syriani, J. Gray, and H. Vangheluwe. “Modeling a Model Transformation Language”. In: *Domain Engineering: Product Lines, Languages, and Conceptual Models*. Ed. by I. Reinhartz-Berger et al. 2013-05, pp. 211–237. ISBN: 978-3-642-36653-6. DOI: [10.1007/978-3-642-36654-3_9](https://doi.org/10.1007/978-3-642-36654-3_9) (cit. on p. 12).
- [53] D. Systèmes. *Cameo Systems Modeler*. URL: <https://www.3ds.com/products-services/catia/products/no-magic/cameo-systems-modeler/> (visited on 2023-05-16) (cit. on p. 19).
- [54] S. Systems. *EnterpriseArchitect*. 2021. URL: <https://sparxsystems.com/products/ea/> (visited on 2023-05-15) (cit. on p. 17).
- [55] M. D. Team. *Matplotlib: Visualization with Python*. Matplotlib Development Team. URL: <https://matplotlib.org/> (cit. on p. 87).
- [56] S. Van Mierlo and H. Vangheluwe. “Statecharts: A Formalism to Model, Simulate and Synthesize Reactive and Autonomous Timed Systems”. In: *Foundations of Multi-Paradigm Modelling for Cyber-Physical Systems*. Ed. by P. Carreira, V. Amaral, and H. Vangheluwe. Cham: Springer International Publishing, 2020, pp. 155–176. ISBN: 978-3-030-43946-0. DOI: [10.1007/978-3-030-43946-0_6](https://doi.org/10.1007/978-3-030-43946-0_6). URL: https://doi.org/10.1007/978-3-030-43946-0_6 (cit. on p. 3).
- [57] H. Vangheluwe and E. Syriani. *Welcome to the ATOMPM project*. URL: <https://atopm.github.io/#home> (cit. on p. 2).
- [58] K. Zeng, Y. Guo, and C. K. Angelov. “Graphical Model Debugger Framework for embedded systems”. In: *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. 2010, pp. 87–92. DOI: [10.1109/DATE.2010.5457232](https://doi.org/10.1109/DATE.2010.5457232) (cit. on p. 29).

Listing I.1: Basic Counter

```
<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Counter">
  <inport name="input" />
  <outport name="output" />
  <class name="MainApp" default="true">
    <attribute name="counter" type="number" init-value="0"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment_counter">
        <body>
          self.counter = self.counter + 1
        </body>
      </method>
      <scxml initial="state_A">
        <state id="state_A">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target=" ../state_B" after="10" />
        </state>
        <state id="state_B">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target=" ../state_A" event="move" port="input" />
        </state>
      </scxml>
    </class>
  </diagram>
```

Listing I.2: Counter with Composite States

```

<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Counter">
  <inport name="input" />
  <outport name="output" />
  <class name="MainApp" default="true">
    <attribute name="counter" type="number" init-value="0"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment_counter">
        <body>
          self.counter = self.counter + 1
        </body>
      </method>
      <scxml initial="state_A">
        <state id="state_A" initial="state_A1">
          <state id="state_A1">
            <onentry>
              <script>
                self.increment_counter();
              </script>
            </onentry>
            <onexit>
              <script>
                self.increment_counter();
              </script>
            </onexit>
            <transition target="../state_A2" event="a2" port="input" />
            <transition target="../state_A2" after="10" />
          </state>
          <state id="state_A2">
            <onentry>
              <script>
                self.increment_counter();
              </script>
            </onentry>
            <transition target="../state_A1" event="a1" port="input" />
            <transition target="/state_B" event="b" port="input" />
          </state>
          <onexit>
            <script>
              self.increment_counter();
            </script>
          </onexit>
        </state>
        <state id="state_B">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_A" event="move" port="input" />

```

```

        </state>
    </scxml>
</class>
</diagram>

```

Listing I.3: Counter with History State

```

<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Composite_Counter">
    <inport name="input" />
    <outport name="output" />
    <class name="MainApp" default="true">
        <attribute name="counter" type="number" init-value="0"/>
        <method name="MainApp">
            <body>
                <![CDATA[

                    ]]>
            </body>
        </method>
        <method name="increment_counter">
            <body>
                self.counter = self.counter + 1
            </body>
        </method>
        <scxml initial="state_A">
            <state id="state_A" initial="state_A1">
                <state id="state_A1">
                    <onentry>
                        <script>
                            self.increment_counter();
                        </script>
                    </onentry>
                    <transition target="../state_A2" event="goA2" port="input" />
                </state>
                <state id="state_A2">
                    <onentry>
                        <script>
                            self.increment_counter();
                        </script>
                    </onentry>
                    <transition target="../state_A1" event="goA1" port="input" />
                    <transition target="/state_B" event="goB" port="input" />
                </state>
                <history id="history"/>
            </state>
            <state id="state_B">
                <onentry>
                    <script>
                        self.increment_counter();
                    </script>
                </onentry>
                <transition target="../state_A" event="goA" port="input" />
                <transition target="../state_A/state_A1" event="goA1" port="input" />
                <transition target="../state_A/history" event="back" port="input" />
            </state>
        </scxml>
    </class>

```

```
</diagram>
```

Listing I.4: Counter with Multiple States

```
<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Counter">
  <inport name="input" />
  <outport name="output" />
  <class name="MainApp" default="true">
    <attribute name="counter" type="number" init-value="0"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment_counter">
        <body>
          self.counter = self.counter + 1
        </body>
      </method>
      <scxml initial="state_A">
        <state id="state_A">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_B" after="10" />
          <transition target="../state_C" after="20" />
          <transition target="../state_D" event="d" port="input" />
        </state>
        <state id="state_B">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_A" event="move" port="input" />
        </state>
        <state id="state_C">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_A" event="move" port="input" />
        </state>
        <state id="state_D">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_A" event="move" port="input" />
        </state>
      </scxml>
    </class>
  </diagram>
```

```
</class>
</diagram>
```

Listing I.5: Counter with Parallel States

```
<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Composite_Counter">
  <inport name="input" />
  <outport name="output" />
  <class name="MainApp" default="true">
    <attribute name="counter" type="number" init-value="0"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment_counter">
        <body>
          self.counter = self.counter + 1
        </body>
      </method>
      <scxml initial="parallel">
        <parallel id="parallel" >
          <state id="state_A" initial="state_A1">
            <state id="state_A1">
              <onentry>
                <script>
                  self.increment_counter();
                </script>
              </onentry>
              <transition target="../state_A2" after="5" />
            </state>
            <state id="state_A2">
              <onentry>
                <script>
                  self.increment_counter();
                </script>
              </onentry>
              <transition target="../state_A1" event="move" port="input" />
            </state>
          </state>
          <state id="state_B" initial="state_B1">
            <state id="state_B1">
              <onentry>
                <script>
                  self.increment_counter();
                </script>
              </onentry>
              <transition target="../state_B2" after="5" />
            </state>
            <state id="state_B2">
              <onentry>
                <script>
                  self.increment_counter();
                </script>
              </onentry>
            </state>
          </state>
        </parallel>
      </scxml>
    </class>
  </diagram>
```

```

        <transition target="../state_B1" event="move" port="input" />
    </state>
</state>
</parallel>
</scxml>
</class>
</diagram>

```

Listing I.6: Counter with Scripts

```

<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Counter">
  <inport name="input" />
  <outport name="output" />
  <class name="MainApp" default="true">
    <attribute name="counter" type="number" init-value="0"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment_counter">
        <body>
          self.counter = self.counter + 1
        </body>
      </method>
      <scxml initial="state_A">
        <state id="state_A">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <onexit>
            <script>
              self.increment_counter();
            </script>
          </onexit>
          <transition target="../state_B" after="10" >
            <script>
              self.increment_counter();
            </script>
          </transition>
        </state>
        <state id="state_B">
          <onentry>
            <script>
              self.increment_counter();
            </script>
          </onentry>
          <transition target="../state_A" event="move" port="input" />
        </state>
      </scxml>
    </class>
  </diagram>

```

Listing I.7: Phone

```
<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Phone">
  <top>
    from time import sleep
  </top>
  <inport name="input" />
  <outport name="output" />
  <class name="Phone" default="true">
    <attribute name="battery" type="number" init-value="0" />
    <method name="Phone">
      <body>
        <![CDATA[
          self.min_battery = 10
          self.max_battery = 100
        ]]>
      </body>
    </method>
    <method name="charge_battery">
      <body>
        self.battery = self.battery + 1
      </body>
    </method>
    <scxml initial="powered_off">
      <state id="powered_off" initial="not_charging">
        <state id="not_charging">
          <transition target="/powered_on" event="press_power_button" port="input"
            cond="self.battery_<=>self.min_battery" />
          <transition target="../charging" event="plug_charger" port="input" />
        </state>
        <state id="charging">
          <onentry>
            <script>
              self.charge_battery()
            </script>
          </onentry>
          <transition target="/powered_on" event="press_power_button" port="input"
            cond="self.battery_<=>self.min_battery" />
          <transition target="../not_charging" event="unplug_charger" port="input" />
          <transition target="." after="15" cond="self.max_battery_<=>self.battery" />
        </state>
      </state>
      <parallel id="powered_on">
        <state id="charge" initial="not_charging">
          <state id="not_charging">
            <transition target="../charging" event="plug_charger" port="input" />
          </state>
          <state id="charging">
            <onentry>
              <script>
                self.charge_battery()
              </script>
            </onentry>
            <transition target="../not_charging" event="unplug_charger" port="input" />
            <transition target="." after="15" cond="self.max_battery_<=>self.battery" />
          </state>
        </state>
      </parallel>
    </scxml>
  </class>
</diagram>
```

```

    <state id="apps" initial="homescreen">
      <state id="homescreen">
        <transition target="../app_a" event="go_app_a" port="input"/>
        <transition target="../app_b" event="go_app_b" port="input"/>
      </state>
      <state id="app_a">
        <transition target="../homescreen" event="go_home" port="input"/>
      </state>
      <state id="app_b">
        <transition target="../homescreen" event="go_home" port="input"/>
      </state>
    </state>
    <transition target="/powered_off" event="press_power_button" port="input"/>
  </parallel>
</scxml>
</class>
</diagram>

```

Listing I.8: User Tests Task 5

```

<?xml version="1.0" ?>
<diagram author="Francisco_Simoes" name="Task5">
  <top>
    from time import sleep
  </top>
  <input name="input" />
  <output name="output" />
  <class name="MainApp" default="true">
    <attribute name="x" type="number" init-value="3"/>
    <method name="MainApp">
      <body>
        <![CDATA[

          ]]>
        </body>
      </method>
      <method name="increment">
        <body>
          self.x = self.x + 1
        </body>
      </method>
      <method name="decrement">
        <body>
          self.x = self.x - 1
        </body>
      </method>
      <method name="divideBy2">
        <body>
          self.x = self.x / 2
        </body>
      </method>
      <method name="multiplyBy2">
        <body>
          self.x = self.x * 2
        </body>
      </method>
    <scxml initial="A">
      <state id="A" initial="B">

```

```

<onentry>
  <script>
    self.increment()
  </script>
</onentry>
<onexit>
  <script>
    self.decrement()
  </script>
</onexit>
<state id="B">
  <onentry>
    <script>
      self.divideBy2()
    </script>
  </onentry>
  <onexit>
    <script>
      self.increment()
      self.increment()
      self.increment()
    </script>
  </onexit>
  <transition target="." event="e1" port="input">
    <script>
      self.decrement()
    </script>
  </transition>
  <transition target="../C" event="e2" port="input">
    <script>
      self.multiplyBy2()
    </script>
  </transition>
</state>
<state id="C">
  <transition target="." event="e4" port="input">
    <script>
      self.increment()
    </script>
  </transition>
  <transition target="/D" event="e5" port="input"/>
  <transition target="../B" event="e3" port="input"/>
</state>
<transition target="." event="e6" port="input"/>
</state>
<state id="D">
  <onentry>
    <script>
      self.increment()
    </script>
  </onentry>
</state>
</scxml>
</class>
</diagram>

```

Testing and Debugging in Model-Driven Engineering: **SCCD** Debugger

Researcher: Francisco Simões

Information and User's Participation Consent

The objective of this experiment is to validate and test a Terminal-level application produced as part of a Master's Dissertation. This application is the **SCCD** Debugger. **SCCD** is a modelling formalism that combines Class Diagrams with Statecharts, providing more modelling power to the user. An **SCCD** model consists of several classes linked between them through associations, where each class has an associated statechart that models its behaviour during runtime. **SCCD** models are currently written in XML files, therefore no tasks will include creating new models or making changes to existing ones. All models that will be used in the tasks are provided. The application receives an **SCCD** model in the XML format and will generate code that can be executed, simulating the given model's behaviour and providing several debugging mechanisms to aid the simulation and help understanding the model.

To participate in this experiment, the user should possess some knowledge regarding Statecharts and Class Diagrams. The experiment will be conducted on a laptop while the user is monitored by the researcher. The user's participation is voluntary and no personal or monetary benefit will be provided in return. Each participant will be provided with a participant number by the researcher in the beginning of the experiment.

If you have any question regarding this experiment, please enquire the researcher using the following contact information:

Student: Francisco Simões
Departamento Informática
NOVA School of Science and Technology
Universidade NOVA de Lisboa
fd.simoes@campus.fct.unl.pt

SCCD Debugger: Testing Instructions

The following script contains an installation guide and a set of tasks to be completed using the **SCCD** Debugger. Additionally, this [online form](#) contains a set of questions regarding the executed tasks. The task set will test the following features of the software:

- Generating an execution file for a model and executing its simulation
- Pausing, resuming and stopping the simulation
- Change the simulation mode and the simulation's scale factor
- Placing a breakpoint in the simulation
- Performing steps during the simulation
- Generating and understanding a trace log of the simulation
- Using the help menu during the simulation

Please read the installation guide and task instructions carefully and ask any questions you may deem necessary. For each task, please follow the provided steps and answer the questions regarding that task. Feel free to voice or write any feedback about the software at any point during the experiment if you wish to do so.

Installation Guide

- Download the ZIP folder with the software in the [github page of the project](#) and unzip the folder.
- If you do not have **Python version 2.7** installed on your operative system, please install it, using the following [link](#), where you can download version 2.7.18 of Python. Do not forget to add the Python folder path (C:\Python27) to the path variable of the environment variables.
- Afterwards, you may open a terminal inside the src folder of the unzipped folder of the project.
- Please execute the test.bat file. If your terminal displays different colours, you may start Task 1. Otherwise, please follow the next step: Run a Windows Powershell in Administrator mode and run the following command `Set-ItemProperty HKCU:\Console VirtualTerminalLevel -Type DWORD 1`. The reason for this step is that this software, since it is a terminal-level application, uses ANSI colour codes to provide a better and more understandable interface to the user and some operative systems do not properly display these colours unless they are specifically enabled to do so.
- Feel free to ask the researcher any questions about the installation.

Task 1

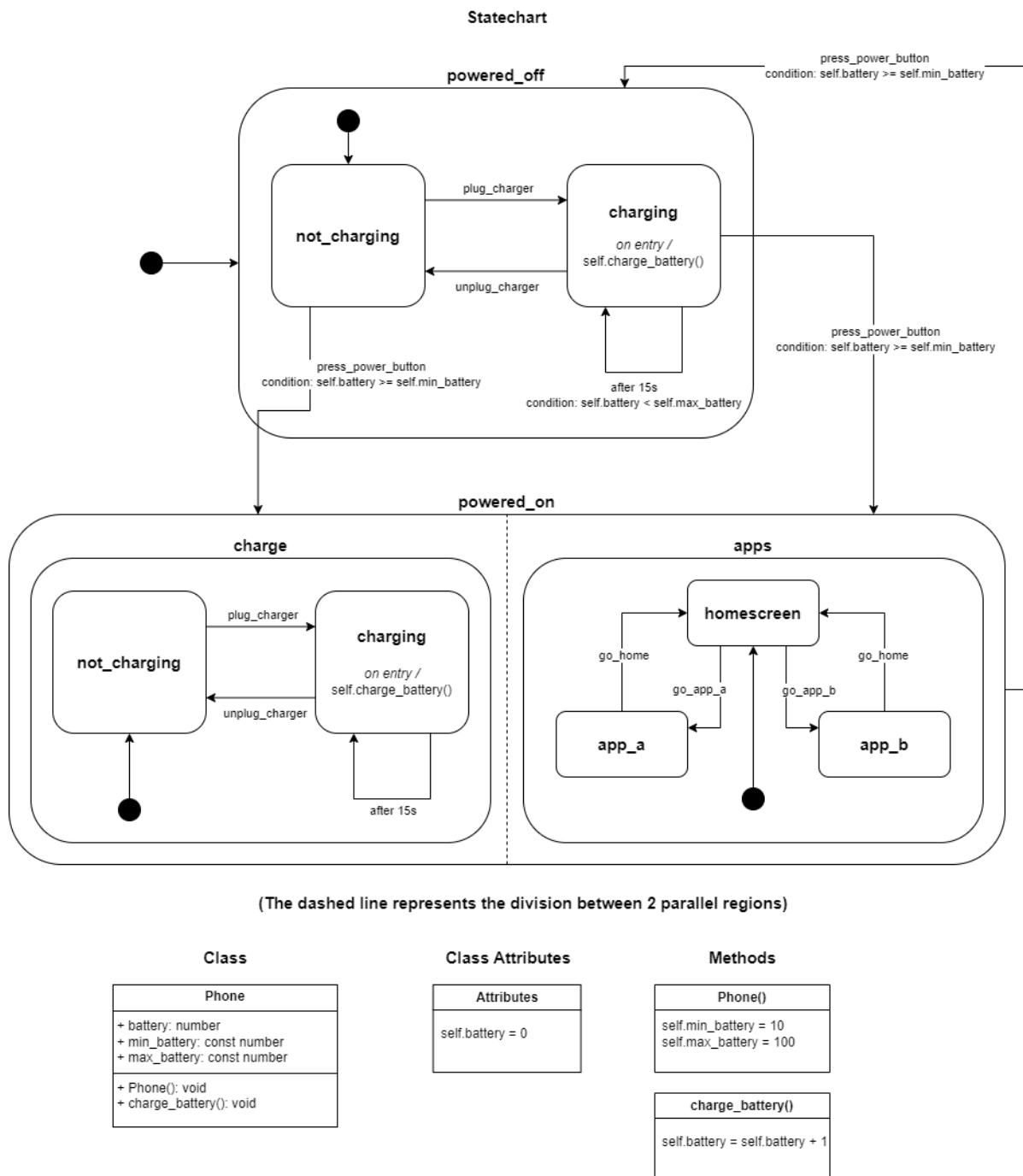


Figure 1. Test Model 1

We want to **run** a simulation of the model in **Figure 1**. To do this we must **generate** the code to run the simulation. Each different model must have its own code generated, this is what the task1.bat file does.

1. Run the task1.bat file. This will generate the file we need.
2. Run the runSim.bat file. This will start the simulation.
3. Write 'help' to bring up the help menu. Observe the different commands.

4. There are a few possible events. Send in the 'plug_charger' event.
5. Wait until the battery variable has the value = 2.
6. Pause the simulation and observe the variable values.
7. Continue the simulation.
8. Stop the simulation and observe its trace log (executionTrace.txt).

Task 2

Now we want to **speed up** the simulation. The **runSim.bat** file contains the command line to start the simulation. This line receives 2 arguments: **-s** and **-f**. These arguments control the simulation mode and its speed scale factor. **-s** may take as a value 0, 1 or 2. 0 is the default **real-time simulation**, used in Task 1. The value 1 corresponds to the **scaled real-time simulation**, which is scaled using the factor provided in **-f**. For example, **-f 3** will make the simulation 3x faster. Note the value provided in **-f** will only have any effect if **-s** is 1. The value 2 corresponds to the **as-fast-as-possible simulation**, which automatically sets the speed scale factor to infinity.

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run 2x faster.
2. Run the runSim.bat file to start the simulation.
3. Send in the 'plug_charger' event.
4. Observe the simulation and notice the increase in simulation speed.
5. Stop the simulation and observe its trace log (executionTrace.txt).

Task 3

Now we want to revert the changes made to the simulation mode and speed scale factor on the previous task, so we will **change** the values of **-s** and **-f back**. We want to observe the values of the class attributes at some point of the execution, so we will place a **breakpoint**.

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run at its default speed. (-s must be set to 0)
2. Open the breakpoints.xml file in any text editor. Place a state breakpoint on the "/powered_off/charging" state, so the simulation will pause when the phone of our model starts charging. (This file contains 3 commented breakpoints as an example of each type of breakpoint you can create; a comment is denoted by '<!--' in the start and '-->', so do not include these in the created breakpoint)
3. Run the task1.bat to generate the code with the breakpoint.
4. Run the runSim.bat file to start the simulation.
5. Send in the event to plug the charger of the phone.
6. Notice the automatic pause on the simulation as we entered the "/powered_off/charging" state.
7. Unpause the simulation.

8. As the phone charges, you may notice the 'step' event is available to skip transitions. Send in the 'step' event. (You may not step when in **Debug Mode**)
9. Stop the simulation and observe its trace log (executionTrace.txt).

Task 4

Now we want to see the phone of our model charge completely but we do not want to wait for the phone to finish charging and we do not want to charge it step-by-step. So we will change the simulation mode to the **as-fast-as-possible simulation** mode (-s 2).

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run on the as-fast-as-possible simulation mode. (-s must be set to 2)
2. Clear the breakpoints created in the previous task and run the task1.bat again and then start the simulation using the runSim.bat file.
3. Send the appropriate events to plug the charger of the phone, turn the phone on and open app a.
4. Stop the simulation and observe its trace log (executionTrace.txt)..

Task 5

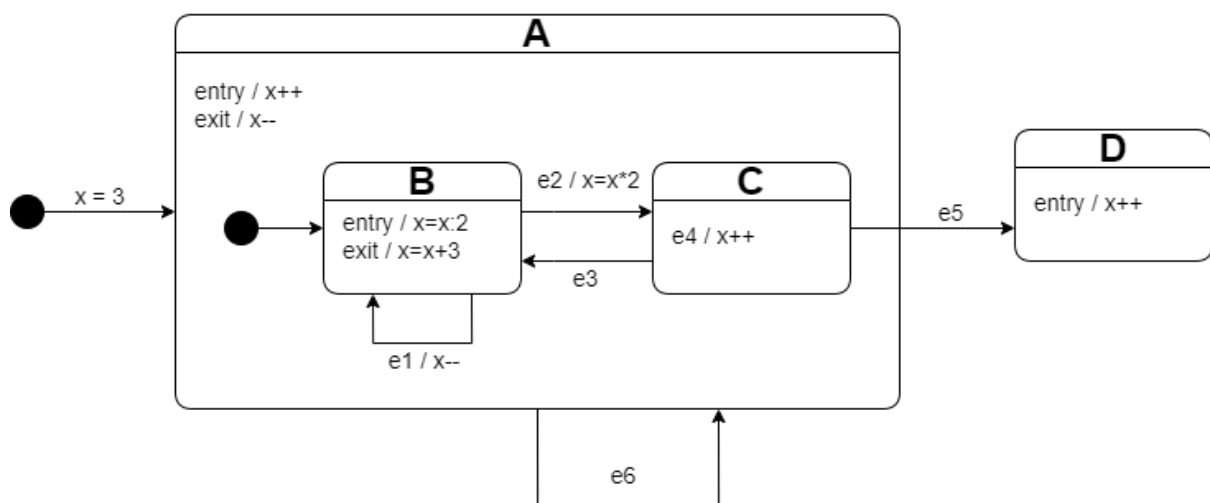


Figure 2. Test Model 2

Now we will observe a different model. Take a look at the model in **Figure 2**. Use the task5.bat to generate the code and start the execution of the model. Please use the tool to answer some questions about the model.

Test Form - SCCD Debugger

The following form contains a set of tasks to be completed using the **SCCD** Debugger as well as a few questions about these tasks. This form is a complement of the document "Testing and Debugging in Model-Driven Engineering: **SCCD** Debugger". As such, please read the document carefully before proceeding to the tasks on this form.

** Indica uma pergunta obrigatória*

1. I have read the document "Testing and Debugging in Model-Driven Engineering: **SCCD** Debugger" and I agree with the conditions. I wish to proceed with the experiment.

Marcar apenas uma oval.

☐ Yes

☐ No

Participation Information

Please answer some questions about you.

2. Participant Number (provided by the investigator) *

3. Age *

4. Gender *

Marcar apenas uma oval.

- ☐ Female
- ☐ Male
- ☐ Prefer not to say
- ☐ Outra: _____

5. How many monitors are you using to make this test? *

Marcar apenas uma oval.

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4+

6. Highest Obtained Academic Degree (or on-going) *

Marcar apenas uma oval.

- ☐ High School
- ☐ Bachelor's Degree
- ☐ Master's Degree
- ☐ Doctor's Degree
- ☐ Outra: _____

7. What is your academic background? *

Marcar apenas uma oval.

- ☐ Computer Science
- ☐ Outra: _____

8. For how long have you used Statecharts? *

Marcar apenas uma oval.

- ☐ I have never used Statecharts
- ☐ 1 to 6 months
- ☐ 6 months to 1 year
- ☐ 1 to 2 years
- ☐ More than 2 years

9. If you have used Statecharts, in what contexts did you use them? Select all that apply.

Marcar tudo o que for aplicável.

- ☐ Academic context
- ☐ Professional context
- ☐ Personal context
- ☐ Outra: _____

10. For how long have you used Class Diagrams? *

Marcar apenas uma oval.

- ☐ I have never used Class Diagrams
- ☐ 1 to 6 months
- ☐ 6 months to 1 year
- ☐ 1 to 2 years
- ☐ More than 2 years

11. If you have used Class Diagrams, in what contexts did you use them? Select all that apply.

Marcar tudo o que for aplicável.

- ☐ Academical context
☐ Professional context
☐ Personal context
☐ Outra: _____

12. Have you used Statechart simulation tools in the past? *

Marcar apenas uma oval.

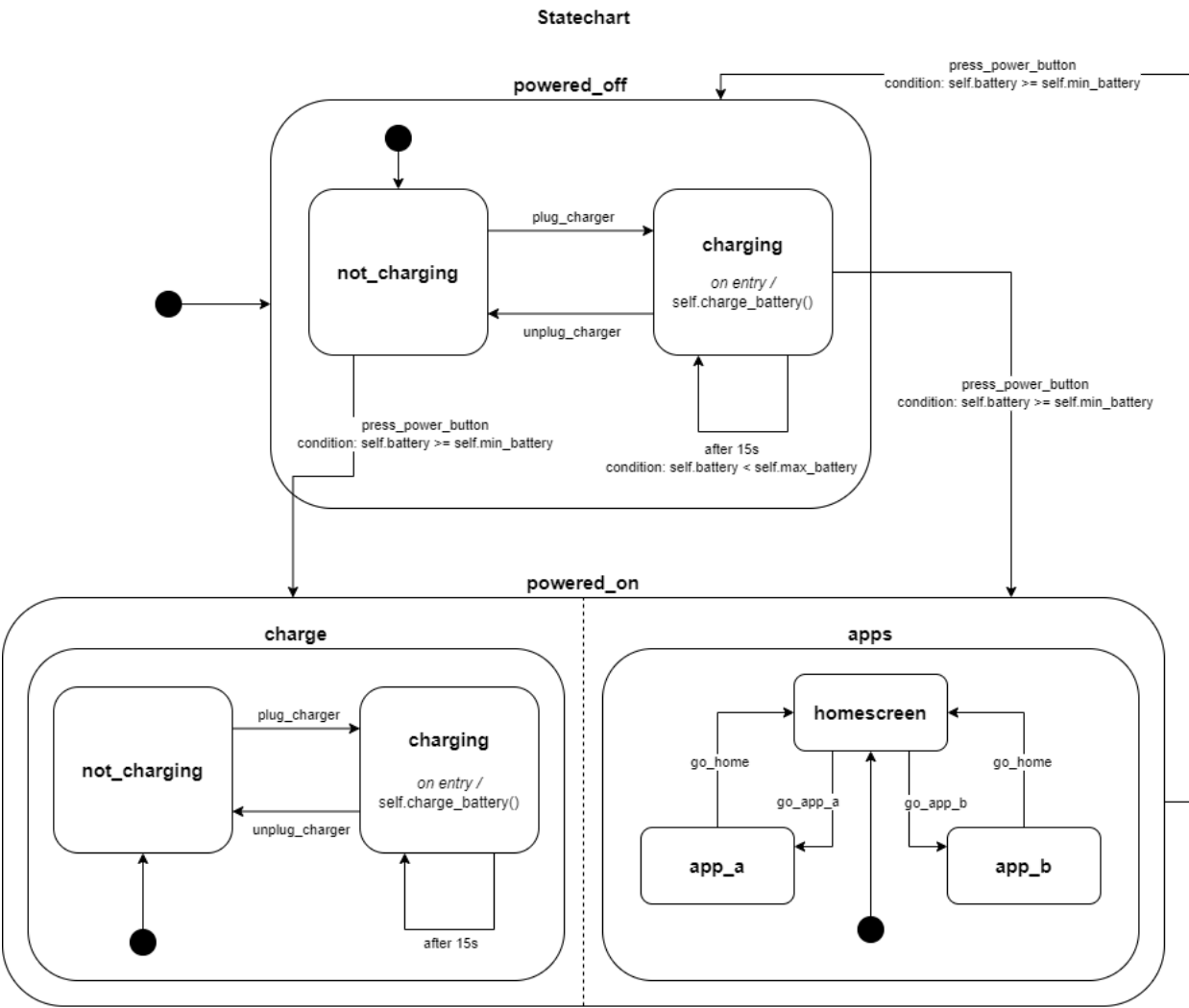
- ☐ Yes
☐ No

Task 1

We want to **run** a simulation of the model in **Figure 1**. To do this we must **generate** the code to run the simulation. Each different model must have its own code generated, this is what the task1.bat file does.

1. Run the task1.bat file. This will generate the file we need.
2. Run the runSim.bat file. This will start the simulation.
3. Write 'help' to bring up the help menu. Observe the different commands.
4. There are a few possible events. Send in the 'plug_charger' event.
5. Wait until the battery variable has the value = 2.
6. Pause the simulation and observe the variable values.
7. Continue the simulation.
8. Stop the simulation and observe its trace log (executionTrace.txt).

Figure 1. Test Model 1



(The dashed line represents the division between 2 parallel regions)

Class

Phone

+ battery: number

+ min_battery: const number

+ max_battery: const number

+ Phone(): void

+ charge_battery(): void

Class Attributes

Attributes

self.battery = 0

Methods

Phone()

self.min_battery = 10

self.max_battery = 100

charge_battery()

self.battery = self.battery + 1

13. How easy was it to understand the **help** menu? *

Marcar apenas uma oval.

12345

Very☐☐☐☐☐Very Easy

14. What **events** were available for you to send? (The events available on the initial state) *

15. How easy was it to read and understand the **trace log** in the end? *

Marcar apenas uma oval.

1 2 3 4 5

Very ☐ ☐ ☐ ☐ ☐ Very Easy

16. Is there any information that isn't on the trace log that you think it may be relevant to be there? *

17. If you wish, please leave any general comment you think may be useful.

Task 2

Now we want to **speed up** the simulation. The **runSim.bat** file contains the command line to start the simulation. This line receives 2 arguments: **-s** and **-f**. These arguments control the simulation mode and its speed scale factor. **-s** may take as a value 0, 1 or 2. 0 is the default **real-time simulation**, used in Task 1. The value 1 corresponds to the **scaled real-time simulation**, which is scaled using the factor provided in **-f**. For example, **-f 3** will make the simulation 3x faster. Note the value provided in **-f** will only have any effect if **-s** is 1. The value **-s 2** corresponds to the **as-fast-as-possible simulation**, which automatically sets the speed scale factor to infinity.

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run 2x faster.
2. Run the runSim.bat file to start the simulation.
3. Send in the 'plug_charger' event.
4. Observe the simulation and notice the change in speed.
5. Stop the simulation and observe its trace log (executionTrace.txt).

18. How easy was it to change the **-s** and **-f** arguments in the runSim.bat file? *

Marcar apenas uma oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very Easy

19. How easy was it to notice the change in the simulation speed? *

Marcar apenas uma oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very Easy

20. How useful do you think the **Scaled Real-time Simulation** mode you used can be? *

Marcar apenas uma oval.

1 2 3 4 5

Not ☐ ☐ ☐ ☐ ☐ Very Useful

21. How easy was it to read and understand the **trace log** in the end? *

Marcar apenas uma oval.

1 2 3 4 5

Very ☐ ☐ ☐ ☐ ☐ Very Easy

22. Is there any information that isn't on the trace log that you think it may be relevant to be there? *

23. If you wish, please leave any general comment you think may be useful.

Task 3

Now we want to revert the changes made to the simulation mode and speed scale factor on the previous task, so we will **change** the values of **-s** and **-f back**. We want to observe the values of the class attributes at some point of the execution, so we will place a **breakpoint**.

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run at its default speed. (-s must be set to 0)
2. Open the breakpoints.xml file in any text editor. Place a state breakpoint on the "/powered_off/charging" state, so the simulation will pause when the phone of our model starts charging. (This file contains 3 commented breakpoints as an example of each type of breakpoint you can create; a comment is denoted by '<!--' in the start and '-->', so do not include these in the created breakpoint)
3. Run the task1.bat to generate the code with the breakpoint.
4. Run the runSim.bat file to start the simulation.
5. Send in the event to plug the charger of the phone.
6. Notice the automatic pause on the simulation as we entered the "/powered_off/charging" state.
7. Unpause the simulation.
8. As the phone charges, you may notice the 'step' event is available to skip transitions. Send in the 'step' event. (You may not step when in **Debug Mode**)
9. Stop the simulation and observe its trace log (executionTrace.txt).

24. How easy was it to place the requested **breakpoint**? *

Marcas apenas uma oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very Easy

25. How easy was it to understand how the **step** event works? *

Marcas apenas uma oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very Easy

26. How easy was it to read and understand the **trace log** in the end? *

Marcar apenas uma oval.

1 2 3 4 5

Very ☐ ☐ ☐ ☐ ☐ Very Easy

27. Breakpoints can be placed in specific states, timestamps or when certain variable values hit a given threshold. Is there any other type of breakpoint you would consider useful? *

28. Is there any other type of steps you would consider useful to provide to the user? *

29. Is there any information that isn't on the trace log that you think it may be relevant to be there? *

30. If you wish, please leave any general comment you think may be useful.

Task 4

Now we want to see the phone of our model charge completely but we do not want to wait for the phone to finish charging and we do not want to charge it step-by-step. So we will change the simulation mode to the **as-fast-as-possible simulation** mode (-s 2).

1. Open the runSim.bat in any text editor. Edit the -s and -f arguments so the simulation will run on the as-fast-as-possible simulation mode. (-s must be set to 2)
2. Clear the breakpoints created in the previous task and run the task1.bat again and then start the simulation using the runSim.bat file.
3. Send the appropriate events to plug the charger of the phone, turn the phone on and open app a.
4. Stop the simulation and observe its trace log.

31. How easy was it to understand the sequence of events that happened? *

Marcar apenas uma oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very Easy

32. How useful do you think the **As-fast-as-Possible Simulation** mode you used can be? *

Marcar apenas uma oval.

	1	2	3	4	5	
Not	☐	☐	☐	☐	☐	Very Useful

33. How easy was it to read and understand the **trace log** in the end? *

Marcar apenas uma oval.

1 2 3 4 5

Very ☐ ☐ ☐ ☐ ☐ Very Easy

34. After trying the 3 different simulation modes (**Real-time Simulation, Scaled Real-time Simulation and As-fast-as-possible Simulation**), is there any other simulation mode you think would be useful to provide to the user? *

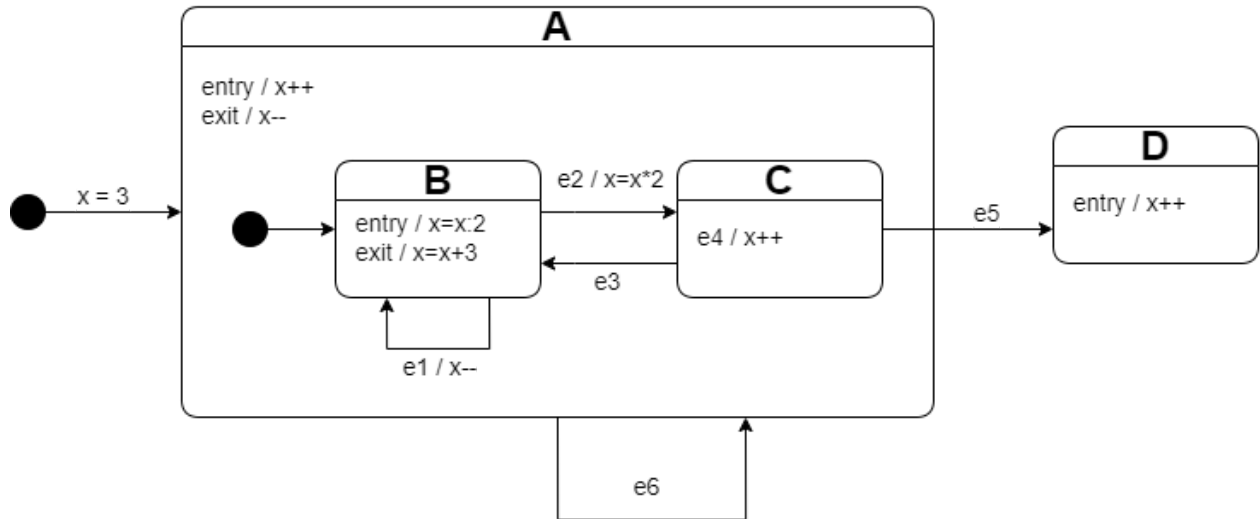
35. Is there any information that isn't on the trace log that you think it may be relevant to be there? *

36. If you wish, please leave any general comment you think may be useful.

Task 5

Now we will observe a different model. Take a look at the model in **Figure 2**. Use the task5.bat to generate the code and start the execution of the model. Please use the tool to answer some questions about the model.

Figure 2. Test Model 2



37. What is the value of x after the occurrence of the event chain e2, e3, e2, e4, e5 ? *

38. What was the value of x during the occurrence of the event chain of the previous question right after event e3 ? *

39. Is there any information that isn't on the trace log that you think it may be relevant to be there? *

40. If you wish, please leave any general comment you think may be useful.
-

System Usability Questionnaire

This section will present some statements regarding the usability of the tool you have just tested. Please answer whether you agree or disagree with these statements.

41. I think that I would like to use this system frequently. *

Marcar apenas uma oval.

12345

StroStrongly agree

42. I found the system unnecessarily complex. *

Marcar apenas uma oval.

12345

StroStrongly agree

43. I thought the system was easy to use. *

Marcar apenas uma oval.

12345

StroStrongly agree

44. I think that I would need the support of a technical person to be able to use this system. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

45. I found the various functions in this system were well integrated. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

46. I thought there was too much inconsistency in this system. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

47. I would imagine that most people would learn to use this system very quickly. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

48. I found the system very cumbersome to use. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

49. I felt very confident using the system. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

50. I needed to learn a lot of things before I could get going with this system. *

Marcar apenas uma oval.

1 2 3 4 5

Stro ☐ ☐ ☐ ☐ ☐ Strongly agree

Task Load Questionnaire

Now please answer a few questions about the work load of the tasks you have performed, from a general point of view.

51. How mentally demanding was the task? *

Marcar apenas uma oval.

1 2 3 4 5 6 7 8 9 10

Very ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Very High

52. How physically demanding was the task? *

Marcar apenas uma oval.

	1	2	3	4	5	6	7	8	9	10	
Very	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	Very High

53. How hurried or rushed was the pace of the task? *

Marcar apenas uma oval.

	1	2	3	4	5	6	7	8	9	10	
Very	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	Very High

54. How successful were you in accomplishing what you were asked to do? *

Marcar apenas uma oval.

	1	2	3	4	5	6	7	8	9	10	
Perf	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	Failure

55. How hard did you have to work to accomplish your level of performance? *

Marcar apenas uma oval.

	1	2	3	4	5	6	7	8	9	10	
Very	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	Very High

56. How insecure, discouraged, irritated, stressed, and annoyed were you? *

Marcar apenas uma oval.

	1	2	3	4	5	6	7	8	9	10	
Very	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	Very High

General Questions about the Tool

Please answer some general questions about the tool you have tested.

57. How helpful do you think this tool can be to help understanding more complex models? *

Marcar apenas uma oval.

	1	2	3	4	5	
Not	☐	☐	☐	☐	☐	Very Helpful

58. How useful do you think this tool can be if it is used for educational purposes? *
(E.g. to help students understand how Statecharts work)

Marcar apenas uma oval.

	1	2	3	4	5	
Not	☐	☐	☐	☐	☐	Very Useful

59. How useful do you think this tool can be if it is used for professional purposes? *
(E.g. to help debugging complex models of real-life systems)

Marcar apenas uma oval.

	1	2	3	4	5	
Not	☐	☐	☐	☐	☐	Very Useful

60. It is planned as future work for this tool to add a UI to the debugger as well as a ★ UI to create **SCCD** models. How much would this tool benefit from these additions?

Marcar apenas uma oval.

1 2 3 4 5

It would benefit a lot ☐ ☐ ☐ ☐ ☐ It would benefit a lot

61. Are there other features you think that could be useful for this tool to have?

62. Leave any comment you may have regarding the terminal interface.

63. If you wish, please leave any general comment you think may be useful.

Google Formulários

